MAANG / FAANG SYSTEM DESIGN MASTERY SUITE (BILINGUAL)

Aptala Anlatır Gibi Sistem Tasarımı

Explain Like I’m 5: MAANG System Design Interview Masterclass

10 Bölüm • 60 Ders • Mimari Şemalar • Java 25 (Loom) • Sıfırdan Staff+ Seviyeye

---

🏛️ Temel Yapı Taşları

L4/L7 LB, Rate Limiting, Cache-Aside, Sharding, Consistent Hashing, LSM-Trees

⚡ Dağıtık Sistemler

Raft Consensus, Kafka Streams, Snowflake IDs, Saga vs 2PC, Circuit Breaker

🎯 MAANG Vaka Çalışmaları

TinyURL, WhatsApp, NewsFeed, YouTube, Uber H3, Stripe Payments & Staff+ Tradeoffs

Yayın Sürümü: 1.0.0 • Java 25 & Modern Cloud Architecture • 2026

Önsöz ve Başlarken / Preface & Getting Started

🇹🇷 Bu Eğitim Hakkında

Bu kitap; Meta, Amazon, Apple, Netflix, Google (MAANG) ve benzeri birinci sınıf teknoloji şirketlerinin Sistem Tasarımı (System Design) mülakatlarına hazırlanan yazılım mühendisleri ve mimarlar için özel olarak hazırlanmıştır.

En karmaşık dağıtık sistem mimarileri (Consistent Hashing, LSM-Trees, Raft Consensus, Kafka Partitions, Gorilla TSDB, H3 Uber Geohashing, Snowflake IDs, Saga Pattern vb.) **bir aptala anlatır gibi** en yalın analojiler, mimari ASCII akış şemaları ve modern **Java 25 (Loom, Virtual Threads, Records, IO)** kodlarıyla ele alınmıştır.

60 derslik bu müfredat, sizi junior bir geliştiriciden milyonlarca eşzamanlı isteği ve petabaytlarca veriyi sakinlikle yöneten bir **Staff+ Principal Architect** zihniyetine ulaştırmayı hedefler.

🇬🇧 About This Masterclass

This masterclass is meticulously engineered for software engineers, tech leads, and aspiring architects preparing for high-stakes System Design rounds at Tier-1 companies (MAANG / FAANG).

We demystify high-concurrency distributed systems—ranging from Consistent Hashing and Raft Consensus to Kafka Event Streams, TSDB compression, and Geo-distributed Ledgers—using crystal-clear **Explain Like I'm 5 (ELI5)** metaphors, architectural schematics, and clean **Java 25** implementations (featuring Project Loom Virtual Threads, pattern matching, and concise records).

Over 60 progressive lessons grouped into 10 structured modules, this curriculum transforms abstract theoretical knowledge into decisive Staff+ architectural leadership.

İçindekiler / Table of Contents

🇹🇷 Bu Eğitim Hakkında 2
🇬🇧 About This Masterclass 2

BÖLÜM 1 / MODULE 1

Temel İlkeler, Ölçeklenebilirlik ve Mülakat Çerçevesi

Core Fundamentals, Scalability & The MAANG Interview Framework

MAANG mülakatlarında başarılı olmanın 4 adımlı altın çerçevesi, dikey/yatay ölçekleme, P99 gecikme metrikleri, SLA/SLO ve CAP/PACELC teoremleri.

The 4-step MAANG interview blueprint, vertical vs horizontal scaling, tail latencies (P99), high availability math (SLA 9s), and CAP/PACELC trade-offs.

1

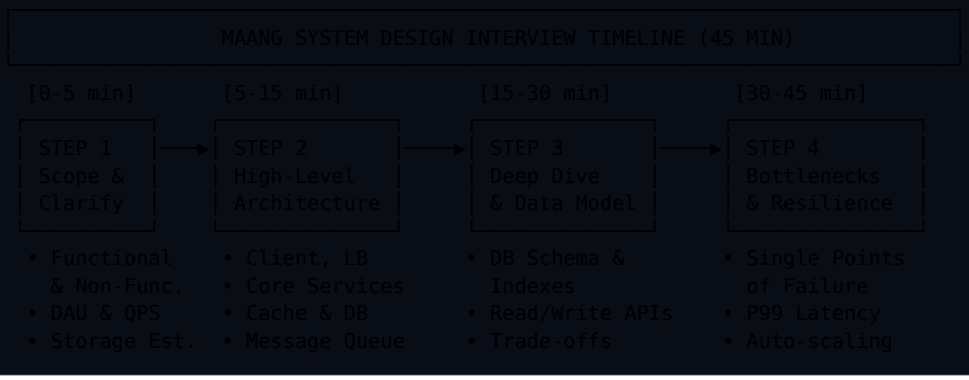
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

4 Adımlı MAANG Mülakat Çerçevesi (RADAR Metodu)

The 4-Step MAANG Interview Blueprint (RADAR Method)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir inşaat mühendisi mülakatında 'Bana bir bina yap' dediklerinde hemen elinize tuğla alıp harç karmaya başlamazsınız! 'Kaç kişilik bir bina? 2 katlı bir yazlık mı, yoksa 80 katlı rüzgara dayanıklı bir gökdelen mi?' diye sorarsınız. MAANG mülakatı da böyledir: Mülakatçı size bilerek muğlak bir soru sorar ('Twitter tasarla'). Hemen kod yazmaya başlarsanız elenirsiniz! Önce gereksinimleri netleştirir (Scope), ardından kaba taslağı çizer (High-Level), kritik parçaların röntgenini çeker (Deep Dive) ve sistemi zorlayıp darboğazları çözersiniz (Bottlenecks)!

🇬🇧 ENGLISH (ELI5)

If a master architect is asked: 'Build me a structure', they don't immediately start mixing cement! They ask: 'Is this a 2-family wooden cabin or an 80-story earthquake-proof skyscraper?' MAANG interviews operate on the exact same premise: interviewers give intentionally vague prompts ('Design Twitter'). If you dive straight into drawing databases, you fail immediately. You must follow the 4 steps: clarify the scope, outline high-level blocks, deep-dive into bottlenecks, and harden failure domains!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

45 dakikalık MAANG sistem tasarımı mülakatının altın zaman planı: 1) Kapsam & Gereksinimler (5 dk): Fonksiyonel (Tweet atma, akış okuma) ve Fonksiyonel Olmayan (100M DAU, P99 < 200ms, 99.99% erişilebilirlik, read-heavy 100:1) hedefler. 2) Yüksek Seviye Mimari (10 dk): İstemci, DNS, CDN, LB, API Gateway, Mikroservisler ve Depolama blokları. 3) Derinlemesine Tasarım (15 dk): Veri şeması, NoSQL vs SQL seçimi, önbellek katmanı ve fan-out stratejisi. 4) İyileştirme & Darboğazlar (15 dk): SPOF (Single Point of Failure), partition stratejisi, rate limiter ve felaket kurtarma.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The rigorous 45-minute MAANG system design framework: 1) Scope & Clarification (5 min): Differentiate functional vs non-functional constraints (100M DAU, Read-to-Write ratio 100:1, P99 < 200ms, 99.99% availability). 2) High-Level Architecture (10 min): Map clients, DNS, CDN, L4/L7 Load Balancers, Gateways, Stateless Services, Cache, and Persistence. 3) Core Deep Dive (15 min): Schema design, data models, sharding keys, and query paths. 4) Bottlenecks & Operational Resiliency (15 min): Mitigate Single Points of Failure (SPOFs), backpressure, and cross-datacenter failover.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Mülakat Metriklerini ve SLA Değerlerini Temsil Eden Record Modeli
public record SystemRequirement(
    String systemName,
    long dailyActiveUsers,
    int readWriteRatio,
    double targetAvailabilityPercentage,
    int targetP99LatencyMillis
) {
    public long calculateDailyReadQps() {
        long dailyWrites = dailyActiveUsers * 2; // Kullanıcı başı 2 yazma
        long dailyReads = dailyWrites * readWriteRatio;
        return dailyReads / 86_400; // Saniyelik ortalama QPS
    }

    public long calculatePeakQps() {
        return calculateDailyReadQps() * 2; // Zirve yük çarpanı: 2x
    }
}

// Java 25 Main metodu ile çalıştırma:
void main() {
    var twitterReq = new SystemRequirement("Twitter Feed", 200_000_000L, 100, 99.99, 150);
    IO.println("Ortalama Okuma QPS: " + twitterReq.calculateDailyReadQps());
    IO.println("Zirve (Peak) Okuma QPS: " + twitterReq.calculatePeakQps());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

En ölümcül mülakat hatası: Soru sorulur sorulmaz sessiz kalıp doğrudan çizim yapmaya başlamaktır. Mülakat bir monolog değil, gelecekteki iş arkadaşınızla yaptığınız mimari bir istişaredir (collaborative dialogue).

🇬🇧 INTERVIEW TRAP & BOTTLENECK

The single most fatal mistake is jumping straight into drawing database boxes in absolute silence. A system design round is an interactive architectural collaboration, not an exam paper.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• İlk 5 dakikayı mutlaka fonksiyonel ve non-fonksiyonel gereksinimleri netleştirmeye ayırın.

• Tasarımı bir kerede mükemmel yapmaya çalışmayın; basitten başlayıp ölçekleyin.

• Her mimari kararın arkasında bir 'Trade-off' (ödünleşim) gerekçesi sunun.

🎯 Key Takeaways for the Interview

• Spend the initial 5 minutes strictly clarifying functional bounds and non-functional SLAs.

• Start with an intuitive baseline architecture, then evolve it under simulated scale.

• Always articulate the trade-offs and rationale behind every database or caching choice.

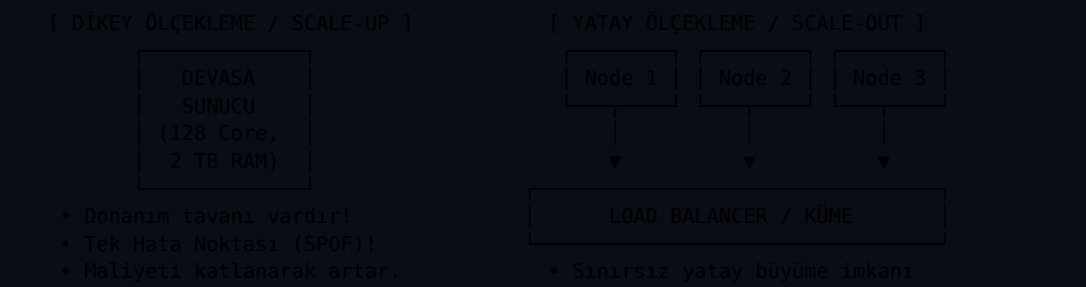
2

BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Dikey vs Yatay Ölçekleme (Scale-Up vs Scale-Out) & Donanım Sınırları

Vertical vs Horizontal Scaling & Hardware Bottlenecks

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Diyelim ki bir pizzacınız var ve siparişlere yetişemiyorsunuz. Dikey ölçekleme (Scale-Up), dükkana dünyanın en kaslı, 8 kollu süper aşçısını transfer etmektir. Bir yere kadar harikadır ama o aşçı da yorulur, hasta olursa dükkan kapanır ve maaşı bir servettir! Yatay ölçekleme (Scale-Out) ise mutfağa standart 10 tane normal aşçı koyup siparişleri aralarında paylaştırmaktır. Aşçılardan biri bayılsa bile diğer 9 aşçı pizza yapmaya devam eder!

🇬🇧 ENGLISH (ELI5)

Imagine a bustling pizzeria running out of oven capacity. Vertical scaling (Scale-Up) is hiring an expensive superhero chef with 8 arms. It works initially, but even superhuman chefs have physical limits, and if they catch the flu, your pizzeria goes bankrupt! Horizontal scaling (Scale-Out) is hiring 10 ordinary cooks and distributing orders across them. If cook 3 twists an ankle, the other 9 keep cooking seamlessly!

🔍 MAANG Seviyesi Teknik Derinlik — 🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dikey ölçekleme (Scale-Up), sunucunun CPU, RAM veya SSD kapasitesini artırmaktır. Avantajı: Kod değişikliği gerektirmez, dağıtık sistem karmaşıklığı yoktur. Dezavantajı: Donanım sınırları (fizik kuralları), üstel maliyet artışı ve Tek Hata Noktası (SPOF). Yatay ölçekleme (Scale-Out), sisteme daha fazla ucuz/standart emtia makine (commodity hardware) eklemektir. Durumsuz (stateless) servisler için kusursuzdur; ancak veritabanlarında veri tutarlılığı, replikasyon ve ağ gecikmesi karmaşıklığı getirir.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Vertical scaling (Scale-Up) augments an existing node with more cores, RAM, and NVMe drives. Pros: Zero distributed networking overhead and seamless operational simplicity. Cons: Hard hardware ceilings, exponential cloud cost curves, and single point of failure (SPOF). Horizontal scaling (Scale-Out) pools clusters of commodity nodes behind load balancers. Trivial for stateless application tiers, but introduces distributed consensus, sharding, and network partition complexities on stateful databases.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Virtual Threads (Project Loom) ile Yatay İşçi Havuzu Simülasyonu
import java.util.concurrent.*;

public class HorizontalWorkerPool {
    public static void main(String[] args) throws InterruptedException {
        // Java 25: Milyonlarca hafif sanal iş parçacığı (Virtual Threads)
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 1; i <= 5; i++) {
                final int workerId = i;
                executor.submit(() -> {
                    IO.println("İşçi Düğüm #" + workerId + " görev yürütüyor: "
                        + Thread.currentThread());
                });
            }
        } // Executor otomatik kapanır ve tüm görevlerin bitmesini bekler
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Veritabanını hemen 100 parçaya bölerim (Sharding)' demeyin! Önce Read-Replica, Cache (Redis) ve Dikey Büyütme alternatiflerini tüketip, sharding'in getirdiği iki fazlı onay (2PC) ve cross-partition join zorluklarını bildiğinizi vurgulayın.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not scream 'Database Sharding!' at minute 10. Sharding introduces massive operational overhead like cross-partition joins and re-balancing. Always exhaust Read Replicas, Caching, and Vertical headroom first.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- Uygulama katmanı (Stateless) daima yatay ölçeklenmelidir.
- Stateless application tiers should always be scaled horizontally behind load balancers.
- Veritabanı katmanında dikey ölçekleme sınırına ulaşılınca Okuma Replikaları ve ardından Sharding devreye girer.
- Scale databases via Read Replicas and caching before resorting to horizontal sharding.
- Yatay ölçekleme hata izolasyonu (fault isolation) sağlayarak sistemi çökmelere karşı korur.
- Horizontal scaling unlocks true fault tolerance by eliminating single points of failure.

3

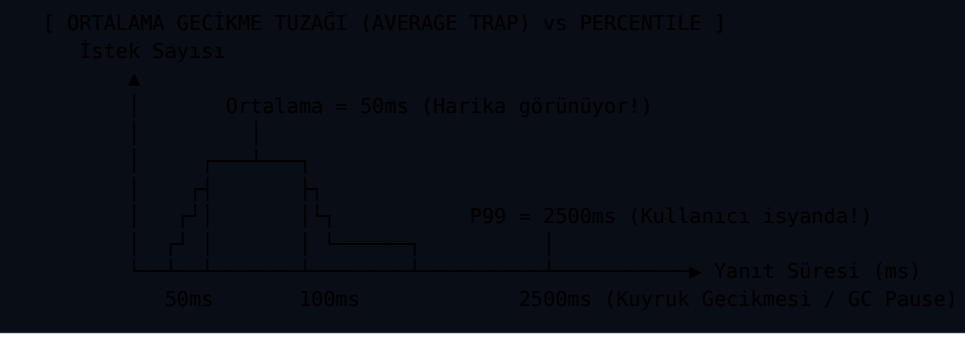
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Gecikme (Latency), Verim (Throughput) ve Ağ Temelleri (P99 Yanılgısı)

Latency, Throughput & Tail Latency (The P99 Fallacy)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir havaalanı pasaport kuyruğunu düşünün. 100 yolcudan 99'u pasaporttan 10 saniyede geçiyor; ancak 1 yolcu şüpheli valiz araması yüzünden 45 dakika bekliyor. Havaalanı müdürü 'Ortalama bekleme süremiz sadece 30 saniye!' derse bu istatistik gerçeği yansıtır mı? O 45 dakika bekleyen yolcu Twitter'da olay çıkarır! MAANG şirketleri ortalamaya (Average) asla inanmaz; P99'a (yani en şanssız %1'lik kullanıcının yaşadığı gecikmeye) bakar!

🇬🇧 ENGLISH (ELI5)

Consider an airport security checkpoint. 99 travelers breeze through in 10 seconds, but 1 unlucky passenger gets held for a 45-minute bag search. If the airport director claims: 'Our average wait time is only 35 seconds!', it masks a nightmare! That stalled passenger will rage on social media. MAANG engineers never track simple averages; they obsess over P99 and P99.9 (the tail latency suffered by the unluckiest 1% of users)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Gecikme (Latency), bir isteğin istemciden çıkıp sunucuda işlenip geri dönene kadar geçen süredir (RTT + processing). Verim (Throughput / QPS), sistemin birim zamanda tamamladığı işlem sayısıdır. Ortalama gecikme (Mean), uzun kuyruk gecikmelerini (Tail Latency), JVM GC duraklamalarını (Stop-the-World) ve disk IO darboğazlarını gizler. Modern bir web sayfasında 100 alt mikroservis çağrısı yapılıyorsa, ortalama bir kullanıcının en az 1 kez P99 gecikmesine çarpma olasılığı: 1 - (0.99)^100 = %63.4'tür! Yani kullanıcıların üçte ikisi yavaşlığa maruz kalır.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Latency is the elapsed duration for a round-trip request (network RTT + queueing + execution). Throughput (QPS/RPS) is system processing capacity per unit of time. The average (mean) latency is dangerously misleading because it masks tail latency spikes caused by garbage collection, lock contention, and TCP packet drops. If a web request aggregates calls across 100 microservices, the probability that a single end-user experiences a P99 tail latency is 1 - (0.99)^100 ≈ 63.4%! Tail latencies dominate user perception.

⚡ Modern Java 25 Uygulaması (Loom & Records) / ⚡ Java 25 Implementation

```java
// Java 25: Percentile (P50, P90, P99) Hesaplayıcı Motoru
import java.util.*;

public class LatencyPercentileTracker {
    private final List<Long> latencies = new ArrayList<>();

    public synchronized void recordLatency(long durationMillis) {
        latencies.add(durationMillis);
    }

    public synchronized long getPercentile(double percentile) {
        if (latencies.isEmpty()) return 0;
        List<Long> sorted = new ArrayList<>(latencies);
        Collections.sort(sorted);
        int index = (int) Math.ceil((percentile / 100.0) * sorted.size()) - 1;
        return sorted.get(Math.max(0, index));
    }
}

void main() {
    var tracker = new LatencyPercentileTracker();
    // 99 hızlı istek (10-30ms), 1 adet felaket yavaş istek (2500ms)
    for (int i = 0; i < 99; i++) tracker.recordLatency(20);
    tracker.recordLatency(2500);

    IO.println("P50 (Medyan): " + tracker.getPercentile(50) + " ms");
    IO.println("P99 (Tail):   " + tracker.getPercentile(99) + " ms");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı / ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçınız 'Sistem yanıt süreleriniz nasıl?' dediğinde 'Ortalama 50ms' deyip geçerseniz Senior seviyesinin altında kalırsınız. 'P50 medyanımız 30ms, P99 kuyruk gecikmemiz ise 120ms altında garanti edilir' şeklinde yanıt verin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Reporting simple average latencies during system design flags you as a junior engineer. Always discuss percentiles: 'Our median P50 is 25ms, while our P99 tail latency is bounded below 150ms.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar / 🎯 Key Takeaways for the Interview

- Ortalama gecikme (Mean), sistemdeki darboğazları ve memnuniyetsiz kullanıcıları gizler.
- Yüzlerce mikroservisin olduğu mimarilerde P99 gecikmesi tüm sistemi domine eder.
- P99'u düşürmek için timeout sınırları, circuit breaker ve spekülatif yeniden denemeler kullanılır.

- Mean latency masks catastrophic tail latency anomalies and user dissatisfaction.
- In fan-out microservice topologies, P99 tail latencies compound exponentially.
- Mitigate tail latencies using strict deadlines, speculative retries, and circuit breakers.

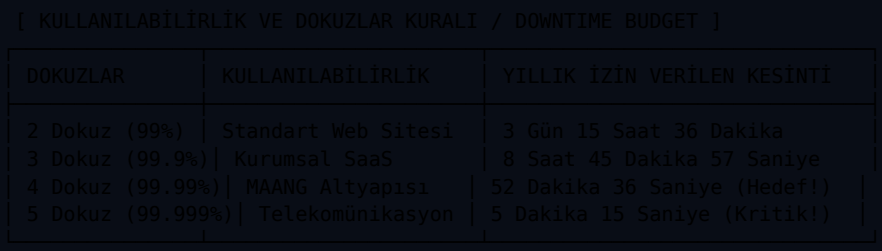
4

BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Güvenilirlik ve Kullanılabilirlik (SLA, SLO, SLI & 9'lar Kuralı)

Reliability & Availability (SLA, SLO, SLI & The Rule of 9s)

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir su tesisatçısı düşünün: 'Haftada bir gün musluktan su akmazsa sorun etmeyin' derse (%85 kullanılabilirlik) o tesisatçıyla çalışmazsınız. Ancak bir hastanede 'Yılda sadece 5 dakika elektrikler kesilebilir' derlerse (%99.999 - 5 dokuz), bu ameliyathanenin jeneratörlerle, çift hatlarla ve kusursuz bir mimariyle korunduğunu gösterir. MAANG mülakatında 'Sistemim %99.99 ayakta kalacak' dediğinizde, yılda sadece 52 dakika hata lüksünüz olduğunu bilerek konuşmalısınız!

🇬🇧 ENGLISH (ELI5)

Imagine a water utility saying: 'If water vanishes for an entire day every month, don't worry about it' (85% availability). You would fire them instantly! But an intensive-care hospital says: 'Power failure cannot exceed 5 minutes over an entire calendar year' (99.999% - Five Nines). That demands redundant generators, dual power grids, and fault-tolerant architecture. When promising 99.99% availability in a MAANG interview, you are budgeting a razor-thin 52 minutes of total annual downtime!

🔍 MAANG Seviyesi Teknik Derinlik — 🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Üç temel güvenilirlik kavramı: 1) SLI (Service Level Indicator): Gerçek zamanlı ölçülen metrik (ör. Başarılı HTTP isteklerinin yüzdesi). 2) SLO (Service Level Objective): Mühendislik ekibinin kendine koyduğu iç hedef (ör. Ayda %99.95 başarı). 3) SLA (Service Level Agreement): Müşteriye verilen yasal ve finansal taahhüt (ör. %99.9'un altına düşerse para iadesi). Hata Bütçesi (Error Budget): SLO'nuz %99.9 ise, %0.1'lik hata bütçeniz vardır. Bu bütçe tükenirse yeni özellik dağıtımları durdurulur ve sadece stabilite çalışılır.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The trifecta of Site Reliability Engineering: 1) SLI (Service Level Indicator): The empirical, real-time metric (e.g., successful request ratio over 30 days). 2) SLO (Service Level Objective): The internal target committed by the engineering team (e.g., 99.95% availability). 3) SLA (Service Level Agreement): The legally binding contractual commitment to paying clients with penalty credits. Error Budget: If your SLO is 99.9%, you have an error budget of 0.1%. When the error budget burns, feature deployments freeze in favor of reliability engineering.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: SLA & Hata Bütçesi (Error Budget) Hesaplama Motoru
public record AvailabilityCalculator(double availabilityPercentage) {
    private static final long SECONDS_IN_YEAR = 365L * 24 * 3600;

    public double calculateAnnualDowntimeSeconds() {
        double failureRate = (100.0 - availabilityPercentage) / 100.0;
        return SECONDS_IN_YEAR * failureRate;
    }

    public String formatDowntime() {
        long totalSecs = (long) calculateAnnualDowntimeSeconds();
        long hours = totalSecs / 3600;
        long mins = (totalSecs % 3600) / 60;
        long secs = totalSecs % 60;
        return hours + " sa " + mins + " dk " + secs + " sn";
    }
}

void main() {
    var fourNines = new AvailabilityCalculator(99.99);
    IO.println("99.99% Yıllık Maksimum Kesinti: " + fourNines.formatDowntime());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçınıza 'Sistemim %100 ayakta kalacak' demeyin! Dağıtık sistemlerde %100 kullanılabilirlik fiziksel olarak imkansızdır (ağ kabloları kopar, veri merkezleri su basar). %99.99 (Four Nines) gerçekçi kurumsal altın standarttır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim 100% availability in a system design interview. 100% is mathematically impossible across distributed networks. Stating 99.99% demonstrates architectural maturity.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- SLI neyi ölçtüğünüz, SLO hedefiniz, SLA ise müşteriye verdiğiniz yasal sözdür.
- 99.99% (Dört Dokuz) yılda 52 dakikadan az kesinti demektir.
- Hata bütçesi (Error Budget), inovasyon hızı ile sistem stabilitesi arasındaki dengeyi sağlar.

- SLI is the metric, SLO is the internal goal, SLA is the contractual penalty threshold.
- Four Nines (99.99%) permits less than 52.6 minutes of downtime per year.
- Error budgets balance release velocity against platform stability.

5

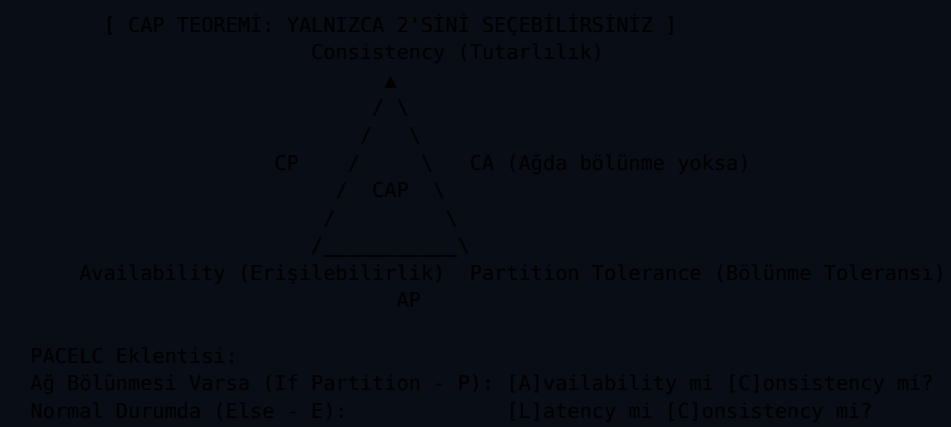
BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

CAP Teoremi ve PACELC: Ağ Bölünmesinde Tutarlılık mı, Erişilebilirlik mi?

CAP Theorem & PACELC: Consistency vs Availability

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bankanın Ankara ve İstanbul şubelerini düşünün. İki şube arasındaki telefon ve internet kablosu bir kaza sonucu koptu (Ağ Bölünmesi - Partition). Şimdi Ankara şubesine bir müşteri gelip hesabındaki son 1.000 TL'yi çekmek istiyor. Banka ne yapmalı? 1) 'İnternet koptu, işlem yapamam' deyip müşteriyi geri çevirirse Tutarlılığı (Consistency - CP) seçmiş olur. 2) 'Parayı verelim gitsin, kablo bağlanınca bakarız' derse Erişilebilirliği (Availability - AP) seçmiş olur ama müşteri aynı parayı İstanbul'dan da çekmiş olabilir! İkisini aynı anda garanti edemezsiniz!

🇬🇧 ENGLISH (ELI5)

Imagine two bank branches in New York and London. A submarine cable snaps, severing their communication line (Network Partition). A customer walks into London wanting to withdraw their last $1,000. What does the bank do? 1) Refuse the withdrawal until connection restores: chooses Consistency over Availability (CP). 2) Hand over the cash: chooses Availability over Consistency (AP), risking that the customer simultaneously withdrew the same $1,000 in New York! You cannot have both!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Eric Brewer'ın CAP Teoremi: Dağıtık bir veri deposu Consistency (Lineerize edilebilir anlık tutarlılık), Availability (Her sağlıklı düğümün hata yerine yanıt dönmesi) ve Partition Tolerance (Ağ paketleri kaybolsa da sistemin çalışması) özelliklerinden aynı anda ancak ikisini sağlayabilir. Dağıtık ağlarda kablo kopmaları ve gecikmeler kaçınılmaz olduğundan 'P' zorunludur; dolayısıyla gerçek dünya seçimi CP veya AP arasındadır. Daniel Abadi'nin PACELC teoremi bunu genişletir: Ağ bölünmesi yokken bile sistem Düşük Gecikme (Latency) mi yoksa Güçlü Tutarlılık (Consistency) mi hedefliyor? (Örn: MongoDB = PC/EC, Cassandra = PA/EL).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Brewer's CAP Theorem proves that a distributed data store can guarantee at most two of: Consistency (linearizability), Availability (every non-failing node returns a non-error response), and Partition Tolerance (network packet drops/splits). Since network partitions are an inevitable physical reality, 'P' is non-negotiable, boiling trade-offs down to CP vs AP. Abadi's PACELC expands this: If partitioned (P), trade Availability (A) vs Consistency (C); Else (E), trade Latency (L) vs Consistency (C). Example: Dynamo/Cassandra = PA/EL, HBase/Spanner = PC/EC.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: CAP/PACELC Modeli ve Quorum Tutarlılık Kontrolcüsü
public class QuorumConsistencyChecker {
    // R (Okuma Düğümü), W (Yazma Düğümü), N (Toplam Replika)
    // Güçlü Tutarlılık Kuralı: R + W > N
    public static boolean isStronglyConsistent(int totalNodes, int writeQuorum, int readQuorum) {
        return (writeQuorum + readQuorum) > totalNodes;
    }

    public static void main(String[] args) {
        int N = 3; // 3 Replikasyonlu Küme
        int W = 2; // Çoğunluk yazma (Quorum Write)
        int R = 2; // Çoğunluk okuma (Quorum Read)

        boolean strong = isStronglyConsistent(N, W, R);
        IO.println("R=" + R + ", W=" + W + ", N=" + N + " -> Güçlü Tutarlı mı? " + strong);
        // R + W = 4 > 3 -> En az 1 düğüm hem en son yazmayı hem okumayı görür!
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçılar 'Ben hem %100 tutarlı hem de her durumda anında cevap veren bir sistem istiyorum' diyerek sizi tuzağa düşürmeye çalışır. Tuzağa düşmeyin: Ağ bölünmesinde CAP gereği birinden ödün vermek zorunda olduğunuzu netçe ifade edin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Interviewers often bait candidates by asking for 100% consistency AND 100% availability during network partitions. Stand firm: explain that network splits make this mathematically impossible under CAP.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Gerçek dünyada ağ bölünmeleri (P) kaçınılmazdır; seçim CP veya AP arasındadır.

• Finansal ve bakiye sistemleri CP'yi, sosyal medya akışları ve beğeniler AP'yi tercih eder.

• PACELC teoremi, sistemin normal zamanlardaki Latency vs Consistency takasını açıklar.

🎯 Key Takeaways for the Interview

• Network partitions (P) are unavoidable; real architectures choose between CP and AP.

• Financial ledger ledgers favor CP; social media feeds and likes favor AP.

• PACELC clarifies the everyday trade-off between latency and consistency when no partition exists.

6

BÖLÜM: TEMEL İLKELER, ÖLÇEKLENEBILIRLIK VE MÜLAKAT ÇERÇEVESI | MODULE: CORE FUNDAMENTALS, SCALABILITY & THE MAANG INTERVIEW FRAMEWORK

Arka Plan Tahminleri (Back-of-the-Envelope Estimation): QPS, RAM ve Disk

Back-of-the-Envelope Capacity Estimations (QPS, RAM, Storage)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ MAANG KAPASİTE TAHMİNİ MATRİSİ (CHEAT SHEET) ]

+-----------------------------+-----------------------------------+
| 1 Gün = 86,400 Saniye       | Kolay hesap için ~100,000 (10^5) saniye al! |
| 1 Milyon İstek / Gün        | ~12 QPS                           |
| 100 Milyon İstek / Gün      | ~1,200 QPS (Zirve: ~2,400 QPS)    |
| 1 Milyar İstek / Gün        | ~12,000 QPS                       |
+-----------------------------+-----------------------------------+
| VERİ BOYUTLARI              | 1 Char = 1 Byte | UUID = 16-36 B | Long = 8B |
| 100M tweet x 200 Byte       | 20 GB / gün -> 5 Yılda: ~36 TB depolama |
| %20 Önbellek Kuralı         | Günlük okuma verisinin %20'si RAM'de tutulur |
+-----------------------------+-----------------------------------+
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir düğün organize ettiğinizi düşünün. Kaç gram et, kaç bardak su almanız gerektiğini hesaplamazsanız ya düğünün ortasında yemek biter rezil olursunuz ya da tonlarca et çürür paranız batar! Mülakatta da 'Twitter tasarlıyorum' dediğinizde mülakatçı kaç sunucuya, kaç terabayt diske ve ne kadar RAM'e ihtiyacınız olduğunu bilmek ister. Kimse sizden kuruşu kuruşuna kesin matematik beklemez; kabaca peçete arkasında yapılan (Back-of-the-envelope) mantıklı mühendislik tahminleri bekler!

🇬🇧 ENGLISH (ELI5)

Planning a wedding without estimating food means either guests starve midway through and riot, or you overbuy tons of meat that rots and bankrupts you! In a MAANG interview, if you say 'I will store all messages in memory', the interviewer will ask: 'How many gigabytes is that?' Nobody expects 5-decimal calculus; they expect rapid, disciplined back-of-the-envelope napkin math to sanity-check architectural feasibility!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında kapasite hesaplama matematiği: 1) Zaman Sabiti: 1 gün = 86,400 saniye ≈ 10^5 sn. 2) QPS Hesabı: 100M günlük istek / 100,000 sn ≈ 1,000 QPS (Zirve yük için 2x çarpanı alınır: 2,000 QPS). 3) Depolama Hesabı: Her kayıt 500 Byte ise, günlük veri = 100M x 500B = 50 GB/gün. 5 yıllık depolama = 50 GB x 365 x 5 ≈ 91 TB (3 kopya replikasyon ile 270 TB). 4) Önbellek RAM Hesabı (Pareto 80/20 Kuralı): İsteklerin %80'i verinin %20'sinden gelir. Günlük 50 GB verinin %20'si = 10 GB RAM önbellek yeterlidir.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Back-of-the-envelope capacity arithmetic: 1) Time approximation: 1 day = 86,400s ≈ 10^5s. 2) QPS: 100M requests/day ÷ 10^5s ≈ 1,000 QPS average (multiply by 2-3x for peak traffic = 2,500 QPS). 3) Storage: If an entity payload is 500 Bytes, 100M items/day = 50 GB/day. 5-year retention: 50 GB × 365 × 5 ≈ 91.25 TB (with 3x replication factor = 273 TB). 4) Memory Sizing (Pareto 80/20 Rule): 80% of read traffic targets 20% of data. Cache 20% of daily read volume: 50 GB × 0.20 = 10 GB Redis memory footprint.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Peçete Arkası Kapasite Tahmin Motoru (Capacity Estimator)
public record CapacityEstimator(long dailyRequests, int payloadBytes) {
    public long averageQps() {
        return dailyRequests / 86_400;
    }

    public long peakQps() {
        return averageQps() * 2;
    }

    public double dailyStorageGigabytes() {
        return (dailyRequests * (double) payloadBytes) / (1024 * 1024 * 1024);
    }

    public double fiveYearStorageTerabytes(int replicationFactor) {
        return (dailyStorageGigabytes() * 365 * 5 * replicationFactor) / 1024;
    }

    public double cacheMemoryGigabytes(double paretoRatio) {
        return dailyStorageGigabytes() * paretoRatio;
    }
}

void main() {
    var estimator = new CapacityEstimator(100_000_000L, 500);
    IO.println("Ortalama QPS:    " + estimator.averageQps());
    IO.println("Zirve (Peak) QPS: " + estimator.peakQps());
    IO.println("Günlük Depolama: " + String.format("%.2f GB", estimator.dailyStorageGigabytes()));
    IO.println("5 Yıllık (3x Rep):" + String.format("%.2f TB",
estimator.fiveYearStorageTerabytes(3)));
    IO.println("RAM Önbellek:    " + String.format("%.2f GB", estimator.cacheMemoryGigabytes(0.20)));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Hesaplama yaparken 86,400 ile tam bölmeye çalışıp vakit kaybetmeyin. Mülakatçıya 'Hesabı kolaylaştırmak için 1 günü 100.000 saniye kabul ediyorum' deyin; bu pratik mühendislik refleksiniz için artı puan kazandırır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not waste precious interview minutes dividing by 86,400 by hand. Explicitly tell the interviewer: 'I will approximate 86,400 as 100,000 seconds for rough sizing'; they will appreciate your pragmatism.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- 1 gün ≈ 100,000 saniye kuralı QPS hesaplarını saniyeler içinde çözmenizi sağlar.
- Zirve (Peak) QPS her zaman ortalama QPS'in 2 ila 3 katı olarak kabul edilir.
- Pareto 80/20 kuralı sayesinde günlük verinin %20'sini önbelleğe almak yeterlidir.

- Use 1 day ≈ 10^5 seconds to compute baseline QPS within 5 seconds.
- Dimension service throughput for Peak QPS (typically 2x to 3x average QPS).
- The Pareto 80/20 heuristic sizes distributed cache memory at 20% of daily active working set.

BÖLÜM 2 / MODULE 2

Yük Dengeleme, Ağ Protokolleri ve API Ağ Geçitleri

Load Balancing, Networking Protocols & API Gateways

Trafiği akıllıca yönlendirme sanatı: Anycast DNS, L4 vs L7 Load Balancer, gRPC vs WebSockets, API Gateway deseni ve Dağıtık Rate Limiting algoritmaları.

The art of traffic routing: Anycast DNS, Layer 4 vs Layer 7 load balancing, gRPC vs WebSockets, API Gateway patterns, and Distributed Rate Limiting algorithms.

7

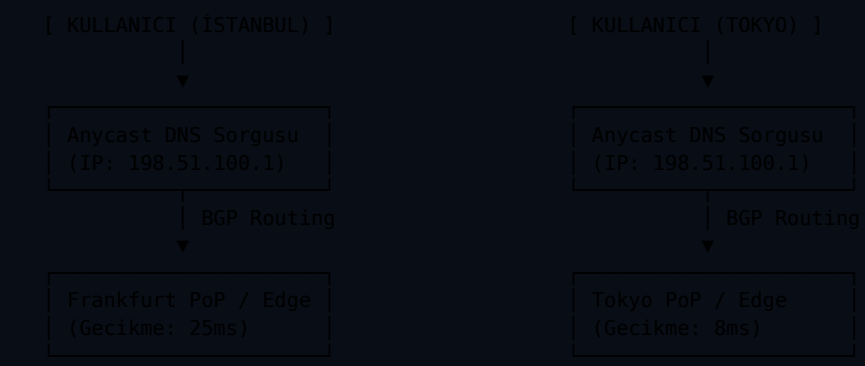
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

DNS ve Coğrafi Yönlendirme: Anycast, Geo-DNS ve Latency-Based Routing

DNS & Geo-Routing: Anycast, GeoDNS & Latency Routing

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Dünyanın neresinde olursanız olun acil ambulans çağırmak için tek bir numara (örneğin 112) çevirirsiniz. Ama İstanbul'dan aradığınızda telefonunuz Tokyo'daki hastaneye değil, sokağınızın başındaki istasyona bağlanır! İşte `Anycast DNS` tam olarak budur: Bütün dünyaya tek bir IP adresi ilan edersiniz (BGP yönlendirme ile); internet yönlendiricileri kullanıcının isteğini ona ışık hızıyla en yakın olan veri merkezine teslim eder!

🇬🇧 ENGLISH (ELI5)

No matter what neighborhood you live in, dialing an emergency number routes you to the closest ambulance garage, not one across the Pacific Ocean! That is `Anycast BGP Routing`. Multiple servers across the globe announce the exact same IP address. Autonomous internet routers automatically steer a user's packets along the shortest physical BGP path to their nearest edge Point of Presence (PoP)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Geleneksel Unicast mimarisinde her sunucu tekil bir IP adresine sahiptir. Anycast'te ise dünyanın dört bir yanındaki onlarca veri merkezi BGP (Border Gateway Protocol) üzerinden aynı IP bloğunu anons eder. İstemci DNS sorgusu attığında en yakın PoP (Point of Presence) yanıt verir. Geo-DNS ise istemcinin IP adresine bakarak coğrafi konuma özel farklı IP'ler döner (örneğin ABD kullanıcısına us-east, Avrupa kullanıcısına eu-west IP'si). Latency-Based Routing ise AWS Route 53 gibi sistemlerin küresel ağ gecikmelerini sürekli ölçerek en düşük RTT sunan bölgeye yönlendirmesidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Unicast binds an IP address uniquely to one machine. In Anycast, distributed PoPs globally announce identical IP prefixes via BGP. Routers forward packets toward the topologically nearest network hop. GeoDNS inspects client recursive resolver IPs to synthesize geographically tailored responses. Latency-Based Routing (e.g. AWS Route 53) continuously probes edge-to-region RTTs to resolve DNS to the lowest-latency origin datacenter.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Simüle Edilmiş Coğrafi DNS Çözümleyici (GeoDNS Resolver)
import java.util.Map;

public record GeoDnsResolver(Map<String, String> regionDatacenters, String defaultDatacenter) {
    public String resolveTargetIp(String clientCountryCode) {
        return regionDatacenters.getOrDefault(clientCountryCode, defaultDatacenter);
    }
}

void main() {
    var geoDns = new GeoDnsResolver(Map.of(
        "TR", "195.175.25.10 (Frankfurt Edge)",
        "JP", "210.140.10.20 (Tokyo Edge)",
        "US", "198.51.100.5 (Virginia Edge)"
    ), "198.51.100.5 (Default US-East)");

    IO.println("Türkiye'den gelen istek -> " + geoDns.resolveTargetIp("TR"));
    IO.println("Japonya'dan gelen istek -> " + geoDns.resolveTargetIp("JP"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

DNS tabanlı yönlendirmede DNS önbellekleme (TTL - Time To Live) riskini unutmayın! TTL 300 saniye (5 dakika) ise, bir veri merkezi çöktüğünde kullanıcıların DNS önbelleği 5 dakika boyunca ölü sunucuya gitmeye devam eder. Bu yüzden DNS tek başına anlık failover için yetersizdir; önüne Anycast IP konmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

DNS TTL caching delays failover! If TTL is 300 seconds, clients continue sending traffic to a crashed datacenter for up to 5 minutes. Never rely solely on DNS for sub-second failover; front it with Anycast BGP routing.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Anycast, DDoS saldırılarını küresel PoP noktalarına dağıtarak soğurur.

• Geo-DNS ve Latency-Based Routing kullanıcıyı fiziksel olarak en yakın merkeze bağlar.

• Düşük TTL failover süresini kısaltır ancak kök DNS sunucularına binen sorgu yükünü artırır.

🎯 Key Takeaways for the Interview

• Anycast absorbs multi-terabit DDoS traffic by dispersing packets across dozens of edge PoPs.

• GeoDNS and latency routing minimize round-trip times by binding users to adjacent regions.

• Lowering DNS TTL accelerates disaster failover at the expense of higher DNS query load.

8

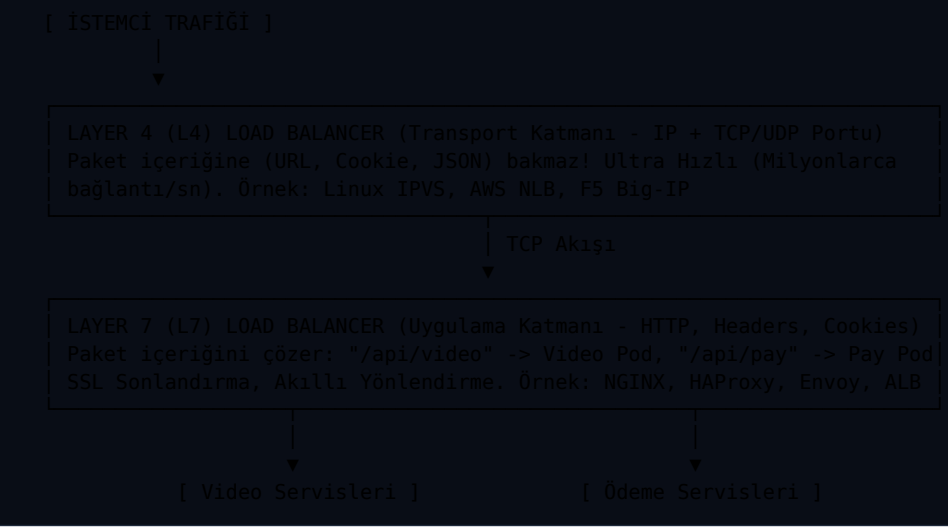
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Yük Dengeleyiciler (L4 vs L7): TCP vs HTTP, NGINX, HAProxy, AWS ALB

Load Balancers (L4 vs L7): TCP vs HTTP, NGINX, HAProxy, AWS ALB

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

L4 ve L7 yük dengeleyiciyi bir postanenin iki farklı görevlisi gibi hayal edin: L4 görevlisi, sadece zarfın üzerindeki sokak numarasına bakar (IP ve Port). Zarfın içini ASLA açmaz; paketi gözü kapalı ışık hızında doğru binaya fırlatır! L7 görevlisi ise zarfı açar, içindeki mektubu okur (HTTP URL'i, Cookie, Başlıklar). 'Aaa bu bir fatura ödemesiymiş, bunu muhasebe masasına götüreyim; bu bir video talebiymiş, bunu sinema salonuna göndereyim' der. L4 süper hızlıdır, L7 ise süper zekidir!

🇬🇧 ENGLISH (ELI5)

Think of L4 and L7 load balancers as two mail sorting clerks: The L4 clerk only glances at the building address on the outside envelope (IP and TCP Port). They NEVER open the envelope; they fling packets at blistering wire speed! The L7 clerk tears the envelope open, reads the letter (HTTP Headers, Cookies, URL paths), and decides: 'This letter says /video, send it to the media cluster; this says /checkout, route to the payment gateway.' L4 offers raw throughput; L7 delivers smart content routing!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

OSI 4. Katman (L4), Taşıma katmanında (TCP/UDP) çalışır. NAT (Network Address Translation) veya DSR (Direct Server Return) kullanır. HTTP paketini çözmediği (SSL terminasyonu yapmadığı) için CPU maliyeti çok düşüktür; saniyede milyonlarca eşzamanlı TCP bağlantısını yönetebilir. OSI 7. Katman (L7), Uygulama katmanında çalışır. TLS/SSL el sıkışmasını sonlandırır (SSL Termination), HTTP istek başlıklarını, URI yolunu (`/users` vs `/orders`), JWT tokenlarını ve Cookie oturumlarını inceler. Bu sayede mikroservis yönlendirmesi, A/B testi ve rate limiting yapabilir; ancak CPU ve bellek maliyeti daha yüksektir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Layer 4 (L4) operates on TCP/UDP transport segments using NAT or Direct Server Return (DSR). Because it avoids terminating TLS or parsing HTTP buffers, L4 introduces microsecond latency and scales to tens of millions of concurrent connections (AWS NLB, Maglev). Layer 7 (L7) operates on application-level HTTP/HTTPS semantics. It terminates TLS, parses headers, inspects cookies, and routes requests based on URI paths (`/stream` vs `/auth`). L7 enables canary routing, sticky sessions, and rate limiting at higher CPU overhead.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: L7 İçerik Tabanlı Yönlendirme Simülasyonu (Path-Based Routing)
public class Layer7Router {
    public record HttpRequest(String path, String method, String clientIp) {}

    public String routeRequest(HttpRequest req) {
        // Java 25 Pattern Matching ile L7 URL Yönlendirmesi:
        return switch (req.path()) {
            case String p when p.startsWith("/api/v1/auth")     -> "AUTH_SERVICE_CLUSTER";
            case String p when p.startsWith("/api/v1/payments") -> "PAYMENT_PCI_CLUSTER";
            case String p when p.startsWith("/static/")         -> "CDN_S3_STORAGE";
            default                                             -> "CORE_BACKEND_CLUSTER";
        };
    }
}

void main() {
    var router = new Layer7Router();
    var payReq = new Layer7Router.HttpRequest("/api/v1/payments/charge", "POST", "10.0.0.1");
    IO.println("İstek Hedefi: " + router.routeRequest(payReq));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Tüm trafiği doğrudan tek bir NGINX L7 load balancer'a bağlarım' demeyin! Yüksek ölçekte (milyonlarca QPS) mimari standart şöyledir: Anycast DNS -> L4 Load Balancer (AWS NLB / Maglev) -> L7 Load Balancer Filosu (NGINX / Envoy) -> Mikroservisler.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never funnel millions of raw internet connections directly into a single L7 NGINX node. The enterprise standard is a two-tier hierarchy: Anycast DNS -> L4 NLB cluster -> Fleet of autoscaled L7 Envoy/NGINX proxies.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• L4 paket içeriğine bakmaz, ultra yüksek verim (throughput) ve düşük gecikme sunar.

• L7 HTTP başlıklarına ve URL'e göre akıllı mikroservis yönlendirmesi yapar.

• Modern mimarilerde L4 önde donanım kalkanı, L7 arkada uygulama beyni olarak birlikte kullanılır.

🎯 Key Takeaways for the Interview

• L4 routes on IP:Port tuples, offering extreme connection throughput and sub-millisecond latencies.

• L7 inspects HTTP/gRPC semantics for intelligent path-based and header-based routing.

• Two-tier load balancing places L4 at the edge fronting a dynamic fleet of L7 proxies.

9

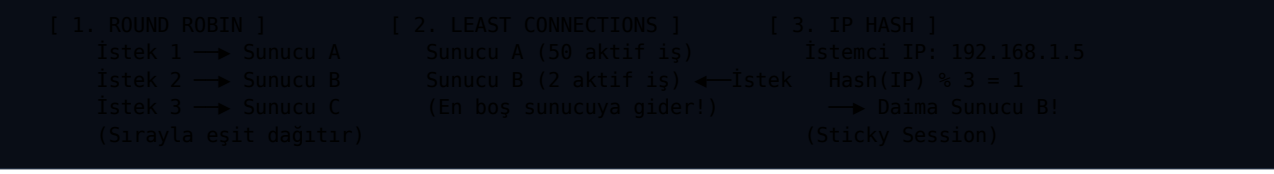
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Yük Dengeleme Algoritmaları: Round Robin, Least Connections, Weighted ve IP Hash

Load Balancing Algorithms: Round Robin, Least Conn, Weighted & IP Hash

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir bankada 3 gişe memuru olduğunu düşünün. Round Robin, sıra numarasını sırayla 1-2-3-1-2-3 diye memurlara dağıtan robot gibidir; ancak 1. memurun önündeki müşteri 1 saat sürecek devasa bir miras işlemi yapıyorsa o gişenin önü kilitlenir! Least Connections ise akıllı bir şeftir: Kafasını kaldırır, 'Hanginizin önünde daha az bekleyen insan var?' diye bakar ve yeni müşteriyi en boş memura gönderir. IP Hash ise 'Ali Bey sen daima 2 numaralı memura git' diyen kuraldır (aynı kullanıcı hep aynı sunucuya gider)!

🇬🇧 ENGLISH (ELI5)

Picture 3 bank tellers. Round Robin is a naive ticket dispenser handing turns sequentially: 1, 2, 3, 1, 2, 3. But if Teller 1 is stuck in a 2-hour corporate audit, their desk clogs completely while Teller 2 sits bored! Least Connections is a vigilant floor manager who checks: 'Who currently has the fewest active clients?' and dispatches the newcomer there. IP Hash calculates a hash of the customer's ID, consistently assigning that specific person to the exact same teller every visit (Session Stickiness)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Yük dengeleme algoritmaları ikiye ayrılır: 1) Statik Algoritmalar: Sunucu yükünü umursamaz. Round Robin (sırayla), Weighted Round Robin (güçlü sunucuya daha fazla ağırlık, ör. 8 core makineye 4x istek), IP Hash (istemci IP'sine göre yapışkan oturum - Sticky Session). 2) Dinamik Algoritmalar: Gerçek zamanlı sunucu metriğini izler. Least Connections (aktif TCP bağlantısı en az olan sunucuya gönderir - uzun süren işlemler ve WebSockets için idealdir), Weighted Least Connections, Least Response Time (en hızlı yanıt veren sunucuyu tercih eder).

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Balancing algorithms categorize into: 1) Static Schemes: Agnostic to real-time server load. Round Robin (simple cycle), Weighted Round Robin (proportional weights assigned to heftier bare-metal nodes), IP/Consistent Hashing (binds client IP to persistent backend instances for stateful caching). 2) Dynamic Schemes: Real-time telemetry feedback. Least Connections (routes to node with minimum active sockets —vital for persistent WebSockets and heavy database queries), Weighted Least Connections, and Least Response Time (prioritizes instances demonstrating lowest TTFB).

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Thread-Safe Weighted Round Robin & Least Connections Simülatörü
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class LoadBalancingSim {
    public record BackendNode(String id, int weight, AtomicInteger activeConnections) {}

    // 1. Round Robin
    public static class RoundRobinBalancer {
        private final List<BackendNode> nodes;
        private final AtomicInteger counter = new AtomicInteger(0);

        public RoundRobinBalancer(List<BackendNode> nodes) { this.nodes = nodes; }

        public BackendNode getNextNode() {
            int idx = Math.abs(counter.getAndIncrement() % nodes.size());
            return nodes.get(idx);
        }
    }

    // 2. Least Connections
    public static BackendNode selectLeastConnection(List<BackendNode> nodes) {
        return nodes.stream()
                .min((a, b) -> Integer.compare(a.activeConnections().get(),
b.activeConnections().get()))
                .orElseThrow();
    }
}

void main() {
    var n1 = new LoadBalancingSim.BackendNode("Node-A", 1, new AtomicInteger(42));
    var n2 = new LoadBalancingSim.BackendNode("Node-B", 1, new AtomicInteger(5));
    var n3 = new LoadBalancingSim.BackendNode("Node-C", 1, new AtomicInteger(18));

    var best = LoadBalancingSim.selectLeastConnection(List.of(n1, n2, n3));
    IO.println("Least Connection Seçimi: " + best.id() + " (Aktif: " + best.activeConnections() +
")");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

IP Hash ile oturum sabitleme (Sticky Session) dağıtık mimarilerde bir anti-pattern'dir! Bir sunucu çöktüğünde o sunucuya bağlı tüm kullanıcıların oturumu uçar ve yük dengesizleşir. Modern MAANG mimarilerinde sunucular tamamen durumsuz (Stateless) tutulur, oturumlar Redis'te saklanır ve Least Connections kullanılır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Sticky Sessions via IP Hash represent an architectural anti-pattern in cloud microservices. If an instance restarts, connected users lose state. Keep backends strictly stateless, store tokens in Redis, and leverage Least Connections.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Kısa süren tipik REST istekleri için Weighted Round Robin idealdir.

• Uzun süren veritabanı veya WebSocket bağlantıları için Least Connections şarttır.

• Durumsuz (stateless) tasarım sayesinde sunucu çökmeleri istemciyi etkilemez.

🎯 Key Takeaways for the Interview

• Weighted Round Robin excels for short-lived, uniform HTTP REST transactions.

• Least Connections is critical for long-lived WebSockets and compute-heavy transactions.

• Stateless servers combined with external token stores render sticky sessions obsolete.

10

BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Ağ İletişim Protokolleri: HTTP/1.1 vs HTTP/2 vs HTTP/3 QUIC, WebSockets, gRPC, SSE

Network Protocols: HTTP/1.1 vs HTTP/2 vs HTTP/3, WebSockets, gRPC, SSE

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram

```
[ PROTOKOL EVRİMİ ]
HTTP/1.1 : Tek TCP bağlantısında tek istek (Head-of-Line Blocking!)
HTTP/2   : Tek TCP üzerinde Multiplexing (Binary Framing, Header Compression)
HTTP/3   : UDP Tabanlı QUIC! (Sıfır TCP Head-of-Line, 0-RTT El Sıkışma!)

[ İKİ YÖNLÜ vs TEK YÖNLÜ İLETİŞİM ]
WebSockets : Tam İki Yönlü (Full-Duplex) Sürekli TCP Kanalı (Chat, Finans)
SSE (Event): Sunucudan İstemciye Tek Yönlü Canlı Veri Akışı (Borsa, Skor)
gRPC       : HTTP/2 Üzerinde Protobuf ile Ultra Hızlı Mikroservis İletişimi
```

💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

HTTP/1.1 tek şeritli dar bir otoyola benzer; öndeki kamyon arıza yaparsa arkadaki 100 araba bekler (Head-of-Line Blocking). HTTP/2 bu yola 10 şerit ekler; aynı kablodan aynı anda onlarca veri akar. HTTP/3 (QUIC) ise asfaltı kaldırıp manyetik raylı trene geçmektir: TCP'nin hantal onay mekanizmalarını atar, UDP üzerinden sıfır gecikmeyle uçar! WebSockets iki kişinin telefonla kesintisiz konuşmasıdır; SSE ise televizyondaki canlı haber bültenini tek yönlü dinlemektir!

🇬🇧 ENGLISH (ELI5)

HTTP/1.1 is a single-lane mountain road: if a lumber truck stalls, all 50 cars behind it freeze (Head-of-Line Blocking). HTTP/2 widens it into a 10-lane superhighway, multiplexing dozens of concurrent streams over one TCP pipe. HTTP/3 (QUIC) throws away asphalt entirely: it swaps fragile TCP for lightning-fast UDP with zero-RTT handshakes! WebSockets is an open telephone call (full-duplex chat); Server-Sent Events (SSE) is tuning into an emergency FM radio station (unidirectional live streaming)!

🔍 MAANG Seviyesi Teknik Derinlik — 🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) HTTP/1.1: Metin tabanlıdır; her istek için yeni TCP veya keep-alive gerektirir. 2) HTTP/2: Binary framing katmanı getirir. Tek TCP bağlantısında stream multiplexing ve HPACK başlık sıkıştırması yapar; ancak paket kaybında TCP seviyesinde HoL blocking yaşar. 3) HTTP/3 (QUIC): Google'ın geliştirdiği UDP tabanlı protokoldür. Her akış (stream) birbirinden tamamen bağımsızdır; bir paket kaybolsa bile diğer akışlar etkilenmez. 4) gRPC: Google'ın HTTP/2 üzerinde Protocol Buffers (ikili serileştirme) kullanan, JSON'dan 7-10 kat daha hızlı ve tip güvenli RPC protokolüdür; iç mikroservis haberleşmesinin standardıdır.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) HTTP/1.1: Text-oriented with head-of-line blocking. 2) HTTP/2: Binary framing layer enabling multiplexed bidirectional streams over a shared TCP connection with HPACK header compression; still suffers TCP-level HoL blocking on packet drops. 3) HTTP/3 (QUIC): Replaces TCP with UDP-based transport, eliminating multi-stream HOL blocking and slashing handshakes to 0-RTT connection resumptions across mobile cell towers. 4) gRPC: High-throughput RPC framework using HTTP/2 multiplexing and Protocol Buffers serialization; de facto industry standard for microservice-to-microservice internal transit.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: gRPC vs REST Veri Serileştirme Karşılaştırma Modeli
// Protocol Buffers Mantığı (Java 25 Record ile Temsil Edilmiş Binary Kompakt Yapı)
public record UserProfileProto(long userId, String username, String email, int karma) {
    // JSON Boyutu: ~95 Byte  |  Protobuf İkili Boyutu: ~28 Byte (3.4 kat daha küçük!)
    public byte[] toMockProtobufBytes() {
        // İkili serileştirme simülasyonu: Ağ trafiğini ve serileştirme CPU yükünü minimize eder
        return ("B:" + userId + ":" + karma).getBytes();
    }
}

void main() {
    var user = new UserProfileProto(1049281L, "aliturgut", "ali@example.com", 942);
    IO.println("Simüle Protobuf Boyutu: " + user.toMockProtobufBytes().length + " bytes");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'İç mikroservisler arasında da JSON REST kullanırım' demeyin. Dış dünyadan istemciler için JSON REST uygundur; ancak iç mikroservisler arasında yüksek verim, düşük CPU tüketimi ve tip güvenliği için gRPC (Protobuf) tercih edilmelidir.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Don't default to JSON REST across internal microservices. Use REST at the perimeter API Gateway, but standardize on gRPC over HTTP/2 internally for compact binary footprints and code-generated contracts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

- HTTP/3 QUIC mobil cihazların baz istasyonu değişiminde bile bağlantıyı koparmaz.
- gRPC ikili Protocol Buffers formatı ile JSON'a kıyasla ağ ve CPU tasarrufu sağlar.
- Canlı iki yönlü sohbet için WebSockets, tek yönlü canlı akışlar için SSE tercih edilir.

- HTTP/3 QUIC connection IDs migrate seamlessly across Wi-Fi and cellular switches without dropping.
- gRPC cuts payload serialization overhead by up to 70% compared to verbose JSON strings.
- Use WebSockets for bidirectional duplex streams (chat) and SSE for server pushes (tickers).

11

BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

API Gateway Deseni: Kimlik Doğrulama, SSL Sonlandırma ve Protokol Dönüşümü

The API Gateway Pattern: Auth, SSL Termination & Protocol Translation

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir ülkeye girerken sınır kapısındaki devasa pasaport kontrol binasını düşünün. Ülkedeki 100 farklı şehre gitmek isteyen her turist önce o tek kapıdan geçer: Pasaport kontrol edilir (Kimlik doğrulama), vizesi var mı bakılır (Yetkilendirme), valizinde yasaklı madde var mı taranır (Rate limiting & WAF). Şehirlere geçtikten sonra kimse size her dükkanda tekrar tekrar pasaport sormaz! API Gateway işte o sınırdaki akıllı pasaport kapısıdır!

🇬🇧 ENGLISH (ELI5)

Picture a nation's central border checkpoint terminal. Every foreign traveler heading to any of 50 internal cities must pass through this single customs facility: passports are verified (Authentication), visas are inspected (Authorization), and luggage is x-rayed for contraband (WAF & Rate Limiting). Once inside the country, individual shops don't re-examine your passport! The API Gateway is that secure border checkpoint!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

API Gateway deseni, mikroservis mimarilerinde tek bir giriş noktası (Single Entry Point) sunar. Görevleri: 1) SSL/TLS Termination: Ağır şifre çözme işlemini üstlenir, iç ağdaki mikroservislerin CPU yükünü düşürür. 2) Kimlik Doğrulama & Yetkilendirme: Gelen JWT/OAuth2 tokenını doğrular, kullanıcı kimliğini başlığa (`X-User-Id`, `X-Roles`) koyarak iç servislere aktarır. 3) Protokol Dönüşümü: İstemciden gelen HTTP/JSON isteklerini içeride gRPC veya Protobuf formatına dönüştürür. 4) İstek Birleştirme (API Composition): Tek bir mobil ekran için 5 farklı servisten veriyi paralel çekip tek bir JSON cevabı döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The API Gateway serves as the centralized reverse-proxy perimeter for microservices. Core capabilities: 1) SSL/TLS Termination: Offloads cryptographic handshakes from upstream worker pods. 2) Authentication & Token Introspection: Validates JWT/OAuth2 signatures at the edge, injecting sanitized user identity headers (`X-User-Id`, `X-Tenant-Id`) downstream. 3) Protocol Translation: Ingests public HTTP/REST and translates into internal high-speed binary gRPC calls. 4) Request Aggregation (BFF - Backend for Frontend): Batches parallel calls across disparate domains to prevent mobile network chattiness.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: API Gateway Filtre Zinciri (Pipeline Filter Pattern)
import java.util.function.Function;

public class ApiGatewayPipeline {
    public record GatewayRequest(String path, String token, String clientIp) {}
    public record GatewayContext(GatewayRequest request, String resolvedUserId, boolean authenticated)
{}

    public static Function<GatewayContext, GatewayContext> authFilter() {
        return ctx -> {
            if ("secret-jwt-token".equals(ctx.request().token())) {
                return new GatewayContext(ctx.request(), "usr_9981", true);
            }
            return new GatewayContext(ctx.request(), null, false);
        };
    }

    public static Function<GatewayContext, String> router() {
        return ctx -> ctx.authenticated()
            ? "FORWARD -> /internal" + ctx.request().path() + " [UID:" + ctx.resolvedUserId() + "]"
            : "HTTP 401 UNAUTHORIZED";
    }
}

void main() {
    var filter = ApiGatewayPipeline.authFilter().andThen(ApiGatewayPipeline.router());
    var validReq = new ApiGatewayPipeline.GatewayRequest("/orders", "secret-jwt-token", "1.2.3.4");
    IO.println(filter.apply(new ApiGatewayPipeline.GatewayContext(validReq, null, false)));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

API Gateway'in içine iş mantığı (Business Logic) YAZMAYIN! Gateway bir proxy'dir; fatura hesaplama veya veritabanı sorgusu gibi işler eklerseniz Gateway monolitik bir darboğaza ve tek hata noktasına (SPOF) dönüşür.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never inject core business logic into the API Gateway layer! A gateway must remain a lean transport proxy; bloating it with domain business logic creates a brittle distributed monolith bottleneck.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- API Gateway mikroservisleri dış dünyanın karmaşıklığından ve güvenlik risklerinden izole eder.
- SSL sonlandırma iç mikroservislerin CPU kapasitesini iş mantığına ayırmasını sağlar.
- Backend-for-Frontend (BFF) deseni mobil ve web istemcilerine özel optimize edilmiş gateway'ler kurar.

🎯 Key Takeaways for the Interview

- The API Gateway insulates internal microservices from perimeter exposure and protocol churn.
- SSL termination offloads intensive crypto operations from backend compute instances.
- BFF variants dedicate separate gateways tailored specifically to mobile versus web UI needs.

12

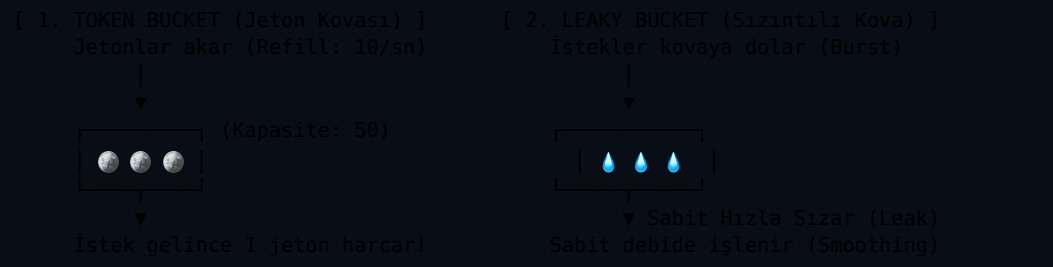
BÖLÜM: YÜK DENGELEME, AĞ PROTOKOLLERI VE API AĞ GEÇITLERI | MODULE: LOAD BALANCING, NETWORKING PROTOCOLS & API GATEWAYS

Dağıtık Hız Sınırlama (Rate Limiting): Token Bucket, Leaky Bucket, Sliding Window

Distributed Rate Limiting: Token Bucket, Leaky Bucket & Sliding Window

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir lunapark trenini düşünün: Trene binebilmek için jeton makinesinden jeton almalısınız (Token Bucket). Makine kovaya her saniye 2 yeni jeton atar; kova en fazla 10 jeton alır. Bir anda 10 kişi gelirse hepsi aynı anda trene biner (Burst). Ama 11. kişi kova dolana kadar kapıda beklemek zorundadır! Rate Limiter işte bu lunapark görevlisidir: Kötü niyetli bir robot veya aşırı heyecanlı bir kullanıcı saniyede 10.000 istek atarak sunucuyu çökertmesin diye 'Sakin ol, kotan doldu! (HTTP 429 Too Many Requests)' der!

🇬🇧 ENGLISH (ELI5)

Imagine an amusement park roller coaster ride requiring tokens (Token Bucket). An automated dispenser drops 2 tokens into a bucket every second, up to a max capacity of 10. If a sudden group of 10 arrives, they grab all 10 tokens and board simultaneously (Burst). But person 11 must wait until new tokens drop! The Rate Limiter is that turnstile guard: shielding databases from aggressive bots or buggy loops by replying with HTTP 429 Too Many Requests!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında en çok sorulan 4 Rate Limiting algoritması: 1) Token Bucket: Amazon ve Stripe'ın tercihidir. Sabit hızda jeton dolar; burst (ani yük) trafiğe izin verir, bellek tüketimi azdır (sadece sayaç + zaman damgası). 2) Leaky Bucket: Trafiği sabit bir çıkış hızında düzleştirir (Traffic Shaping / Smoothing). 3) Fixed Window Counter: Zamanı sabit pencerelere böler (12:00-12:01); pencere sınırında iki katı istek geçmesine izin veren 'Window Boundary' açığı vardır. 4) Sliding Window Log / Counter: Redis ZSET veya bellek sayaçlarıyla kayan pencereyi milisaniye hassasiyetinde tutar; sınır açığını sıfırlar. Dağıtık ortamda Redis + Lua Scripting ile atomik olarak uygulanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The 4 core rate limiting algorithms evaluated in MAANG rounds: 1) Token Bucket: Favored by Amazon/Stripe. Tokens replenish at a fixed rate; effortlessly absorbs short burst traffic with negligible memory (stores only timestamp + token count). 2) Leaky Bucket: Smooths bursty traffic into a constant outflow rate (vital for throttling queues). 3) Fixed Window Counter: Simple counter reset at window boundaries; vulnerable to 2x burst surges across the window boundary. 4) Sliding Window Counter: Blends historical and current window weights to neutralize boundary spikes with minimal Redis memory overhead. Implemented in distributed clusters via atomic Redis Lua scripts.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Thread-Safe Token Bucket Algoritması
import java.time.Instant;

public class TokenBucketRateLimiter {
    private final long capacity;
    private final double refillTokensPerSecond;
    private double availableTokens;
    private long lastRefillTimestamp;

    public TokenBucketRateLimiter(long capacity, double refillTokensPerSecond) {
        this.capacity = capacity;
        this.refillTokensPerSecond = refillTokensPerSecond;
        this.availableTokens = capacity;
        this.lastRefillTimestamp = System.currentTimeMillis();
    }

    public synchronized boolean tryAcquire() {
        refill();
        if (availableTokens >= 1.0) {
            availableTokens -= 1.0;
            return true; // İstek kabul edildi
        }
        return false; // HTTP 429 Too Many Requests
    }

    private void refill() {
        long now = System.currentTimeMillis();
        double elapsedSeconds = (now - lastRefillTimestamp) / 1000.0;
        availableTokens = Math.min(capacity, availableTokens + elapsedSeconds *
refillTokensPerSecond);
        lastRefillTimestamp = now;
    }
}

void main() {
    var limiter = new TokenBucketRateLimiter(3, 1.0); // 3 kapasite, saniyede 1 jeton
    for (int i = 1; i <= 5; i++) {
        boolean allowed = limiter.tryAcquire();
        IO.println("İstek #" + i + " İzin Verildi mi? " + allowed);
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Dağıtık ortamda her web sunucusunun kendi belleğinde local rate limiter tutması mülakatta eksi puandır! 10 sunucu varsa kullanıcı her sunucuya 100 istek atıp toplamda 1.000 istek yapabilir. Hız sınırlayıcı merkezi bir Redis kümesinde veya Envoy filter seviyesinde tutulmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Storing rate limit counters in local JVM memory fails in clustered deployments. 10 pods mean a user can fire 10x their quota via round-robin. Centralize distributed counters within a shared Redis cluster utilizing Lua scripts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Token Bucket algoritması ani yükleri (bursts) tolere ettiği için endüstri standardıdır.

• Dağıtık ortamlarda yarış durumunu (Race Condition) önlemek için Redis Lua Script kullanılır.

• Kotası dolan istemciye HTTP 429 ve 'Retry-After: 30' yanıt başlığı dönülmelidir.

🎯 Key Takeaways for the Interview

• Token Bucket is the industry standard because it gracefully accommodates bursty traffic spikes.

• Redis Lua scripts execute read-and-decrement steps atomically without distributed locks.

• Rate-limited clients must receive HTTP 429 along with a Retry-After header.

BÖLÜM 3 / MODULE 3

Önbellekleme Mimarisi ve CDN Stratejileri

Distributed Caching Architectures & CDN Strategies

Donanım gecikme piramidi, Cache-Aside vs Write-Through, LRU tahliye mekanizması, Redis Cluster, Cache Stampede/Avalanche felaketleri ve Kenar CDN mimarisi.

Hardware latency hierarchy, Cache-Aside vs Write-Through, LRU evictions, Redis clustering, Thundering Herd mitigations, and Edge CDN deployment topologies.

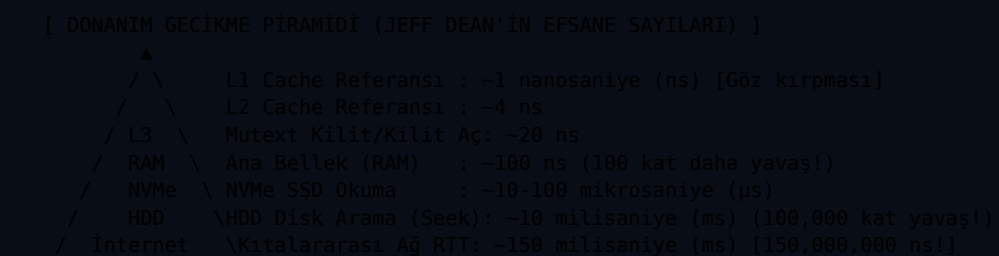
13

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Seviyeleri ve Donanım Piramidi (Her Mühendisin Bilmesi Gereken Sayılar)

Memory Hierarchy & Latency Numbers Every Engineer Must Know

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Donanım hızlarını insan boyutuna uyarlayalım: L1 önbellekten veri okumak, masanızın üzerindeki kağıda bakmak gibidir (1 saniye sürer). RAM'den okumak, odadan çıkıp alt kattaki arşiv odasına yürümektir (1.5 dakika sürer). NVMe SSD'den okumak, komşu şehre arabayla gidip evrak almaktır (1.5 gün sürer). Dönen mekanik hard diskten (HDD) okumak, 3 ay süren bir dünya turuna çıkmaktır! İnternet üzerinden başka bir ülkeden veri çekmek ise Mars'a gidip gelmek gibidir (5 yıl sürer)! İşte önbellek, veriyi Mars'tan veya arşivden alıp masanızın üzerine koyma sanatıdır!

🇬🇧 ENGLISH (ELI5)

Scaling computer latencies to human time: An L1 CPU cache reference is glancing at a sticky note on your desk (1 second). Fetching from RAM is walking downstairs to the building archives (90 seconds). Reading from an NVMe SSD is driving to a neighboring city (1.5 days). Reading from a spinning magnetic HDD is boarding a sailboat for a 3-month trans-Atlantic voyage! Fetching from a cross-continental remote API is a 5-year round trip to Mars! Caching is the supreme discipline of keeping hot data glued directly to your desk!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Jeff Dean'in ünlü 'Numbers Every Programmer Should Know' tablosu sistem tasarımının fiziksel zeminidir: L1 Cache: 0.5-1 ns, L2: 3-4 ns, RAM: 100 ns, SSD rastgele okuma: 16-100 µs, Veri merkezi içi RTT: 500 µs, HDD arama: 10 ms, Kaliforniya-Hollanda RTT: 150 ms. Bir veritabanı sorgusu disk ve ağ IO içerdiğinde milisaniyeler sürerken; RAM'deki bir Redis önbelleği mikrosaniyeler mertebesindedir (1.000 kat daha hızlı). Mülakatta veritabanı üzerindeki okuma yükünü %90 azaltmanın tek yolu bu piramidi doğru katmanlamaktır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Jeff Dean's latency benchmarks dictate distributed system sizing: L1: 1ns, L2: 4ns, RAM: 100ns, NVMe SSD read: 20µs, Datacenter RTT: 0.5ms, Mechanical disk seek: 10ms, Trans-Atlantic RTT: 150ms. Disk operations are orders of magnitude slower than DRAM access. Placing an in-memory Redis or Caffeine layer in front of relational databases reduces latency percentiles from 15ms down to 500µs while absorbing 95% of read pressure.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Donanım Gecikme Skalasını İnsan Zamanına Çeviren Model
public record LatencyScale(String operation, long nanos) {
    // 1 nanosaniyeyi 1 saniye kabul ederek insan ölçeğine çevir
    public String toHumanScale() {
        long seconds = nanos;
        if (seconds < 60) return seconds + " saniye";
        long minutes = seconds / 60;
        if (minutes < 60) return minutes + " dakika";
        long hours = minutes / 60;
        if (hours < 24) return hours + " saat";
        long days = hours / 24;
        if (days < 365) return days + " gün";
        return (days / 365) + " yıl";
    }
}

void main() {
    var l1 = new LatencyScale("L1 Cache", 1L);
    var ram = new LatencyScale("RAM Bellek", 100L);
    var ssd = new LatencyScale("NVMe SSD", 100_000L);
    var net = new LatencyScale("Trans-Atlantik Ağ", 150_000_000L);

    IO.println(l1.operation() + " -> " + l1.toHumanScale());
    IO.println(ram.operation() + " -> " + ram.toHumanScale());
    IO.println(ssd.operation() + " -> " + ssd.toHumanScale());
    IO.println(net.operation() + " -> " + net.toHumanScale());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'RAM ucuz, her şeyi sonsuza kadar Redis'te saklarım' demeyin! RAM, SSD'ye göre gigabayt başına yaklaşık 5-10 kat daha pahalıdır. Ön bellek sadece 'Sıcak Veri' (Hot Working Set) içindir; soğuk veriler daima disk/nesne depolamada tutulmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never declare 'RAM is cheap, I'll cache the entire 100TB database in Redis'. DRAM is 5x to 10x more expensive per gigabyte than NVMe. Caches are exclusively reserved for the 20% hot active working set.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• RAM diskten 1.000 kat, ağ gecikmesinden 1.000.000 kat daha hızlıdır.

• Sistem performansının en büyük düşmanı disk I/O ve ağ RTT gecikmesidir.

• Önbellek, sıcak veriyi donanım piramidinin en tepesine yaklaştırma stratejisidir.

🎯 Key Takeaways for the Interview

• DRAM is 1,000x faster than NVMe disks and 1,000,000x faster than cross-regional networks.

• The primary latency bottlenecks in web services are disk IO seeks and network RTT.

• Caching bridges physical latency penalties by lifting hot data to the top of the pyramid.

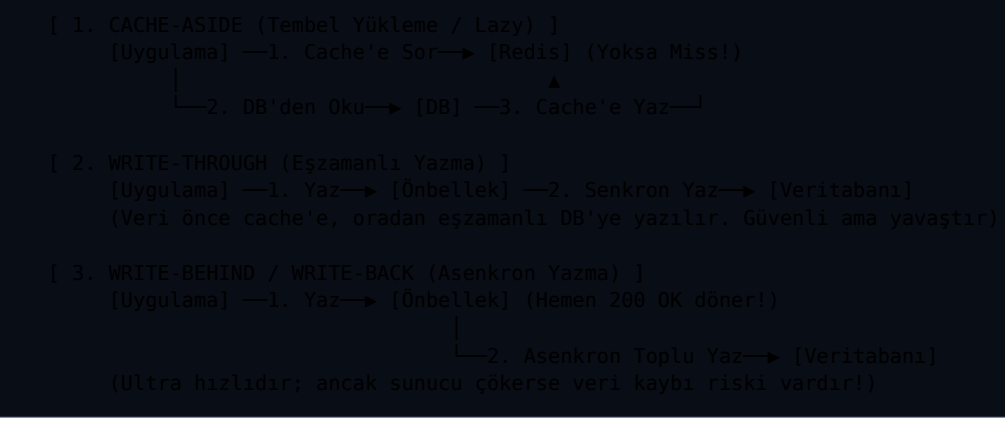
14

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Yazma Stratejileri: Cache-Aside, Write-Through ve Write-Back

Cache Writing Patterns: Cache-Aside, Write-Through & Write-Behind

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir not defteriniz (Cache) ve kalın bir arşiv defteriniz (DB) olsun: 1) Cache-Aside: Bir soru sorulduğunda önce not defterine bakarsınız; yoksa arşivden bulup not defterinize iliştirirsiniz. 2) Write-Through: Yeni bir bilgi geldiğinde aynı anda hem not defterine hem arşive yazıp öyle kafanızı kaldırırsınız (ikisi de hep günceldir ama yavaş yazarsınız). 3) Write-Back: Yeni bilgiyi hemen not defterine karalar ve 'Tamam!' dersiniz; akşama doğru arşiv defterine topluca geçirirsiniz. Ultra hızlıdır ama not defterini çay dökülüp kaybolursa bilgiler silinir!

🇬🇧 ENGLISH (ELI5)

Imagine a pocket notepad (Cache) and a heavy leather ledger (Database): 1) Cache-Aside: When asked a question, check your notepad; on a miss, look up the ledger and scribble the answer into your notepad for next time. 2) Write-Through: When recording an entry, write simultaneously to the notepad AND the ledger before responding (bulletproof consistency, higher write latency). 3) Write-Behind (Write-Back): Scribble into the notepad instantly and confirm completion; asynchronously batch-sync to the ledger later (extreme write speed, but if the notepad burns before sync, data vaporizes)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Cache-Aside (Lazy Loading): En yaygın modeldir (Memcached, Redis). Uygulama kodu önbellek ve veritabanı koordinasyonunu bizzat yönetir. Sadece talep edilen veriler önbelleğe alınır; dezavantajı: Cache Miss durumunda 3 ağ gidiş-gelişi (Cache oku -> DB oku -> Cache yaz). Veri güncellendiğinde DB yazılır ve cache `EVICT` (silinir) edilir. 2) Write-Through: Önbellek bir ara katmandır. Veri önce cache'e yazılır, cache senkron olarak DB'ye yazar. Tutarlılık yüksektir ancak yazma gecikmesi artar. 3) Write-Behind / Write-Back: Veri cache'e yazılır ve anında yanıt dönülür; arka plan worker'ları veriyi kuyruktan alıp DB'ye asenkron toplu (batch) yazar. Yoğun yazma trafiği olan sistemlerde (oyun skorları, sayaçlar, IoT sensör verileri) kullanılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Cache-Aside: The application directly orchestrates cache and persistence. Misses fetch from DB and populate the cache. Updates write to DB and evict stale cache keys. 2) Write-Through: The cache acts as the main persistence abstraction. Writes update cache first, which synchronously flushes to DB before returning, guaranteeing strict read consistency at the penalty of elevated write latency. 3) Write-Behind (Write-Back): Writes hit the in-memory cache and return immediately (sub-millisecond ACK); background workers batch-drain dirty writes to DB asynchronously. Delivers unmatched write throughput for metrics, telemetry, and live view counters at the risk of transient data loss on node crashes.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Cache-Aside & Write-Behind Simülasyonu
import java.util.concurrent.*;

public class CachePatterns {
    private final ConcurrentMap<String, String> cache = new ConcurrentHashMap<>();
    private final BlockingQueue<String> writeBehindQueue = new LinkedBlockingQueue<>();

    // 1. Cache-Aside Okuma Mantığı
    public String getWithCacheAside(String key) {
        // Cache kontrolü
        String cached = cache.get(key);
        if (cached != null) return cached;

        // DB simülasyonu (Cache Miss)
        String dbValue = "DB_VAL_FOR_" + key;
        cache.put(key, dbValue);
        return dbValue;
    }

    // 2. Write-Behind (Asenkron Kuyruğa Atma)
    public void writeBehind(String key, String value) {
        cache.put(key, value); // RAM'e hemen yaz
        writeBehindQueue.offer(key + "=" + value); // DB worker'ına havale et
    }
}

void main() {
    var patterns = new CachePatterns();
    IO.println("Cache-Aside Değeri: " + patterns.getWithCacheAside("user:101"));
    patterns.writeBehind("score:99", "1500");
    IO.println("Write-Behind Başarılı (Bellek Güncellendi)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Cache-Aside deseninde güncelleme yaparken önbelleğe yeni değeri YAZMAYIN (Cache Update)! Eski değeri SİLİN (Cache Eviction / Invalidation)! İki eşzamanlı yazma işlemi yarıştığında (Race Condition), önbellekte bayat veri kalabilir. En güvenli kural: DB'ye yaz, cache'i sil!

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

During mutations in Cache-Aside, do NOT update the cache in-place! DELETE/EVICT the key instead! In concurrent write races, in-place updates leave stale state. The golden rule: Commit to DB, then EVICT the cache key.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Cache-Aside okuma ağırlıklı (read-heavy) sistemlerin tartışmasız standart desenidir.
- Veri güncellendiğinde önbelleği güncellemek yerine anahtarı silmek (evict) yarış durumlarını önler.
- Write-Behind devasa yazma hacimlerini (yüksek QPS) veritabanına toplu aktarmak için idealdir.

🎯 Key Takeaways for the Interview

- Cache-Aside is the default pattern for read-heavy distributed architectures.
- Evicting cache keys upon DB mutations avoids dangerous concurrency race conditions.
- Write-Behind consolidates massive write bursts into efficient batch database flushes.

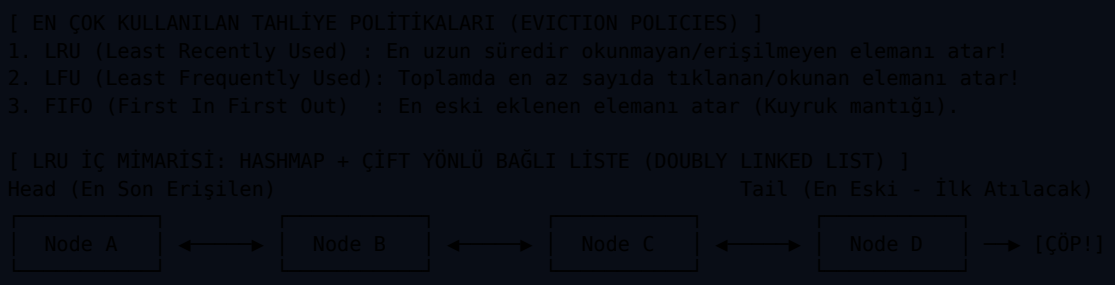
15

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Tahliye Algoritmaları: LRU, LFU, FIFO ve Java 25 LRU Cache

Cache Eviction Algorithms: LRU, LFU, FIFO & Java 25 LRU Cache

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Çalışma masanızın üzerinde en fazla 5 kitap koyabileceğiniz bir alan olduğunu hayal edin (Önbellek boyutu). 6. kitabı koymak için birini rafa kaldırmalısınız. LRU (Least Recently Used), 'En uzun süredir kapağını bile açmadığım kitabı kaldırayım' diyen mantıktır. LFU (Least Frequently Used) ise 'Sene boyunca toplamda en az okuduğum kitabı kaldırayım' der. Masadaki kitaplardan birini her okuduğunuzda onu masanın en önüne (Head) çekersiniz; masanın en arkasında kalan kitap (Tail) ise ilk atılacak kurbandır!

🇬🇧 ENGLISH (ELI5)

Imagine your desk only fits 5 textbooks. To place a 6th book, you must return one to the library shelf. LRU (Least Recently Used) evicts the book you haven't touched in the longest time. LFU (Least Frequently Used) counts tally marks and evicts the book with the lowest total read count. Whenever you consult a book, you slide it to the front of your desk. The book lingering neglected at the far corner is the first victim evicted on overflow!

🔍 MAANG Seviyesi Teknik Derinlik 🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında en çok kodlatılan veri yapısı LRU Cache'tir. Bir LRU Cache iki temel işlem için $O(1)$ zaman karmaşıklığı sunmalıdır: 1) `get(key)`: Anahtarı bulmalı ve düğümü en başa (Head) taşımalıdır. 2) `put(key, value)`: Yeni düğümü başa eklemeli, kapasite aşıldığında sondaki (Tail) düğümü silmelidir. Bu $O(1)$ garantisi HashMap ($O(1)$ anahtar araması) ile Doubly Linked List'in ($O(1)$ düğüm çıkarma ve ekleme) birleşimiyle sağlanır. LFU ise frekans sayacı tutar; ancak ani popülerlik dalgalanmalarına (flash crowds) uyum sağlaması zordur.

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The LRU Cache is a premier FAANG coding interview staple. It guarantees $O(1)$ time complexity for both `get` and `put` operations. This is engineered by coupling a Hash Table (for $O(1)$ pointer lookups) with a Doubly Linked List (for $O(1)$ node extractions and insertions at Head and Tail boundaries). When a node is touched, it splices out of its current position and prepends to the Head. On capacity overflow, the Tail node drops. LFU maintains frequency buckets but suffers from historical inertia where old items refuse to evict despite going cold.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: O(1) Zaman Karmaşıklığına Sahip Gerçek LRU Cache Implementasyonu
import java.util.*;

public class LruCache<K, V> {
    private static class Node<K, V> {
        K key; V value;
        Node<K, V> prev, next;
        Node(K k, V v) { this.key = k; this.value = v; }
    }

    private final int capacity;
    private final Map<K, Node<K, V>> map = new HashMap<>();
    private final Node<K, V> head = new Node<>(null, null);
    private final Node<K, V> tail = new Node<>(null, null);

    public LruCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.prev = head;
    }

    public synchronized V get(K key) {
        Node<K, V> node = map.get(key);
        if (node == null) return null;
        moveToHead(node);
        return node.value;
    }

    public synchronized void put(K key, V value) {
        Node<K, V> node = map.get(key);
        if (node != null) {
            node.value = value;
            moveToHead(node);
        } else {
            if (map.size() >= capacity) {
                map.remove(tail.prev.key);
                removeNode(tail.prev);
            }
            Node<K, V> newNode = new Node<>(key, value);
            map.put(key, newNode);
            addToHead(newNode);
        }
    }

    private void removeNode(Node<K, V> node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    private void addToHead(Node<K, V> node) {
        node.next = head.next;
        node.prev = head;
        head.next.prev = node;
        head.next = node;
    }

    private void moveToHead(Node<K, V> node) {
        removeNode(node);
        addToHead(node);
    }
}

void main() {
    var lru = new LruCache<String, Integer>(2);
    lru.put("A", 1);
    lru.put("B", 2);
    IO.println("Get A: " + lru.get("A")); // A başa geçti, B arkaya düştü
    lru.put("C", 3);                      // Kapasite taştı -> B atıldı!
    IO.println("Get B (Tahliye Edildi): " + lru.get("B")); // null döner!
    IO.println("Get C: " + lru.get("C"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Java'nın `LinkedHashMap(capacity, 0.75f, true)` sınıfı LRU mantığını hazır sunar. Ancak MAANG mülakatında hazır kütüphane çağırmak yerine HashMap + Doubly Linked List veri yapısını kendi elinizle çizip açıklamanız beklenir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

While Java provides `LinkedHashMap(..., accessOrder=true)`, invoking it without explaining the underlying Doubly Linked List and Hash Map duality will disqualify your algorithmic rigor in Staff rounds.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LRU en son erişilen veriyi korur ve zamansal yerellik (temporal locality) için mükemmeldir.
- O(1) süre garantisi HashMap ile Çift Yönlü Bağlı Liste kombinasyonuyla elde edilir.
- Redis varsayılan olarak yaklaşık (approximate) LRU algoritması kullanarak bellekten tasarruf eder.

🎯 Key Takeaways for the Interview

- LRU leverages temporal locality, keeping recently accessed objects warm.
- Combining a Hash Map with a Doubly Linked List yields strict O(1) lookups and evictions.
- Production Redis uses probabilistic sampling to approximate LRU with zero pointer overhead.

16

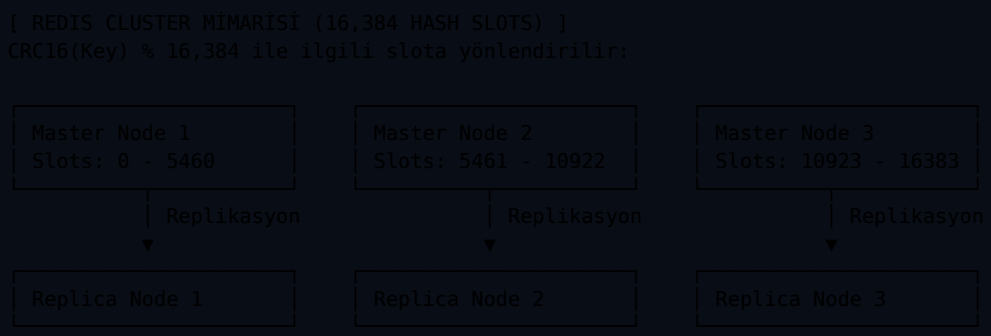
BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Dağıtık Önbellek: Redis ve Memcached Mimarisi (Sentinel vs Cluster)

Distributed Cache: Redis vs Memcached (Sentinel & Cluster Sharding)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Redis ile Memcached iki ünlü yarış arabası gibidir: Memcached sadece dümdüz ileri giden, çok hafif ve basit bir drag yarışçısıdır (Sadece Key-Value tutar, çok çekirdek kullanır). Redis ise İsviçre çakısı gibi donatılmış bir Formula 1 aracıdır! Listeler, Kümeler (Sets), Sıralı Kümeler (Sorted Sets), Bitmap'ler, Pub/Sub ve diske kaydetme (Persistence) yetenekleri vardır. Redis Cluster ise bu Formula 1 filosunun 3 farklı pistte (16,384 yuva - hash slot) yarışarak yükü paylaşmasıdır!

🇬🇧 ENGLISH (ELI5)

Redis and Memcached are two performance sports cars: Memcached is a stripped-down dragster built for raw multi-threaded speed (pure Key-Value strings). Redis is a hypercar engineered with an entire Swiss Army knife! It natively manipulates Hashes, Lists, Sets, Sorted Sets (ZSET for leaderboards), Bitmaps, HyperLogLogs, and Pub/Sub streams with disk durability (RDB/AOF). Redis Cluster partitions keys across 16,384 distributed hash slots for horizontal scaling!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Memcached: Multi-threaded mimariye sahiptir; çok çekirdekli makinelerin CPU gücünü tam kullanır; veri tipleri yoktur (sadece binary string); disk kalıcılığı yoktur. Redis: Çekirdek motoru tek iş parçacıklıdır (Single-threaded event loop), bu sayede race condition ve kilit (lock) kargaşası olmadan komutları atomik yürütür (Network IO ise multithreaded'dir). Redis Mimarileri: 1) Redis Sentinel: Yüksek erişilebilirlik (HA) sağlar; Master çökerse otomatik Replica'yı Master yapar (Failover). 2) Redis Cluster: Yatay ölçekleme (Sharding) sağlar. Anahtarlar `CRC16(key) % 16384` formülüyle 16,384 Hash Slot'a bölünür ve master düğümler arasında paylaştırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Memcached utilizes a multithreaded architecture maximizing multi-core symmetric multiprocessing, but supports only basic string key-values without disk durability. Redis runs an ultra-fast non-blocking single-threaded event loop, guaranteeing atomic executions across rich data structures (Lists, ZSETs, Streams, Hashes) without mutex locking overhead. Scalability options: 1) Redis Sentinel provides automatic master-replica failover monitoring. 2) Redis Cluster delivers true horizontal sharding: keys map to 16,384 hash slots via `CRC16(key) % 16384`, distributing partitions across autonomous master-replica shards with zero single points of failure.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Redis Hash Slot Hesaplayıcı Simülasyonu (CRC16 % 16384)
public class RedisClusterSlotRouter {
    private static final int TOTAL_SLOTS = 16384;

    // Basitleştirilmiş Hash Slot Hesaplama
    public static int calculateSlot(String key) {
        // Redis Hash Tag Desteği: {user:101}.orders -> Sadece {} içi hashlenir!
        int start = key.indexOf('{');
        int end = key.indexOf('}');
        String hashKey = (start != -1 && end > start) ? key.substring(start + 1, end) : key;

        return Math.abs(hashKey.hashCode() % TOTAL_SLOTS);
    }
}

void main() {
    String key1 = "product:iphone16";
    String key2 = "user:9941";
    String key3 = "{tenant:acme}.orders";
    String key4 = "{tenant:acme}.invoices"; // key3 ile KESİNLİKLE aynı slot'a gider!

    IO.println(key1 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key1));
    IO.println(key2 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key2));
    IO.println(key3 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key3));
    IO.println(key4 + " -> Slot #" + RedisClusterSlotRouter.calculateSlot(key4));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Redis Cluster'da çoklu anahtar (MGET, MSET veya Transactions) çalıştırırken anahtarlar farklı slotlardaysa 'CROSSSLOT Keys in request don't hash to the same slot' hatası patlar! İlişkili verilerin aynı slota düşmesini sağlamak için Hash Tag `\{tenant_id\}.key` sözdizimini kullanın.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Redis Cluster throws CROSSSLOT errors when batching multi-key commands across different hash slots. Enforce colocation using Hash Tags (`\{user_id\}.profile` and `\{user_id\}.orders`) so related keys evaluate to the exact same shard.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis zengin veri tipleri ve atomik işlemleriyle modern sistemlerin vazgeçilmezidir.
- Redis Cluster, 16,384 Hash Slot yapısıyla veriyi onlarca sunucuya şeffafça böler.
- Hash Tags (\{tag\}) tek işlemde birden fazla anahtarı sorgulamak için slot sabitleme sağlar.

🎯 Key Takeaways for the Interview

- Redis is the premier caching engine due to its rich native data types and atomic primitives.
- Redis Cluster transparently shards datasets across 16,384 deterministic hash slots.
- Hash Tags ensure co-partitioning for multi-key transactional pipelines on clustered shards.

17

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Felaketleri: Cache Avalanche, Stampede (Thundering Herd) ve Penetration

Cache Catastrophes: Avalanche, Stampede (Thundering Herd) & Penetration

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ 1. CACHE AVALANCHE (ÇIĞ ETKİSİ) ]
Milyonlarca anahtarın TTL'i aynı anda (ör. gece 00:00) biter!
Önbellek ──► %100 Boşalır ──► Milyonlarca İstek Veritabanına Hücum Eder ──► DB ÇÖKER!
Çözüm: TTL değerine rastgele Jitter (± 5 dakika) ekleyin!

[ 2. CACHE STAMPEDE / THUNDERING HERD (SÜRÜ İZDIHAMI) ]
Çok popüler tek bir anahtarın (Messi'nin profili) TTL'i biter.
Saniyede 50.000 istek aynı anda DB'ye koşup o tek satırı hesaplamaya çalışır!
Çözüm: Dağıtık Kilit (Mutex / Redlock) veya Arka Planda Proaktif Yenileme!

[ 3. CACHE PENETRATION (DELME SALDIRISI) ]
Saldırgan DB'de OLMAYAN rastgele ID'ler ister (ID: -9999999).
Cache Miss ──► DB sorgusu (Bulunamadı) ──► Cache'e yazılmaz ──► Saldırı DB'yi boğar!
Çözüm: Bloom Filter veya Boş Sonucu (Null Value) kısa TTL ile önbelleğe alma!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bu üç felaketi lokantada anlayalım: 1) Çığ Etkisi (Avalanche): Mutfaktaki tüm hazır yemeklerin aynı anda saat 12:00'de bozulup çöpe atılmasıdır; kapıdaki 500 müşteri doğrudan ocağın başına üşüşür ve mutfak yanar! 2) Sürü İzdihamı (Stampede): Menüdeki en sevilen tatlı biter; 1.000 müşteri aynı anda garsona 'Hemen aşçıya söyle bu tatlıyı yapsın' der, aşçı 1.000 kez aynı tatlıyı pişirmeye çalışırken bayılır! 3) Delme Saldırısı (Penetration): Kötü niyetli biri garsona 'Menüde olmayan Ejderha Çorbası var mı?' diye 100.000 kez sorar. Garson her seferinde depoya gidip bakar, depoyu kilitler!

🇬🇧 ENGLISH (ELI5)

Three restaurant nightmares: 1) Cache Avalanche: Every prepared buffet dish expires at precisely 12:00 PM; a swarm of 500 hungry diners surges straight into the kitchen, suffocating the stoves! 2) Cache Stampede (Thundering Herd): The signature chocolate cake runs out; 10,000 diners concurrently order the chef to bake a new cake, suffocating the pastry station! 3) Cache Penetration: A malicious troll repeatedly asks: 'Do you have Dragon Steak?' (an item that doesn't exist). The waiter marches into the basement warehouse 50,000 times, grinding service to a halt!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Cache Avalanche: Yüzbinlerce önbellek anahtarının aynı TTL ile süresinin dolması sonucu isteklerin doğrudan veritabanını çökertmesidir. Çözüm: TTL süresine rastgele gürültü (Jitter) eklemek (`TTL = 3600 + rand(0, 300)`). 2) Cache Stampede: Yüksek QPS'li tek bir sıcak anahtarın süresi dolduğunda binlerce eşzamanlı thread'in DB'ye hücum etmesidir. Çözüm: Dağıtık Kilit (Mutex): Sadece ilk thread DB'ye gider, diğerleri bekler veya proaktif XFetch algoritması. 3) Cache Penetration: Veritabanında bulunmayan anahtarların sorgulanmasıyla cache'in delinip DB'nin boğulmasıdır. Çözüm: A) Bloom Filter: Olasılıksal veri yapısıyla elemanın DB'de kesinlikle olmadığını $O(1)$ sürede bellekten söyleyerek isteği reddeder. B) Null Caching: Boş sonucu kısa bir TTL (60 sn) ile önbelleğe almak.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Cache Avalanche: Mass simultaneous TTL expirations collapse the database under an instantaneous traffic waterfall. Mitigation: Inject stochastic TTL jitter (`ttl = base_ttl + random_jitter`). 2) Cache Stampede (Thundering Herd): A hyper-popular hot key expires, triggering thousands of concurrent cache misses that hammer the database to compute the identical payload. Mitigation: Mutex locking (only one thread computes, others poll) or probabilistic early expiration (XFetch algorithm). 3) Cache Penetration: Adversaries query non-existent keys (e.g., negative UUIDs), bypassing cache and degrading backend disk IO. Mitigation: Deploy a space-efficient Bloom Filter at the gateway to reject non-existent keys in O(1) time, or cache `NULL` payloads with transient TTLs.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Cache Stampede (Mutex Kilit) & Avalanche (Jitter) Çözümleri
import java.util.concurrent.*;
import java.util.concurrent.locks.ReentrantLock;

public class CacheDefenses {
    private final ConcurrentMap<String, String> cache = new ConcurrentHashMap<>();
    private final ConcurrentMap<String, ReentrantLock> locks = new ConcurrentHashMap<>();

    // 1. Avalanche Koruması: Rastgele Jitter ile TTL Üretimi
    public int calculateTtlWithJitter(int baseTtlSeconds, int jitterRangeSeconds) {
        int jitter = ThreadLocalRandom.current().nextInt(jitterRangeSeconds);
        return baseTtlSeconds + jitter; // Örn: 3600 + [0-300] sn
    }

    // 2. Stampede Koruması: Mutex Kilit ile Tekil DB Sorgusu
    public String getWithMutex(String key) {
        String val = cache.get(key);
        if (val != null) return val;

        ReentrantLock lock = locks.computeIfAbsent(key, k -> new ReentrantLock());
        lock.lock();
        try {
            // Çift Kontrol (Double-Checked Locking)
            val = cache.get(key);
            if (val != null) return val;

            // Yalnızca TEK bir thread DB'ye gider!
            val = "HEAVY_DB_RESULT_FOR_" + key;
            cache.put(key, val);
            return val;
        } finally {
            lock.unlock();
        }
    }
}

void main() {
    var defense = new CacheDefenses();
    IO.println("Jitter'lı TTL: " + defense.calculateTtlWithJitter(3600, 300) + " saniye");
    IO.println("Stampede Korumalı Sonuç: " + defense.getWithMutex("hot:product:101"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bloom Filter her şeyi kesin bilir' demeyin! Bloom Filter'ın doğasında 'False Positive' (aslında yokken var diyebilme) payı vardır; ancak 'False Negative' (varken yok deme) riski sıfırdır. Bu ayrımı vurgulamak mülakatçıyı büyüler.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim Bloom Filters are 100% accurate. Bloom Filters exhibit false positives (may report an item exists when it doesn't), but zero false negatives. Articulating this trade-off marks architectural mastery.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• TTL sürelerine mutlaka rastgele Jitter ekleyerek Çığ Etkisini (Avalanche) önleyin.

• Sıcak anahtarlar için Mutex kilidi kullanarak Sürü İzdihamını (Stampede) engelleyin.

• Bloom Filter ve Null Caching ile veritabanını Delme Saldırılarına (Penetration) karşı koruyun.

🎯 Key Takeaways for the Interview

• Inject stochastic jitter to key TTLs to eliminate concurrent cache avalanche cascades.

• Use distributed mutex locks to ensure only one thread recomputes a hot expired key.

• Shield databases from penetration attacks using in-memory Bloom Filters and Null Caching.

18

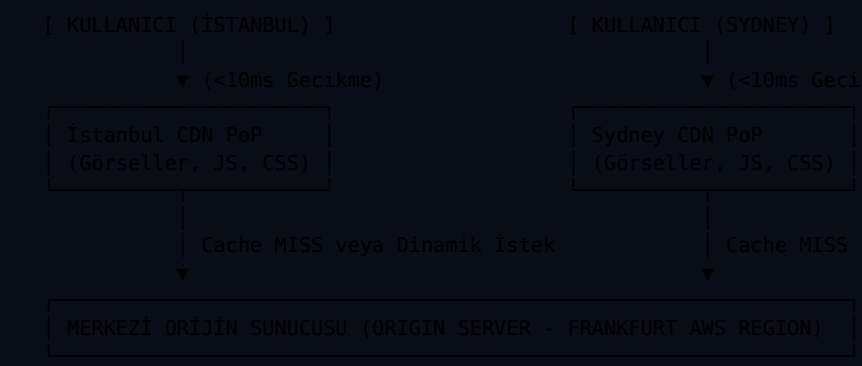
BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

CDN (Content Delivery Network) ve Kenar Hesaplama (Edge Computing & PoP)

CDNs & Edge Computing: PoPs, Anycast & Edge Workers

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Netflix'teki 4K bir filmi izlemek için her seferinde Kaliforniya'daki ana sunucuya bağlanıp okyanusun altındaki kabloları ısıtmak zorunda kalsaydınız film donardı! Bunun yerine Netflix, o filmin kopyalarını mahallenizdeki internet servis sağlayıcısının (ISP) içine yerleştirilmiş özel kutulara (CDN PoP - Point of Presence) koyar. Siz filmi izlerken Kaliforniya'dan değil, 5 kilometre ötenizdeki kutudan çekersiniz. CDN işte bu küresel film ve fotoğraf kütüphanesidir!

🇬🇧 ENGLISH (ELI5)

If streaming a 4K movie on Netflix required pulling bytes directly from an origin database in Los Angeles over trans-oceanic cables, video would buffer endlessly! Instead, Netflix pre-stages encrypted copies of popular media into custom appliances (Open Connect CDN PoPs) embedded directly inside your city's local ISP exchange. You stream from 5 miles away, not 7,000 miles across the ocean! That is a Content Delivery Network (CDN)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

CDN (İçerik Dağıtım Ağı), dünya genelinde yüzlerce Point of Presence (PoP) noktasında çalışan ters proxy sunucuları ağıdır. 1) Statik İçerik Hızlandırma: Resimler, videolar, JS/CSS dosyaları origin'e gitmeden kenarda sunulur. Origin bant genişliği tüketimini %80-%95 azaltır. 2) Dinamik İçerik Hızlandırma: Dinamik API istekleri önbelleğe alınamasa bile, istemci ile PoP arasındaki TCP/TLS el sıkışması yerel tamamlanır ve origin ile PoP arasında açık tutulan optimize BGP tünelleri (Route Optimization) üzerinden akıtılır. 3) Edge Computing (Cloudflare Workers, AWS Lambda@Edge): JWT doğrulama, A/B testi, bot koruması ve coğrafi yönlendirme kodları doğrudan kenar sunucularda (Edge) çalıştırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

A Content Delivery Network (CDN) is a globally distributed mesh of caching proxies deployed across hundreds of edge Points of Presence (PoPs). 1) Static Acceleration: Serves media, images, CSS, and binaries locally, offloading 90%+ of origin egress costs. 2) Dynamic Site Acceleration (DSA): Even uncacheable dynamic API payloads accelerate via early TLS handshakes at the edge PoP, traversing persistent, route-optimized BGP backbones to origin. 3) Edge Compute (Cloudflare Workers, Fastly Compute): Executes lightweight serverless logic (JWT validation, A/B routing, geolocation geofencing, bot mitigation) directly at edge nodes within 5ms of clients.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: CDN Cache-Control Başlık Üreticisi ve Edge Karar Motoru
public record CdnPolicy(int maxAgeSeconds, int sharedMaxAgeSeconds, boolean isPublic) {
    public String toHeaderValue() {
        return (isPublic ? "public, " : "private, ")
            + "max-age=" + maxAgeSeconds
            + ", s-maxage=" + sharedMaxAgeSeconds // s-maxage CDN için geçerlidir!
            + ", stale-while-revalidate=60";
    }
}

void main() {
    // Statik Profil Resmi Politikası: Tarayıcı 1 saat, CDN 24 saat önbelleğe alsın
    var avatarPolicy = new CdnPolicy(3600, 86400, true);
    IO.println("Cache-Control Başlığı: " + avatarPolicy.toHeaderValue());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'CDN sadece statik resimler içindir' demeyin! Modern CDN'ler (Cloudflare, Fastly, CloudFront); WAF (Web Application Firewall), DDoS kalkanı, TLS sonlandırma, dinamik rota hızlandırma ve Edge Compute (Sunucusuz kod çalıştırma) yapar.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never dismiss CDNs as simple image repositories. Modern edge networks handle DDoS scrubbing, WAF inspection, TLS termination, dynamic route optimization, and edge compute execution.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• CDN statik içerikleri kullanıcıya en yakın PoP noktasından milisaniyeler içinde sunar.

• Origin sunucusuna binen bant genişliği ve sunucu maliyetini %90'a varan oranda düşürür.

• Edge Workers ile kimlik kontrolü ve yönlendirme mantığı istemciye en yakın noktada yürütülür.

🎯 Key Takeaways for the Interview

• CDNs deliver static assets from local PoPs with sub-10ms round-trip latencies.

• Edge caching shields origin infrastructures, cutting egress transit bills by up to 90%.

• Edge computing offloads perimeter authentication and traffic routing directly to the network fringe.

BÖLÜM 4 / MODULE 4

Veritabanı Mimarisi, Bölümleme ve Replikasyon

Database Architecture, Sharding & Replication Topologies

SQL vs NoSQL seçim matrisi, NoSQL türleri, Single/Multi-Leader replikasyon, Sharding stratejileri, Consistent Hashing ve B-Tree vs LSM-Tree indeksleme motorları.

SQL vs NoSQL trade-offs, NoSQL families, Quorum replication, Sharding topologies, Consistent Hashing rings, and B-Tree vs LSM-Tree storage engines.

19

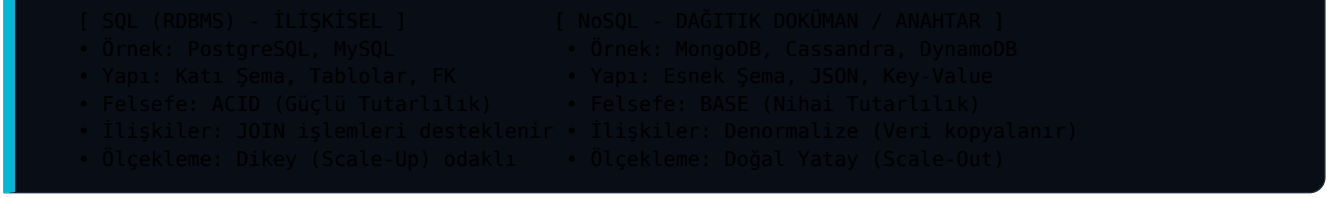
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

İlişkisel (SQL) vs NoSQL: ACID vs BASE ve Doğru Veri Modeli Seçimi

SQL vs NoSQL: ACID vs BASE & Picking the Right Data Store

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

SQL veritabanı, her satırı ve sütunu cetvelle çizilmiş, katı kuralları olan bir devlet tapu sicil dairesi gibidir: Her şey kanunlara (ACID) uyar; iki tapu birbiriyle çakışamaz (JOIN), ama defter büyüdükçe binaya yeni kat çıkmak (Dikey ölçekleme) zorunda kalırsınız. NoSQL ise devasa bir kargo aktarma merkezindeki pratik plastik koliler gibidir (BASE): Her kolinin içine istediğiniz şekilde eşya koyarsınız (esnek JSON şeması); 1 milyon koli gelirse yan tarafa 100 tane yeni depo inşa edip (Yatay ölçekleme) kolileri depolara fırlatırsınız!

🇬🇧 ENGLISH (ELI5)

A relational SQL database is a notarized government land registry: strictly structured tables, foreign keys, and absolute legal validity (ACID guarantees). Everything cross-references seamlessly via JOINs, but expanding means building a taller skyscraper (vertical scaling). NoSQL is an automated Amazon fulfillment sorting warehouse: shipping containers packed with self-contained JSON records (BASE philosophy). When traffic explodes, you don't build taller; you roll out 100 new commodity warehouse bays (horizontal scale-out)!

🔍 MAANG Seviyesi Teknik Derinlik

🔍 MAANG-Level Technical Deep-Dive

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) SQL (ACID): Atomicity, Consistency, Isolation, Durability. Finansal işlemler, e-ticaret ödemeleri ve karmaşık ilişkisel sorgular (JOIN) için vazgeçilmezdir. Zorluğu: Milyarlarca satıra ulaştığında sharding ve cross-node join yapmak aşırı zordur. 2) NoSQL (BASE): Basically Available, Soft state, Eventual consistency. Şemasızdır (Schema-on-read); yatayda onlarca sunucuya yayılmak üzere tasarlanmıştır. Veriler sorgu desenine göre denormalize edilir (JOIN yapılmaz, veriler iç içe gömülür). Seçim Kriteri: Veriniz yüksek ilişkisel mi, yoksa saniyede yüzbinlerce okuma/yazma gerektiren analitik/akış verisi mi?

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) SQL (ACID): Strict schemas with normalized relations, foreign key constraints, and transactional isolation. Essential for financial ledgers, transactional checkout baskets, and complex relational graphs. Scaling ceiling: Cross-shard joins are computationally prohibitive and multi-master clustering introduces heavy consensus locking. 2) NoSQL (BASE): Basically Available, Soft State, Eventual Consistency. Schemaless flexibility (schema-on-read) built ground-up for horizontal sharding. Documents denormalize data to prioritize O(1) primary key queries over cross-table joins.

⚡ Modern Java 25 Uygulaması (Loom & Records) / ⚡ Java 25 Implementation

```
// Java 25: SQL (Normalized) vs NoSQL (Denormalized) Veri Modeli
public class DataModelComparison {

    // 1. SQL Tarzı: Normalize Edilmiş İlişkisel Yapı (Ayrı Tablolar + ID Referansı)
    public record SqlOrder(long orderId, long customerId, java.math.BigDecimal totalAmount) {}
    public record SqlOrderItem(long itemId, long orderId, String productName, int quantity) {}

    // 2. NoSQL Tarzı: Denormalize Edilmiş Doküman (Tek Parça, Sıfır JOIN!)
    public record NoSqlOrderDocument(
        String orderId,
        String customerName,
        String customerEmail,
        java.util.List<OrderItemDoc> items,
        java.math.BigDecimal totalAmount
    ) {
        public record OrderItemDoc(String product, int qty, java.math.BigDecimal price) {}
    }
}

void main() {
    var item = new DataModelComparison.NoSqlOrderDocument.OrderItemDoc("MacBook Pro M4", 1, new
java.math.BigDecimal("2500"));
    var doc = new DataModelComparison.NoSqlOrderDocument("ord_101", "Ali Bozkurt", "ali@test.com",
java.util.List.of(item), new java.math.BigDecimal("2500"));
    IO.println("NoSQL Tekil Doküman Oluşturuldu: ID=" + doc.orderId() + ", Ürün=" +
doc.items().getFirst().product());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı / ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçının 'Hangi veritabanını seçersin?' sorusuna hemen 'MongoDB!' veya 'PostgreSQL!' diye atlamayın. Önce veri erişim modelini (Access Pattern) analiz edin: 'Veri ilişkisel mi? Güçlü ACID gerekiyor mu? Yoksa yüksek hacimli zaman serisi veya anahtar-değer erişimi mi var?' diyerek gerekçenizi temellendirin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never blurt out 'I'll use MongoDB!' before analyzing the query access patterns. State the decision matrix: 'Because our workload requires complex relational multi-table transactions, I recommend PostgreSQL' or 'Because our writes hit 500K QPS with simple key-value reads, Cassandra is optimal.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar / 🎯 Key Takeaways for the Interview

- Finansal tutarlılık ve zengin JOIN sorguları için PostgreSQL/MySQL tercih edilir.
- Yüksek hacimli, şeması değişken ve yatay ölçeklenebilir veriler için NoSQL seçilir.
- Modern mimarilerde Hibrit (Polyglot Persistence) yaklaşım kullanılır (ör. Siparişler SQL'de, Akış Cassandra'da).

- RDBMS (PostgreSQL/MySQL) remains optimal for transactional ACID and relational queries.
- NoSQL fits high-throughput, denormalized, horizontally scalable workloads.
- Enterprise architectures adopt Polyglot Persistence: SQL for orders, NoSQL for feeds.

20

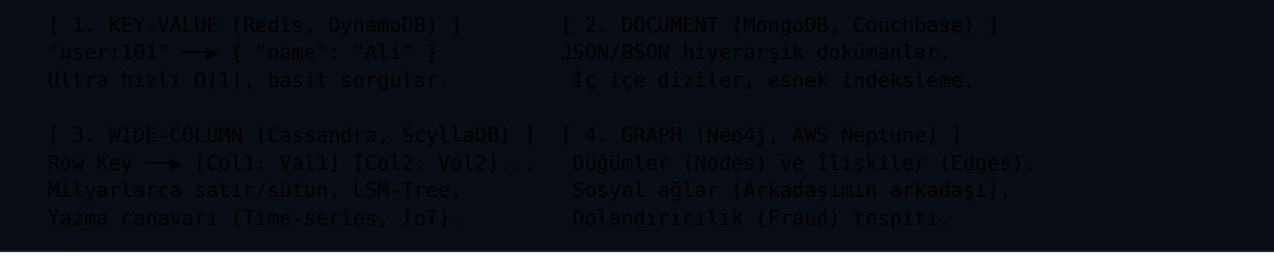
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

NoSQL Aileleri: Document, Key-Value, Wide-Column (Cassandra) ve Graph

The 4 NoSQL Families: Document, Key-Value, Wide-Column & Graph

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

4 farklı alet çantasını düşünün: 1) Key-Value (Redis): Otel vestiyeri gibidir; fiş numarasını verirsiniz, ceketi saniyesinde alırsınız (hızlı ama ceketin cebinde ne olduğunu arayamazsınız). 2) Document (MongoDB): Lüks bir dosya klasörüdür; içinde müşterinin tüm faturaları, fotoğrafları tek bir dosyadadır. 3) Wide-Column (Cassandra): Sonsuz genişlikte devasa bir muhasebe tablosudur; saniyede 1 milyon satır faturayı tıkır tıkır kaydeder (yazma şampiyonu). 4) Graph (Neo4j): Dedektiflerin panosundaki iplerle birbirine bağlanmış fotoğraflardır; 'Ali'nin arkadaşının takip ettiği sayfaları beğenen kim var?' sorusunu şıp diye çözer!

🇬🇧 ENGLISH (ELI5)

Four specialized power tools: 1) Key-Value (Redis, DynamoDB): A coat check room; you hand over ticket 42, you get coat 42 in 1 millisecond (super fast, but you cannot search coats by collar size). 2) Document (MongoDB): A manila folder storing self-contained rich JSON documents with nested arrays. 3) Wide-Column (Cassandra): An infinite spreadsheet engineered with LSM-Trees that swallows 1,000,000 writes/sec without sweating (IoT, telemetry). 4) Graph (Neo4j): A detective's corkboard with red yarn connecting suspects (Nodes) and crimes (Edges); answering 'Who is a friend-of-a-friend that likes this page?' in milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

NoSQL 4 ana kategoriye ayrılır: 1) Key-Value: En basit, en hızlı veri modeli (Redis, Memcached, DynamoDB). Anahtar üzerinden O(1) okuma/yazma. 2) Document Store: JSON/BSON dokümanları (MongoDB, Couchbase). İkincil indeksler (Secondary Indexes) desteklenir; e-ticaret ürün katalogları ve içerik yönetim sistemleri için uygundur. 3) Wide-Column (Columnar Store): Google Bigtable ve Apache Cassandra mimarisi. Veriler sütun aileleri (Column Families) ve Row Key bazında LSM-Tree ile saklanır. Yüksek yazma verimi (Write Throughput) ve zaman serisi (Metrics, Chat geçmişi) için liderdir. 4) Graph Database: Düğümler (Nodes), kenarlar (Edges) ve özellikler (Properties) (Neo4j, Amazon Neptune). Çok katmanlı ilişkisel gezinme (Graph Traversal) sorgularında SQL JOIN'lerinin aksine performans kaybı yaşamaz.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Key-Value (Redis, DynamoDB): Pure O(1) hash maps keyed by unique strings. Minimalist, sub-millisecond, highly partitionable. 2) Document (MongoDB, Couchbase): Semi-structured JSON trees permitting rich secondary indexes and aggregation pipelines, ideal for polymorphic product catalogs. 3) Wide-Column (Cassandra, ScyllaDB): Bigtable-derived sparse multidimensional sorted maps. Optimized with append-only LSM-Trees for extreme write throughput and linear horizontal scaling across IoT feeds, audit logs, and message timelines. 4) Graph (Neo4j, Neptune): Pointer-chasing graph traversal engine executing deep relationship pathfinding (fraud rings, social graphs, recommendation networks) without explosive multi-way SQL JOIN overhead.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Graph Veritabanı Mantığı (Düğümler ve İlişkiler Simülasyonu)
import java.util.*;

public class GraphDatabaseSim {
    public record Node(String id, String name) {}
    public record Edge(Node from, Node to, String relation) {}

    private final Map<Node, List<Edge>> adjacencyList = new HashMap<>();

    public void addFriendship(Node a, Node b) {
        adjacencyList.computeIfAbsent(a, k -> new ArrayList<>()).add(new Edge(a, b, "FRIENDS_WITH"));
        adjacencyList.computeIfAbsent(b, k -> new ArrayList<>()).add(new Edge(b, a, "FRIENDS_WITH"));
    }

    public List<Node> getFriendsOfFriends(Node person) {
        Set<Node> fofs = new HashSet<>();
        for (Edge friendEdge : adjacencyList.getOrDefault(person, List.of())) {
            Node friend = friendEdge.to();
            for (Edge fofEdge : adjacencyList.getOrDefault(friend, List.of())) {
                if (!fofEdge.to().equals(person)) {
                    fofs.add(fofEdge.to());
                }
            }
        }
        return new ArrayList<>(fofs);
    }
}

void main() {
    var graph = new GraphDatabaseSim();
    var ali = new GraphDatabaseSim.Node("1", "Ali");
    var bob = new GraphDatabaseSim.Node("2", "Bob");
    var can = new GraphDatabaseSim.Node("3", "Can");

    graph.addFriendship(ali, bob);
    graph.addFriendship(bob, can); // Ali -> Bob -> Can

    IO.println("Ali'nin Arkadaşının Arkadaşları: " +
graph.getFriendsOfFriends(ali).getFirst().name());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Cassandra gibi Wide-Column sistemlerde 'İstediğim sütuna göre sonradan SELECT WHERE çekerim' diyemezsiniz! Cassandra'da sorgular Partition Key ve Clustering Key'e göre KATI bir şekilde önceden modellenmelidir; sorgu desenini bilmeden tablo tasarlanamaz.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

In Cassandra, you cannot run arbitrary ad-hoc queries with WHERE clauses on non-key columns without table scans. In Wide-Column stores, table schemas are rigidly designed around explicit query access patterns.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Key-Value basit nesneler ve oturumlar için en hızlı NoSQL türüdür.

• Cassandra LSM-Tree yapısı sayesinde saniyede milyonlarca yazmayı kayıpsız yutar.

• Sosyal ağ ve dolandırıcılık tespiti gibi ilişkisel ağlarda Graph DB rakipsizdir.

🎯 Key Takeaways for the Interview

• Key-Value stores offer unmatched speed for simple session and payload caches.

• Cassandra's append-only LSM-Tree architecture excels at high-velocity writes.

• Graph databases conquer multi-hop relationship pathfinding with zero JOIN degradation.

21

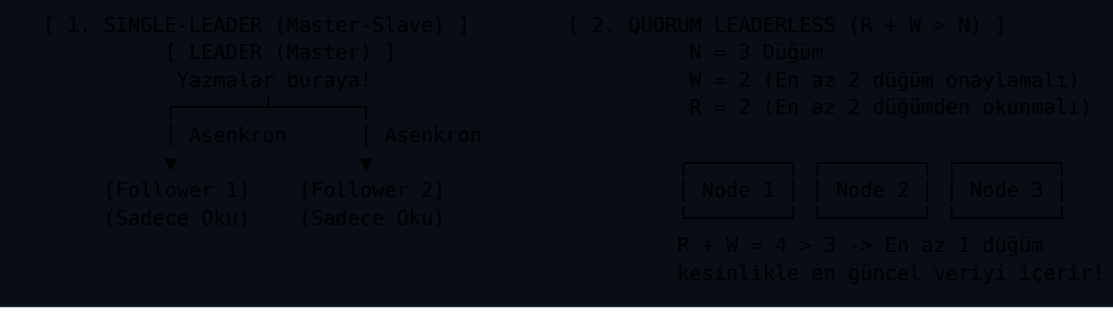
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı Replikasyonu: Single-Leader, Multi-Leader ve Quorum (R + W > N)

Replication Topologies: Single-Leader, Multi-Leader & Quorum (R+W > N)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Single-Leader replikasyon, sınıfta tek bir öğretmenin (Leader/Master) konuşması ve 30 öğrencinin (Followers/Replicas) öğretmeni defterine kopyalaması gibidir. Herkes soru sorup okuyabilir (Okuma ölçeklenir), ama tahtaya sadece öğretmen yazar. Öğretmen bayılırsa öğrencilerden biri yeni öğretmen seçilir (Failover). Quorum (Lidersiz) ise 3 kişilik bir jüri heyetidir: Bir kararın geçerli olması için en az 2 üyenin 'Evet' demesi (Write Quorum: W=2) ve okurken en az 2 üyeye danışılması (Read Quorum: R=2) şarttır. İki çoğunluk kümesi mutlaka en az 1 kişide kesişeceği için doğru bilgi asla kaybolmaz!

🇬🇧 ENGLISH (ELI5)

Single-Leader replication is a lecture hall where only the professor (Leader/Master) speaks and 30 students (Followers/Replicas) transcribe notes into their notebooks. Anyone can read, but only the professor writes. If the professor faints, a student is elected the new professor (Failover). Leaderless Quorum (Dynamo/Cassandra) is a 3-judge jury: to confirm a fact, at least 2 judges must sign off (W=2), and to read, at least 2 judges must be polled (R=2). Because two majorities mathematically overlap, you are guaranteed to read the latest truth!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Single-Leader (Primary-Secondary): Tüm yazmalar Leader düğüme gider; Leader değişiklikleri replikasyon logu (WAL) ile follower'lara asenkron veya yarı-senkron aktarır. Okuma trafiği follower'lara dağıtılarak ölçeklenir; ancak 'Replication Lag' (gecikme) nedeniyle 'Kendi Yazdığını Okuyamama' (Read-Your-Own-Writes) sorunu oluşabilir. 2) Multi-Leader: Birden fazla veri merkezinde yazma kabul edilir; ancak eşzamanlı çakışmalar (Write Conflicts) çözülmelidir (Last-Write-Wins veya CRDT). 3) Leaderless (Dynamo-Style Quorum): Lider yoktur; istemci doğrudan N düğüme yazar ve okur. Quorum Kuralı: $R + W > N$. Eğer 3 replikalı bir kümede $W=2$ ve $R=2$ ise, okunan 2 düğümden en az biri en son yazmayı görmüş olmak zorundadır (Version Vector ile en günceli seçilir).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Single-Leader (Active-Passive): Writes direct strictly to the leader, streaming mutations via Write-Ahead Logs (WAL) to read-replicas. Horizontally scales read bandwidth, but asynchronous replication introduces replication lag and stale reads (mitigated via read-your-own-writes session sticky routing to leader). 2) Multi-Leader (Active-Active): Multiple datacenters accept local writes, requiring conflict resolution strategies like Last-Write-Wins (clock-skew sensitive) or CRDTs. 3) Leaderless Quorum (Amazon Dynamo, Cassandra): Clients coordinate writes across N nodes. The Quorum Equation: $R + W > N$. If N=3, W=2, and R=2, the read set and write set must overlap by at least one node, guaranteeing strong consistency via read-repair or hinted handoff.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Quorum Okuma ve 'En Güncel Versiyonu' Seçme Motoru
import java.util.List;

public class QuorumReadCoordinator {
    public record VersionedValue(String value, long versionTimestamp) {}

    public VersionedValue resolveQuorumRead(List<VersionedValue> responses) {
        // En yüksek zaman damgasına sahip veriyi seç (Last-Write-Wins)
        return responses.stream()
                .max((a, b) -> Long.compare(a.versionTimestamp(), b.versionTimestamp()))
                .orElseThrow();
    }
}

void main() {
    var coordinator = new QuorumReadCoordinator();
    // 3 Düğümden okunan 2 cevap (R=2):
    var node1 = new QuorumReadCoordinator.VersionedValue("User: Ali Bozkurt (Eski)", 1000L);
    var node2 = new QuorumReadCoordinator.VersionedValue("User: Ali Turgut Bozkurt (Yeni)", 1005L);

    var resolved = coordinator.resolveQuorumRead(List.of(node1, node2));
    IO.println("Quorum Çözümlenen Değer: " + resolved.value() + " [Versiyon: " +
resolved.versionTimestamp() + "]");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Asenkron replikasyonda 'Leader çöktüğünde veri kaybolmaz' demeyin! Asenkron replikasyonda Leader'a yazılan ancak henüz Follower'a iletilmeyen son işlemler Leader çökerse sonsuza kadar buharlaşır (Data Loss). Mülakatta yarı-senkron replikasyon veya Raft konsensüsü alternatiflerini tartışın.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

In asynchronous replication, if the leader crashes before WAL packets arrive at replicas, committed writes are permanently lost. Explicitly call out this failover data loss window and present semi-synchronous replication or Raft consensus as hardening options.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Single-Leader okuma ağırlıklı sistemlerde okumayı follower'lara yayarak ölçekler.
- R + W > N quorum formülü lidersiz sistemlerde güçlü tutarlılık sağlar.
- Replication Lag, kullanıcının kendi yaptığı güncellemeyi hemen görememesine yol açabilir.

🎯 Key Takeaways for the Interview

- Single-Leader topologies scale read-heavy workloads across fleets of replicas.
- The quorum condition R + W > N ensures read and write sets overlap on at least one fresh node.
- Replication lag causes stale read anomalies unless read-after-write consistency is enforced.

22

BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı Bölümleme (Sharding) ve Yatay Parçalama: Range, Hash ve Directory

Database Sharding & Horizontal Partitioning: Range, Hash & Directory

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ 1. RANGE-BASED SHARDING (Aralık Tabanlı) ]
Shard 1: Kullanıcı A - H  |  Shard 2: Kullanıcı I - P  |  Shard 3: Kullanıcı Q - Z
(Sıralı sorgular hızlıdır; ancak 'J' harfi aşırı popülerse Shard 2 patlar - Hotspot!)

[ 2. HASH-BASED SHARDING (Karma Tabanlı) ]
MurmurHash3(User_ID) % 4
Key: "usr_101" ⟶ Hash: 3948192 ⟶ % 4 = Shard 0
Key: "usr_102" ⟶ Hash: 8192841 ⟶ % 4 = Shard 1
(Veriyi homojen dağıtır; ancak aralık sorguları (Range Query) imkansızlaşır!)
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Devasa bir kütüphanedeki 10 milyon kitabı tek bir odaya sığdıramadığınızda ne yaparsınız? Kütüphaneyi 3 odaya bölersiniz (Sharding): 1) Range Sharding (Harfe Göre): 1. odaya A-H arası yazarlar, 2. odaya I-P arası konur. Düzenlidir ama o sene 'Harry Potter' patlarsa H odasının kapısında izdiham çıkar (Hotspot), diğer odalar bomboş kalır! 2) Hash Sharding (Rastgele Matematiksel Dağıtım): Kitabın adını matematiksel bir mikserden (Hash fonksiyonu) geçirir ve odalara eşit dağıtırsınız. Odaların yükü dengelenir ama 'Bana 1990'lardaki tüm kitapları getir' derseniz 3 odayı da tek tek aramak zorunda kalırsınız!

🇬🇧 ENGLISH (ELI5)

When 10 million books overflow a library, you split them across 3 separate wings (Sharding): 1) Range Sharding: Wing 1 takes authors A-H, Wing 2 takes I-P, Wing 3 takes Q-Z. It keeps ranges alphabetical, but if J.K. Rowling blows up, Wing 2 is crushed under mob traffic (Hotspotting) while Wing 3 is dead empty! 2) Hash Sharding: You run the book ISBN through a mathematical blender (Hash function) and scatter copies evenly across all 3 wings. Traffic distributes uniformly, but executing a range query ('Find all books written between 1990 and 1995') forces you to search every single room!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Sharding (Yatay Bölümleme), tek bir veritabanının kaldıramayacağı büyüklükteki verileri ve QPS'i bağımsız veritabanı düğümlerine (Shard) bölmektir. Yöntemler: 1) Range-Based Sharding: ID veya tarihe göre aralıklarla bölme (ör. 1-100K Shard 1, 100K-200K Shard 2). Aralık sorguları (Range Scan) verimlidir; ancak sıralı ID'lerde en son shard sürekli sıcak kalır (Hotspot). 2) Hash-Based (Key-Based) Sharding: `Hash(ShardKey) % ShardSayısı`. Veriyi homojen dağıtır, hotspot'ları engeller; ancak shard sayısı değiştiğinde yeniden veri taşıma (Resharding) felaketi yaşanır (Bkz: Consistent Hashing). 3) Directory-Based: Merkezi bir arama tablosunda hangi anahtarın hangi shard'da olduğu tutulur; esnektir ama tablo tek hata noktası (SPOF) olabilir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Sharding partitions massive datasets across autonomous database instances. Strategies: 1) Range-Based Sharding: Allocates discrete contiguous intervals (e.g. A-F, G-M, or date brackets). Excellent for range scans, but susceptible to non-uniform data skew and write hotspots on append-only primary keys. 2) Hash-Based Sharding: Computes `hash(shard_key) % num_shards`. Guarantees uniform distribution and squashes hotspots, but ruins range queries (requires scatter-gather across all shards) and triggers massive rebalancing overhead when expanding nodes. 3) Directory-Based Sharding: Uses a centralized lookup service mapping entity IDs to target shards; highly adaptable but introduces an operational lookup hop and potential bottleneck.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Hash Tabanlı Sharding Yönlendirici (Shard Router)
public class ShardRouter {
    private final int numberOfShards;

    public ShardRouter(int numberOfShards) {
        this.numberOfShards = numberOfShards;
    }

    public int resolveShardId(String shardKey) {
        // MurmurHash benzeri dağıtık pozitif hash simülasyonu
        int hash = shardKey.hashCode() & 0x7fffffff;
        return hash % numberOfShards;
    }
}

void main() {
    var router = new ShardRouter(4); // 4 Veritabanı Parçası (Shard 0, 1, 2, 3)
    IO.println("user_1001 -> Shard #" + router.resolveShardId("user_1001"));
    IO.println("user_1002 -> Shard #" + router.resolveShardId("user_1002"));
    IO.println("user_1003 -> Shard #" + router.resolveShardId("user_1003"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Sharding'in 3 büyük bedeli vardır: 1) Cross-shard JOIN imkansızlaşır (uygulama katmanında yapılmalıdır). 2) Dağıtık işlemler (Distributed Transactions / 2PC) korkunç yavaştır. 3) Shard Rebalancing (yeni makine ekleyip veriyi taşımak) operasyonel bir kabustur. Bu bedelleri mülakatta açıkça sayın!

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Articulate the 3 heavy penalties of sharding: 1) Cross-shard joins become computationally impossible. 2) Cross-shard transactions require agonizingly slow Two-Phase Commits (2PC). 3) Resharding data across new nodes is an operational minefield.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sharding, tek bir makinenin disk ve IO limitleri aşıldığında zorunlu son çaredir.

• Hash-based sharding veriyi eşit dağıtır ancak aralık sorgularını (range query) zorlaştırır.

• Shard Key seçimi (kullanıcı ID vs şirket ID) mimarinin başarısını belirleyen en kritik karardır.

🎯 Key Takeaways for the Interview

• Sharding is the ultimate architectural frontier when single-node IO limits are exhausted.

• Hash sharding prevents write skew at the cost of requiring scatter-gather range queries.

• Selecting the Shard Key determines query locality and system longevity.

23

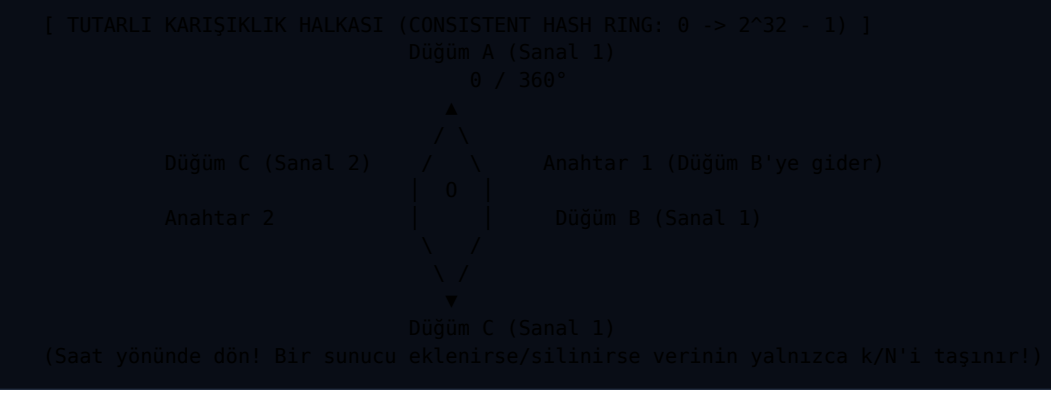
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Tutarlı Karışıklık (Consistent Hashing) & Sanal Düğümler (Virtual Nodes)

Consistent Hashing & Virtual Nodes: The Elastic Scaling Ring

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Geleneksel modülo aritmetiğinde (`Hash % N`), 4 sunucunuz varken 5. sunucuyu eklerseniz sınıftaki herkesin sırası değişir; tüm önbellek geçersiz kalır ve veritabanı çöker! Consistent Hashing ise yuvarlak bir yemek masası gibidir (0'dan 360 dereceye bir çember). Sunucuları masanın etrafına rastgele oturtursunuz. Yeni gelen bir yemeği masanın üzerine koyup saat yönünde döndürürsünüz; yemeği ilk gören sunucu onu afiyetle yer! Masaya yeni bir sandalye koysanız bile sadece o sandalyenin hemen yanındaki birkaç yemek yer değiştirir; geri kalan herkes yemeğini yemeye devam eder!

🇬🇧 ENGLISH (ELI5)

Under naive modular hashing (`hash % N`), expanding from 4 to 5 servers scrambles the destination of 80% of all keys, triggering a total cache wipeout! Consistent Hashing is an infinite circular conference table (0 to 2^32 - 1). You position servers at discrete degrees around the circle. To store a key, hash it to a point on the circle and walk clockwise: the first server node you bump into claims the key! If a new server pulls up a chair, only its immediate neighbor hands over a sliver of keys; the remaining 80% of data stays completely untouched!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Consistent Hashing (Karger ve arkadaşları, MIT 1997), dağıtık önbellek (Memcached, Akamai CDN) ve NoSQL (Amazon DynamoDB, Apache Cassandra) mimarilerinin kalbidir. Avantajı: Kümedeki düğüm sayısı N değiştiğinde, taşınması gereken anahtar sayısı geleneksel mimarideki gibi %100 değil, sadece ortalama k/N kadardır (k = toplam anahtar). Sanal Düğümler (Virtual Nodes / Vnodes): Fiziksel bir sunucunun halka üzerinde tek bir noktada durması 'Hotspot' ve dengesiz yük dağılımına yol açar. Çözüm: Her fiziksel sunucuya halka üzerinde 100-256 adet sanal etiket atanır (`NodeA-1`, `NodeA-2`...). Bu sayede yük çember boyunca mükemmel bir homojenlikle dağılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Consistent Hashing maps both servers and data keys onto a continuous hash ring (typically 0 to 2^32 - 1). Keys route clockwise to the first encountered server token. When node scaling occurs, only k/N keys migrate (k = total keys, N = server count), avoiding mass cache evictions. The Virtual Nodes (Vnodes) Innovation: A physical server represented by a single token risks severe load skew due to random clustering. By projecting each physical node into 100-256 virtual points across the ring perimeter, keys distribute uniformly across hardware proportional to host capacity.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Sanal Düğümlü (Vnodes) Consistent Hash Halkası Implementasyonu
import java.util.*;

public class ConsistentHashRing<T> {
    private final int numberOfReplicas; // Sanal düğüm sayısı
    private final SortedMap<Integer, T> circle = new TreeMap<>();

    public ConsistentHashRing(int numberOfReplicas, Collection<T> nodes) {
        this.numberOfReplicas = numberOfReplicas;
        for (T node : nodes) addNode(node);
    }

    public void addNode(T node) {
        for (int i = 0; i < numberOfReplicas; i++) {
            circle.put(hash(node.toString() + "-vnode-" + i), node);
        }
    }

    public T getNode(String key) {
        if (circle.isEmpty()) return null;
        int hash = hash(key);
        if (!circle.containsKey(hash)) {
            SortedMap<Integer, T> tailMap = circle.tailMap(hash);
            hash = tailMap.isEmpty() ? circle.firstKey() : tailMap.firstKey(); // Saat yönünde dön
        }
        return circle.get(hash);
    }

    private int hash(String val) {
        return val.hashCode() & 0x7fffffff;
    }
}

void main() {
    var ring = new ConsistentHashRing<String>(3, List.of("Server-A", "Server-B", "Server-C"));
    IO.println("Key 'user_9912' Hangi Sunucuda? -> " + ring.getNode("user_9912"));
    IO.println("Key 'order_441' Hangi Sunucuda?  -> " + ring.getNode("order_441"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta sanal düğümleri (Virtual Nodes) anlatmayı unutursanız mülakatçı 'Düğümler halkaya rastgele düşerse biri aşırı yüklenip çökmez mi?' diye soracaktır. Sanal düğümlerin varyansı düşürüp yükü homojenleştirdiğini doğrudan siz söyleyin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Failing to proactively propose Virtual Nodes (Vnodes) invites the interviewer to challenge you on hash skew. Preemptively demonstrate that virtual tokens equalize ring dispersion and reflect asymmetric server hardware.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Consistent Hashing, sunucu ekleme/çıkarmada veri taşıma maliyetini k/N seviyesine indirir.
- Sanal Düğümler (Virtual Nodes) çember üzerindeki yükü mükemmel dengeler ve hotspot'u önler.
- DynamoDB, Cassandra ve Discord gibi devasa dağıtık sistemlerin temel omurgasıdır.

🎯 Key Takeaways for the Interview

- Consistent Hashing bounds key migration to k/N upon node churn, preserving cache stability.
- Virtual Nodes neutralize non-uniform distribution skew across heterogeneous machines.
- Serves as the structural backbone of Amazon DynamoDB, Apache Cassandra, and Discord.

24

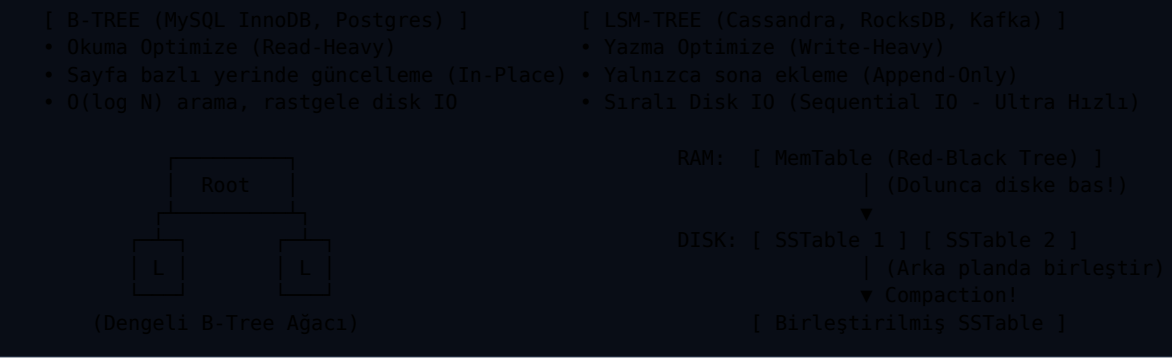
BÖLÜM: VERITABANI MIMARISI, BÖLÜMLEME VE REPLIKASYON | MODULE: DATABASE ARCHITECTURE, SHARDING & REPLICATION TOPOLOGIES

Veritabanı İndeksleme Sırları: B-Tree vs LSM-Tree (Log-Structured Merge-Tree)

Database Indexing Under the Hood: B-Tree vs LSM-Tree

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

B-Tree, kalın ve çok düzenli bir kütüphane indeks fihristi gibidir: Yeni bir kitap geldiğinde fihristin ilgili sayfasına gidip araya sıkıştırarak yazarsınız (Yerinde Güncelleme - In-Place Update). Okurken harikadır, aradığınızı saniyesinde bulursunuz; ama çok fazla yeni kitap gelirse sayfaları silip yeniden düzenlemekten kütüphanecinin eli yorulur. LSM-Tree ise seri bir günlük yazarı gibidir: Asla eski sayfaları silmez! Ne olursa olsun günlüğün en sonuna yazar (Append-Only). Yazmak ışık hızındadır! Akşam olunca da arkada oturup günlükteki tekrarları temize çeker (Compaction)!

🇬🇧 ENGLISH (ELI5)

A B-Tree is a meticulously maintained card catalog in a library: When a new book arrives, the librarian navigates to the exact drawer and slots the card into alphabetical position (In-Place mutation). Reading is instantaneous ($O(log N)$), but heavy writes force costly random disk seeks. An LSM-Tree (Log-Structured Merge-Tree) is a court stenographer typing at 200 words/minute: They NEVER erase or rewrite past pages; they append every new event strictly to the end of the roll (Sequential Disk IO). Later, background janitors compress and deduplicate old logs into sorted tables (Compaction)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) B-Tree / B+Tree: İlişkisel veritabanlarının (MySQL InnoDB, PostgreSQL) kalbidir. Veri 16 KB'lık sayfalarda (Pages) dengeli bir ağaç yapısında tutulur. Sayfalar doğrudan yerinde (In-Place) güncellenir. Rastgele disk IO gerektirdiği için yazma maliyeti yüksektir; ancak okumalar tek bir sayfa aramasıyla tamamlanır ($O(log N)$). 2) LSM-Tree: Modern NoSQL ve depolama motorlarının (RocksDB, Cassandra, LevelDB) kalbidir. Yazmalar önce RAM'deki sıralı bir ağaca (`MemTable`) ve kurtarma günlüğüne (`WAL`) yazılır (sıfır rastgele disk IO!). MemTable dolunca diske sıralı ve değiştirilemez (Immutable) `SSTable` (Sorted String Table) olarak dökülür. Okuma yaparken: MemTable -> Bloom Filter -> SSTable dosyaları taranır. Arka planda `Compaction` işlemi eski sürümleri temizler.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) B-Tree / B+Tree: The classic relational storage engine (Postgres, InnoDB). Organizes fixed-size pages (typically 16KB) in a balanced tree. Updates perform random in-place page writes. Read paths are lightning fast ($O(log N)$ disk seeks), but high-velocity random writes fragment storage and cause heavy write amplification. 2) LSM-Tree (Log-Structured Merge-Tree): Powers modern high-throughput NoSQL engines (Cassandra, RocksDB, ClickHouse). Writes append sequentially to an in-memory sorted tree (`MemTable`) and a durable WAL, converting random writes into maximum-speed sequential IO. Saturated MemTables flush immutably to disk as `SSTables`. Background merge-compaction processes consolidate fragmented segments and evict deleted tombstones.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: LSM-Tree MemTable & SSTable Döküm Mantığı Simülasyonu
import java.util.*;

public class LsmTreeSim {
    // RAM'deki Sıralı Bellek Tablosu (MemTable)
    private final NavigableMap<String, String> memTable = new TreeMap<>();
    private final List<Map<String, String>> ssTablesOnDisk = new ArrayList<>();
    private static final int MEMTABLE_THRESHOLD = 3;

    public synchronized void put(String key, String value) {
        memTable.put(key, value); // RAM'e sıralı yaz (O(log N))
        if (memTable.size() >= MEMTABLE_THRESHOLD) {
            flushToSsTable();
        }
    }

    private void flushToSsTable() {
        // Diske sıralı, değiştirilemez SSTable olarak dök:
        ssTablesOnDisk.add(new HashMap<>(memTable));
        memTable.clear();
        IO.println(">>> MemTable Doldu! Diske SSTable döküldü (Sequential Disk IO). Toplam SSTable: "
+ ssTablesOnDisk.size());
    }

    public synchronized String get(String key) {
        // 1. Önce RAM'deki MemTable'a bak:
        if (memTable.containsKey(key)) return memTable.get(key);
        // 2. Bulunamazsa en yeni SSTable'dan eskiye doğru diski tara:
        for (int i = ssTablesOnDisk.size() - 1; i >= 0; i--) {
            if (ssTablesOnDisk.get(i).containsKey(key)) return ssTablesOnDisk.get(i).get(key);
        }
        return null;
    }
}

void main() {
    var lsm = new LsmTreeSim();
    lsm.put("user_1", "Ali");
    lsm.put("user_2", "Bob");
    lsm.put("user_3", "Can"); // Eşik doldu -> SSTable diske basıldı!
    lsm.put("user_4", "Deniz");

    IO.println("Sorgu 'user_1': " + lsm.get("user_1"));
    IO.println("Sorgu 'user_4': " + lsm.get("user_4"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

LSM-Tree'lerin zayıf karnı 'Read Amplification' (Okuma Büyümesi) ve 'Compaction' sırasında yaşanan disk IO darboğazlarıdır. Mülakatta bu dezavantajı gidermek için SSTable dosyalarının önüne mutlaka Bloom Filter konulduğunu belirtin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

LSM-Trees suffer from read amplification (checking multiple SSTable files on misses) and compaction IO storms. Explicitly state that pairing Bloom Filters with SSTables allows skipping unnecessary disk reads in 99% of queries.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• B-Tree okuma ağırlıklı sistemler (RDBMS) için, LSM-Tree ise yazma canavarları (NoSQL) içindir.

• LSM-Tree sıralı disk yazması (Sequential IO) kullanarak mekanik ve SSD diskleri son hızda sürer.

• Bloom Filter ve Compaction, LSM-Tree mimarisinin okuma performansını koruyan iki emniyet sibobudur.

🎯 Key Takeaways for the Interview

• B-Trees power read-heavy RDBMS; LSM-Trees dominate extreme write-heavy workloads.

• LSM-Trees convert random writes into blazing sequential append IO.

• Bloom Filters and background Compaction keep LSM-Tree read latencies tightly bounded.

BÖLÜM 5 / MODULE 5

Asenkron İletişim, Mesajlaşma ve Olay Tabanlı Mimari

Asynchronous Messaging & Event-Driven Architecture

Zamansal ayrışma, RabbitMQ vs Apache Kafka, Kafka partition ve consumer grupları, Exactly-Once garantileri, Idempotency ve Transactional Outbox deseni.

Temporal decoupling, RabbitMQ vs Apache Kafka, Kafka commit logs & consumer groups, Exactly-Once semantics, Idempotency tokens, and the Transactional Outbox pattern.

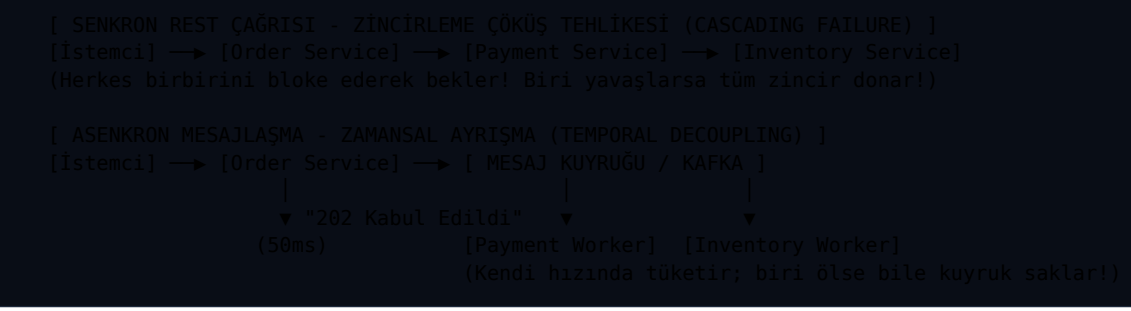
25

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Senkron vs Asenkron İletişim: Zamansal Ayrışma (Temporal Decoupling)

Synchronous vs Asynchronous: Temporal Decoupling & Buffering

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Senkron iletişim, arkadaşınızın evine gidip kapısını çalmak ve kapıyı açana kadar karda kışta 2 saat kapının önünde dikilmektir; arkadaşınız evde yoksa işiniz mahvolur! Asenkron iletişim ise hediyeyi arkadaşınızın posta kutusuna (Mesaj Kuyruğu) bırakıp yolunuza devam etmektir. Arkadaşınız isterse 3 saat sonra, isterse ertesi gün eve gelsin; mektubu kutudan alır ve okur. Sistemlerinizin birbirinin ayakta olmasına bağımlı olmadan çalışması (Zamansal Ayrışma) işte bu posta kutusu sayesindedir!

🇬🇧 ENGLISH (ELI5)

Synchronous communication is ringing your friend's doorbell and standing frozen in a blizzard for 2 hours waiting for them to open up; if they aren't home, you freeze to death! Asynchronous communication is dropping a parcel into their locked mailbox (Message Queue) and going about your business. Whether your friend gets home in 10 minutes or tomorrow morning, the parcel sits safe in the mailbox. Decoupling services in time (Temporal Decoupling) ensures no service halts waiting for another!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Senkron (HTTP REST, gRPC) iletişimde çağıran servis yanıt gelene kadar bir işletim sistemi iş parçacığını (thread) ve soketini bloke eder. Dağıtık mimarilerde A -> B -> C -> D şeklindeki senkron çağrı zincirleri 'Cascading Failure' (zincirleme çöküş) yaratır; D servisi 5 saniye gecikirse A'nın tüm thread havuzu tükenir. Asenkron iletişim (Message Queues) zamansal ayrışma (Temporal Decoupling) sağlar: 1) İstemci isteği kuyruğa atıp anında 'HTTP 202 Accepted' alır. 2) Kuyruk bir tampon (Shock Absorber) görevi görerek ani trafik patlamalarını emer. 3) Tüketici servisler çökseler bile yeniden başladıklarında kuyrukta biriken mesajları kaldıkları yerden işlemeye devam eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Synchronous invocations (REST, gRPC) hold operating system threads and network sockets hostage until upstream downstream services respond. In deep call chains (A -> B -> C -> D), a latency spike in service D exhausts thread pools across all upstream hosts, triggering catastrophic cascading outages. Asynchronous messaging decouples producers and consumers temporally: 1) Producers publish to durable message brokers and return HTTP 202 immediately. 2) Queues act as shock absorbers buffering flash traffic spikes. 3) Downstream worker failure causes zero data loss; consumers catch up monotonically upon restart.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Asenkron İş Kuyruğu ve İşçi Modeli (Virtual Threads Destekli)
import java.util.concurrent.*;

public class AsyncJobQueue {
    public record OrderJob(String orderId, double amount, long timestamp) {}

    private final BlockingQueue<OrderJob> queue = new LinkedBlockingQueue<>(1000);

    public boolean publishJob(OrderJob job) {
        return queue.offer(job); // Kuyruk dolmadığı sürece bloke etmeden at
    }

    public void startWorker() {
        Thread.ofVirtual().start(() -> {
            while (true) {
                try {
                    OrderJob job = queue.take(); // Mesaj gelene kadar bekle
                    IO.println("İşlendi: Sipariş #" + job.orderId() + ", Tutar: $" + job.amount());
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    break;
                }
            }
        });
    }
}

void main() throws InterruptedException {
    var jobQueue = new AsyncJobQueue();
    jobQueue.startWorker();

    jobQueue.publishJob(new AsyncJobQueue.OrderJob("ORD_991", 149.99, System.currentTimeMillis()));
    Thread.sleep(50); // Sanal iş parçacığının işlemesini bekle
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Asenkron mimarinin bedeli 'Eventual Consistency' (Nihai Tutarlılık) ve karmaşık hata yönetimidir. Kullanıcıya 'Siparişiniz alındı, hazırlanıyor' mesajı vermeniz gerekir; kullanıcı anında faturasını göremeyebilir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

The trade-off of asynchronous messaging is eventual consistency and complex failure compensation. The UI must adapt to an asynchronous workflow ('Your order is processing') rather than assuming instantaneous transactional guarantees.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Asenkron iletişim servisler arasındaki zamansal ve mekansal bağımlılığı koparır.
- Mesaj aracıları ani trafik dalgalanmalarında doğal bir amortisör (shock absorber) görevi görür.
- Senkron çağrı zincirleri yerine asenkron olaylar kullanmak zincirleme çöküşleri engeller.

🎯 Key Takeaways for the Interview

- Asynchronous communication eliminates temporal and spatial coupling across services.
- Message queues act as elastic buffers absorbing sudden traffic spikes without dropping requests.
- Replacing synchronous call chains with asynchronous queues prevents cascading system outages.

26

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Mesaj Kuyrukları vs Olay Akışları: RabbitMQ vs Apache Kafka

Message Queues vs Event Streams: RabbitMQ vs Apache Kafka

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ RABBITMQ (Akıllı Broker, Aptal Tüketici - Smart Broker, Dumb Consumer) ]
Mesajlar kuyruğa gelir ──► Tüketici okur ve ACK atar ──► Mesaj kuyruktan SİLİNİR!
(İş kuyrukları, karmaşık yönlendirme - Topic/Header Exchange, RPC için idealdir)

[ APACHE KAFKA (Aptal Broker, Akıllı Tüketici - Dumb Broker, Smart Consumer) ]
Mesajlar sıralı bir kütüğe (Commit Log) yazılır ──► Asla silinmez (Günlerce durur!)
Tüketici kendi okuma noktasını (Offset) yönetir ──► İsteyen başa sarıp tekrar okur!
(Yüksek hacimli olay akışı, metrikler, Event Sourcing, saniyede milyonlarca QPS)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

RabbitMQ bir postane kuryesidir: Çantasından mektubu çıkarır, kapınızı çalar, elinize verir ve imzayı aldıktan sonra (ACK) o mektubu sistemden tamamen siler. Mektup bir daha geri gelmez. Apache Kafka ise gazete arşividir: Yazılan tüm haberler devasa ciltli bir deftere (Append-Only Commit Log) sırayla yapıştırılır ve ASLA yırtılıp atılmaz! Siz bir araştırmacı olarak defterin 50. sayfasına kadar okuyabilirsiniz (Offset); arkadaşınız ise 10. sayfada olabilir. Hatta isterseniz 1. sayfaya geri dönüp tüm yılı baştan sona tekrar okuyabilirsiniz (Replayability)!

🇬🇧 ENGLISH (ELI5)

RabbitMQ is a specialized courier: It pulls a letter from its satchel, rings your doorbell, hands it to you, collects your signature (ACK), and deletes the letter from the system forever. Apache Kafka is a bound newspaper archive: every article published is permanently glued onto an immutable, sequential commit log and NEVER discarded! You can read up to page 50 (Offset), while a colleague reads at page 10. You can even rewind your bookmark back to page 1 and reread an entire decade of history from scratch (Replayability)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) RabbitMQ (AMQP): Mesaj odaklıdır. Akıllı yönlendirme (Exchange tipleri: Direct, Fanout, Topic, Headers). Mesaj tüketici tarafından onaylandığında (ACK) broker'dan temizlenir. Düşük gecikmeli, karmaşık kurallı iş kuyrukları ve bildirimler için mükemmeldir; ancak geriye dönük yeniden okuma (Replay) desteklemez. 2) Apache Kafka: Dağıtık, append-only commit-log mimarisidir. Broker mesajı silmez; mesajlar retention süresi (ör. 7 gün) boyunca diskte sıralı tutulur. Tüketiciler kendi okuma indekslerini (`Offset`) yönetir. Sıfır Kopyalama (Zero-Copy OS sendfile) tekniği sayesinde saniyede milyonlarca olayı (GB/sn bant genişliği) kayıpsız işler. Event Sourcing, CQRS ve analitik akışlar için MAANG standardıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) RabbitMQ (AMQP): Message-oriented broker with sophisticated routing topologies (Exchange bindings: Direct, Topic, Fanout, Headers). Once acknowledged, messages are expunged from queue memory. Tailored for discrete background jobs, task queues, and low-latency notifications; lacks historical replayability. 2) Apache Kafka: Distributed, append-only, partitioned commit log. Messages persist immutably regardless of consumer state according to retention policies (e.g. 7 days or infinite compaction). Consumers advance localized offsets independently. Utilizing Linux kernel zero-copy (`sendfile` bypassing user-space memory buffers) and batch compression, Kafka handles millions of events/sec at multi-gigabyte throughput. De facto backbone for event-driven streaming and audit logging.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Kafka Offset Tabanlı Tüketim Modeli Simülasyonu
import java.util.*;

public class KafkaCommitLogSim {
    private final List<String> commitLog = new ArrayList<>();

    public synchronized long appendEvent(String event) {
        commitLog.add(event);
        return commitLog.size() - 1; // Üretilen Offset değeri
    }

    // Tüketici kendi offset'i ile istediği yerden okur (Replay yeteneği!)
    public synchronized List<String> pollEvents(long fromOffset, int batchSize) {
        List<String> results = new ArrayList<>();
        for (long i = fromOffset; i < commitLog.size() && results.size() < batchSize; i++) {
            results.add(commitLog.get((int) i));
        }
        return results;
    }
}

void main() {
    var kafka = new KafkaCommitLogSim();
    kafka.appendEvent("ORDER_CREATED_#1");
    kafka.appendEvent("ORDER_PAID_#1");
    kafka.appendEvent("ORDER_SHIPPED_#1");

    // Tüketici A 0. offset'ten okuyor:
    IO.println("Consumer A (Tüm Tarihçe): " + kafka.pollEvents(0, 10));
    // Tüketici B sadece 2. offset'ten sonrasını okuyor:
    IO.println("Consumer B (Sadece Kargo): " + kafka.pollEvents(2, 10));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Kafka'yı basit bir e-posta gönderme kuyruğu için kullanırım' demeyin! Küçük ölçekli, basit iş kuyruklarında Kafka'nın partition ve consumer group karmaşıklığı aşırı mühendisliktir (over-engineering); basit kuyruklar için RabbitMQ veya AWS SQS daha uygundur.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not recommend Kafka for trivial background email dispatch queues. Operating Kafka clusters incurs ZooKeeper/KRaft and partition management complexity. For simple task dispatching, propose RabbitMQ or AWS SQS.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- RabbitMQ mesajı işlendikten sonra siler; akıllı yönlendirme ve iş kuyrukları için idealdir.
- Kafka mesajları kütükte (log) saklar; geriye dönük tekrar oynatma (replayability) sunar.
- Zero-copy disk transferi sayesinde Kafka saniyede milyonlarca olayı minimum CPU ile taşır.

🎯 Key Takeaways for the Interview

- RabbitMQ expunges messages upon delivery; ideal for complex routing and job queues.
- Kafka stores immutable commit logs, allowing consumers to rewind and reprocess history.
- Zero-copy OS primitives allow Kafka to stream millions of events/sec at bare-metal speeds.

27

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Apache Kafka İç Mimarisi: Topics, Partitions, Consumer Groups ve Segmentler

Apache Kafka Internals: Topics, Partitions, Consumer Groups & Segments

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ KAFKA TOPIC: "orders" (3 PARTITION'A BÖLÜNMÜŞ) ]
Partition 0: [0][1][2][3][4] ──► Consumer 1 (Pod A)
Partition 1: [0][1][2][3]    ──► Consumer 2 (Pod B)   Consumer Group: "Billing-Service"
Partition 2: [0][1][2]       ──► Consumer 3 (Pod C)
(Her partition yalnızca TEK bir consumer tarafından okunur -> Paralellik & Sıralılık!)

[ PARTITION DISK YAPISI: .log ve .index DOSYALARI ]
Segment 000000.log  (1 GB Dolunca kapanır) ──► Segment 000001.log (Yeni aktif kütük)
Segment 000000.index (Seyrek İndeks - Sparse Index: Binary Search ile O(log N) arama)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kafka Topic'ini 3 şeritli bir otoyol (3 Partition) olarak düşünün. Eğer tek şerit olsaydı tüm arabalar tek sıra halinde giderdi ve trafik tıkanırdı. 3 şerit açarak trafiği 3 katına çıkarırsınız! Şeritlerin çıkışına da 3 kişilik bir gişe görevlisi ekibi (Consumer Group) koyarsınız: Her görevli yalnızca KENDİ şeridini izler. Arabalar kendi şeritlerinde asla birbirini sollayamaz (Şerit içinde kesin sıra garantisi!); ancak şeritler birbirini beklemez!

🇬🇧 ENGLISH (ELI5)

Think of a Kafka Topic as a 3-lane highway (3 Partitions). A single-lane road bottlenecks all cars into a single file crawl. Adding 3 lanes triples traffic throughput! At the highway exit, assign 3 toll clerks belonging to the same union (Consumer Group): each clerk exclusively monitors their dedicated lane. Within a single lane, cars cannot overtake one another (strict in-order sequence within a partition!); but different lanes progress in parallel!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Kafka mimarisinin temel yapı taşları: 1) Topic: Mantıksal veri akışı kategorisi. 2) Partition: Yatay ölçekleme ve paralellik birimidir. Mesajlar anahtarın (Key) hash değerine göre (`MurmurHash2(key) % partition_count`) ilgili bölüme yazılır. Aynı anahtara sahip tüm mesajlar KESİNLİKLE aynı partition'a gider ve sıralı işlenir. 3) Consumer Group: Bir grup içindeki tüketici sayısı partition sayısını aşamaz; her partition o gruptaki yalnızca TEK bir tüketiciye atanır. 4) Disk Segmentleri: Her partition diskte 1 GB'lık segment dosyalarına bölünür (`.log` ve seyrek indeks `.index`). İşletim sistemi sayfa önbelleği (PageCache) kullanılarak disk aramaları RAM hızına çıkarılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Kafka internal architecture: 1) Partitioning: The fundamental unit of scalability. Keyed messages route via `MurmurHash2(key) % num_partitions`, guaranteeing deterministic partition assignment and strict ordering for identical keys (e.g. all events for `user_101`). 2) Consumer Groups: Workload parallelization engine. A partition is consumed by at most one worker per consumer group; exceeding worker counts relative to partition counts leaves surplus consumers idle. 3) Storage Segments: Partitions split on disk into bounded segment files (typically 1GB `.log` alongside `.index` sparse indexes). Kafka relies heavily on the Linux OS PageCache rather than JVM heap memory, avoiding garbage collection pauses while maximizing sequential disk IO.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Kafka Partition Yönlendirme ve Consumer Group Eşleme Simülasyonu
import java.util.*;

public class KafkaPartitionRouter {
    private final int partitionCount;

    public KafkaPartitionRouter(int partitionCount) {
        this.partitionCount = partitionCount;
    }

    public int assignPartition(String messageKey) {
        if (messageKey == null) {
            return ThreadLocalRandom.current().nextInt(partitionCount); // Round-Robin
        }
        // Aynı anahtar DAİMA aynı partition'a gider!
        return Math.abs(messageKey.hashCode() % partitionCount);
    }
}

void main() {
    var router = new KafkaPartitionRouter(3); // 3 Partition'lı Topic
    IO.println("User 'A' Olayı -> Partition #" + router.assignPartition("user:A"));
    IO.println("User 'B' Olayı -> Partition #" + router.assignPartition("user:B"));
    IO.println("User 'A' İkinci Olayı -> Partition #" + router.assignPartition("user:A") + " (Sıra
Korundu!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Tüm sistem için global bir mesaj sırası garanti ederim' derseniz elenirsiniz! Kafka'da mesaj sırası GLOBAL DEĞİLDİR; sadece AYNI PARTITION içinde sıralıdır. Global sıra istiyorsanız Topic'i 1 Partition'a düşürmelisiniz; bu da yatay ölçeklemeyi yok eder.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim Kafka guarantees global ordering across an entire topic. Kafka strictly guarantees ordering ONLY WITHIN A SINGLE PARTITION. Global ordering requires restricting a topic to 1 single partition, which demolishes horizontal throughput.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kafka'da paralellik sınırı partition sayısı kadardır.
- Aynı varlığa ait mesajlar aynı Key ile gönderilerek sıralı işleme garantilenir.
- Kafka JVM heap yerine işletim sistemi PageCache'ini kullanarak GC duraklamalarını sıfırlar.

🎯 Key Takeaways for the Interview

- The partition count establishes the hard ceiling on consumer parallelism per consumer group.
- Passing consistent message keys guarantees partition-level ordering for state machines.
- Kafka relies on OS PageCache rather than JVM heap, neutralizing GC pauses entirely.

28

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Mesaj Teslimat Garantileri: At-Most-Once, At-Least-Once ve Exactly-Once (EOS)

Message Delivery Guarantees: At-Most-Once, At-Least-Once & EOS

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. AT-MOST-ONCE (En Fazla Bir Kez) ]
Mesajı al ──► Hemen ACK at ──► Mesajı İşle (Sunucu ÇÖKERSE -> MESAJ KAYBOLUR!)

[ 2. AT-LEAST-ONCE (En Az Bir Kez - Endüstri Standardı!) ]
Mesajı al ──► Mesajı İşle ──► DB'ye Yaz ──► ACK at (ACK ağda kaybolursa -> MÜKERRER İŞLENİR!)
(Çözüm: Tüketiciyi IDEMPOTENT tasarlamak!)

[ 3. EXACTLY-ONCE SEMANTICS (EOS - Tam Olarak Bir Kez) ]
İki Fazlı Onay (2PC) + Kafka Transaksiyonel Producer + Read-Committed İzolasyonu.
(En güvenli ancak en maliyetli ve karmaşık modeldir)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Birine mektup göndermenin 3 yolu: 1) At-Most-Once (Aptal Postacı): Mektubu posta kutusuna atar ve hemen arkasını dönüp gider. Mektup yağmurda çürürse çürür; asla tekrar denemez (Kayıp olabilir). 2) At-Least-Once (İnatçı Postacı): Kapıyı çalar, mektubu verir. Ancak siz 'Aldım' demeden önce gök gürültüsünden sesinizi duymazsa ertesi gün aynı mektuptan bir tane daha getirir! Mektup asla kaybolmaz ama iki kez teslim edilebilir (Mükerrerlik). 3) Exactly-Once (Noter Tasdikli Postacı): İki tarafın da imza attığı, noter kütüğüne işlenen kusursuz sistemdir; mektup ne kaybolur ne de iki kez gelir!

🇬🇧 ENGLISH (ELI5)

Three ways to deliver a package: 1) At-Most-Once: Drop the parcel on the porch and run. If a porch pirate steals it, tough luck; nobody redelivers (messages can be lost). 2) At-Least-Once: Ring the bell and hand over the parcel. If lightning strikes and the courier doesn't hear your 'Thank you' receipt, they return tomorrow with a duplicate parcel! Nothing is ever lost, but duplicates happen. 3) Exactly-Once: A bonded courier and biometric notary signing the delivery ledger synchronously: the package is delivered exactly one time with zero loss and zero duplicates!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) At-Most-Once: Tüketici mesajı okur okumaz offset'i commit eder, sonra işler. İşleme sırasında çökerse mesaj kaybolur (Sıfır veri kaybı toleransı olan sistemlerde kabul edilemez). 2) At-Least-Once: Tüketici önce mesajı işler, veritabanına kaydeder, en son offset commit eder. Commit sırasında ağ koparsa mesaj tekrar tüketilir (Mükerrerlik riski). Bu model endüstri standardıdır ve tüketici katmanında Idempotency (tekilleştirme) ile çözülür. 3) Exactly-Once Semantics (EOS): Kafka'da `enable.idempotence=true` (PID + Sequence Number) ve `TransactionalId` ile iki fazlı onay (2PC) kullanılarak sağlanır. Kafka'dan Kafka'ya (Stream-to-Stream) akışlarda kusursuzdur; ancak harici bir veritabanına yazarken harici sistemin de 2PC desteklemesi gerekir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) At-Most-Once: Consumers commit offset before processing. If processing crashes, the message is lost forever (unacceptable for financial systems). 2) At-Least-Once: Consumers process payload and persist to DB before committing offsets. If the network drops prior to offset commit, the broker redelivers on restart. This is the enterprise industry standard, pairing at-least-once delivery with idempotent downstream consumers. 3) Exactly-Once Semantics (EOS): Enforced in Kafka via idempotent producers (Producer ID + monotonic Sequence Numbers) and atomic transactions across multiple topic partitions using a Two-Phase Commit transaction coordinator. While native for Kafka-to-Kafka stream pipelines, sinking to third-party databases requires distributed transactions (XA) or idempotent UPSERTs.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Idempotent Tüketici Deseni (At-Least-Once Güvenliği)
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

public class IdempotentConsumer {
    // İşlenmiş benzersiz mesaj ID'leri tablosu (Redis veya DB Unique Constraint)
    private final Set<String> processedMessageIds = ConcurrentHashMap.newKeySet();

    public record Message(String messageId, String payload) {}

    public synchronized boolean processMessage(Message msg) {
        // Eğer bu mesaj ID'si daha önce başarıyla işlendiyse: PAS GEÇ!
        if (!processedMessageIds.add(msg.messageId())) {
            IO.println("⚠ MÜKERRER MESAJ TESPİT EDİLDİ! Pas geçiliyor: " + msg.messageId());
            return false;
        }

        // İlk kez işleniyor:
        IO.println("✔ Mesaj başarıyla işlendi: " + msg.payload());
        return true;
    }
}

void main() {
    var consumer = new IdempotentConsumer();
    var msg = new IdempotentConsumer.Message("msg_uuid_9981", "Bakiye Düş: 50 TL");

    consumer.processMessage(msg); // 1. Teslimat -> İşlenir
    consumer.processMessage(msg); // 2. Ağ Yeniden Denemesi (Retry) -> Tekilleştirilir!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bizim sistemimiz uçtan uca Exactly-Once çalışıyor' iddiasında bulunurken dikkatli olun! Dış dünyadaki bir SMS veya e-posta servisine mesaj gönderirken Exactly-Once fiziksel olarak imkansızdır (SMS atıldıktan sonra elektrik kesilirse SMS iki kez gidebilir). Mülakatçıya gerçek dünyada çözümün 'At-Least-Once + Idempotency' olduğunu söyleyin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Be wary of blindly claiming end-to-end Exactly-Once across heterogeneous systems. Sending an SMS or email cannot be rolled back via two-phase commit. Reassure the interviewer that real-world production relies on At-Least-Once paired with Idempotent consumers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- At-Least-Once teslimat, mesaj kayıplarını sıfırlamak için en güvenilir temel modeldir.
- Mükerrer teslimat riskini bertaraf etmenin anahtarı tüketiciyi Idempotent yapmaktır.
- Kafka EOS, akış işleme (Kafka Streams) içinde tam olarak bir kez işlemeyi garanti eder.

🎯 Key Takeaways for the Interview

- At-Least-Once is the pragmatic distributed foundation to guarantee zero data loss.
- Pairing At-Least-Once with idempotent consumer logic neutralizes duplicate deliveries.
- Kafka transactional APIs guarantee Exactly-Once within internal stream processing topologies.

29

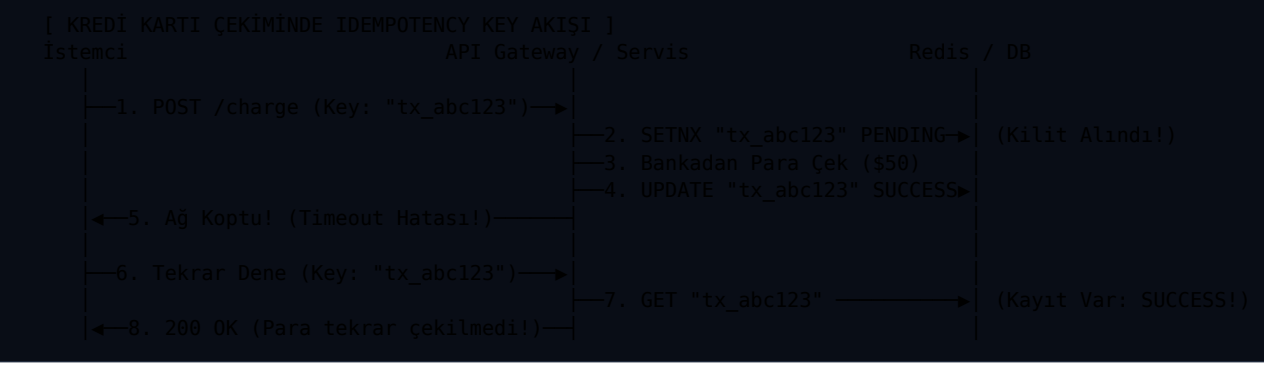
BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Idempotency (Tekrarlanabilirlik): Ağ Yeniden Denemelerinde Müker-rerliği Önleme

Idempotency Patterns: Preventing Double Charges & Duplicates

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Asansör çağrı düğmesini düşünün: Düğmeye bir kez basarsanız ışığı yanar ve asansör kata doğru hareket eder. Sabırsızlanıp düğmeye 10 kez daha basarsanız ne olur? Asansör 10 kat daha hızlı gelmez, 10 tane ayrı asansör de çağrılmaz! Düğmeye 1 kez basmakla 100 kez basmak aynı sonucu üretir. İşte matematikte ve yazılımda buna `Idempotent` (Tekrarlanabilir) denir: Kullanıcı interneti koptu diye 'Öde' butonuna 5 kez tıklasa bile kredi kartından sadece 1 kez 50 TL çekil-melidir!

🇬🇧 ENGLISH (ELI5)

Think of an elevator call button: Pressing it once illuminates the light and dispatches the cab. If you impatiently mash the button 10 more times, does it arrive 10 times faster? Does it spawn 10 separate elevators? No! Pressing it once or 100 times yields the exact same state. That is `Idempotency`. If a customer's phone switches from Wi-Fi to 5G during checkout and re-submits a payment 5 times, their credit card must be charged exactly once!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık sistemlerde ağ zaman aşımları (Timeout) nedeniyle istemciler istekleri yeniden dener (Retry). Bir POST isteği doğası gereği idempotent değildir. Çözüm: 1) İstemci Tarafında Benzersiz Anahtar (Idempotency Key): İstemci isteği göndermeden önce bir UUID üretir (`Idempotency-Key: 9b1deb4d...`) ve HTTP başlığına ekler. 2) Dağıtık Kilit & Durum Tablosu (Redis / DB): API Gateway anahtarı alır; `SET key PENDING NX EX 120` ile atomik kilit oluşturur. İşlem tamamlandığında sonuç (Response Body) bu anahtarın altına kaydedilir. 3) Mükerrer İstek Geldiğinde: Eğer anahtar `SUCCESS` durumundaysa, işlem tekrar çalıştırılmaz; önbellekteki ilk başarılı yanıt aynen geri dönülür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

In distributed microservices, network timeouts inevitably trigger client retries. While GET and PUT are inherently idempotent, HTTP POST mutations are not. Enterprise Idempotency Pattern: 1) Client Idempotency Key: The client generates a unique UUID (`Idempotency-Key: 9b1deb4d...`) passed in headers. 2) Atomic State Locking: The receiver executes an atomic lock (`SET key PENDING NX EX 60`) in Redis or an RDBMS unique table constraint. 3) Response Caching: When the transactional operation succeeds, the final response payload is persisted against the idempotency key. Subsequent retry attempts detect the existing key and return the cached receipt immediately without re-executing side effects.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Idempotency Key Motoru (Ödeme Güvenlik Kalkanı)
import java.util.concurrent.*;

public class PaymentIdempotencyEngine {
    public enum TxStatus { PENDING, COMPLETED }
    public record TxRecord(TxStatus status, String responsePayload) {}

    private final ConcurrentMap<String, TxRecord> store = new ConcurrentHashMap<>();

    public synchronized String processPayment(String idempotencyKey, double amount) {
        TxRecord existing = store.get(idempotencyKey);
        if (existing != null) {
            if (existing.status() == TxStatus.COMPLETED) {
                IO.println("⚡ Önbellekteki Makbuz Döndü: " + existing.responsePayload());
                return existing.responsePayload();
            }
            throw new IllegalStateException("İşlem şu anda başka bir thread tarafından yürütülüyor!");
        }

        // 1. Durumu PENDING yap
        store.put(idempotencyKey, new TxRecord(TxStatus.PENDING, null));

        // 2. Banka POS simülasyonu
        IO.println(">>> Bankadan $" + amount + " çekildi! (Kart İşlemi Yapıldı)");
        String receipt = "RECEIPT_TX_" + System.currentTimeMillis();

        // 3. Durumu COMPLETED yap ve sonucu sakla
        store.put(idempotencyKey, new TxRecord(TxStatus.COMPLETED, receipt));
        return receipt;
    }
}

void main() {
    var engine = new PaymentIdempotencyEngine();
    String key = "req_uuid_550e8400";

    // 1. İstek (Başarılı)
    engine.processPayment(key, 100.0);
    // 2. Ağ hatası nedeniyle aynı isteğin tekrarı (Tekrar para çekilmez!)
    engine.processPayment(key, 100.0);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Idempotency anahtarlarını veritabanında sonsuza kadar saklamayın! Bu tablolar milyonlarca gereksiz anahtarla şişer. Idempotency anahtarlarına 24 veya 48 saatlik bir TTL (Time-To-Live) belirleyin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not persist idempotency keys indefinitely. Storing transient keys permanently bloats storage. Enforce a rolling 24-to-48 hour TTL window, which easily exceeds maximum client retry horizons.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Idempotency Key, ağ hatalarında mükerrer para çekilmesini ve kayıt açılmasını engeller.

• HTTP POST istekleri istemcinin ürettiği benzersiz bir token ile idempotent kılınır.

• Stripe, PayPal ve Adyen gibi ödeme devlerinin tüm API'leri bu desenle korunur.

🎯 Key Takeaways for the Interview

• Idempotency keys eliminate double-billing and duplicate resource creation during retries.

• Pairing client-generated UUID tokens with atomic server-side state locks ensures idempotency.

• Industry payment APIs (Stripe, Adyen) mandate Idempotency-Key headers on all mutations.

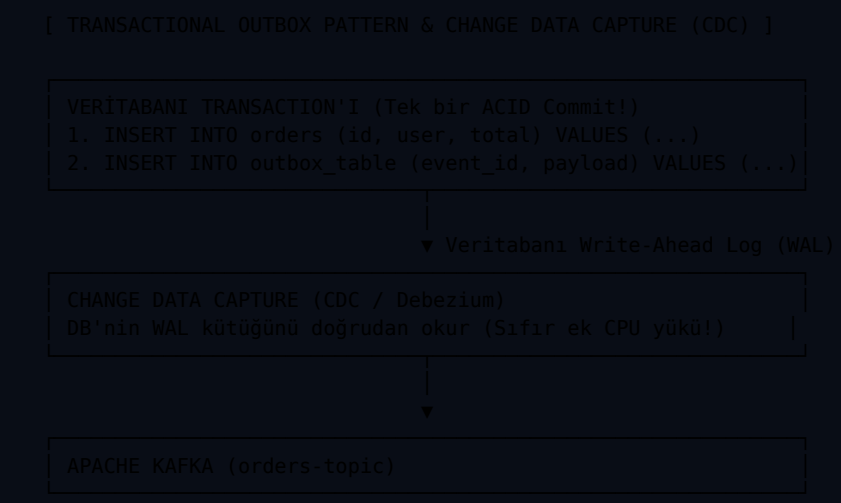
30

BÖLÜM: ASENKRON İLETIŞIM, MESAJLAŞMA VE OLAY TABANLI MIMARI | MODULE: ASYNCHRONOUS MESSAGING & EVENT-DRIVEN ARCHITECTURE

Transactional Outbox Pattern ve Change Data Capture (CDC / Debezium)

Transactional Outbox Pattern & Change Data Capture (CDC)

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette sözleşme imzaladığınızı düşünün. Normalde yapılması gereken iki iş vardır: 1) Sözleşmeyi şirket kasasına kilitlemek (DB kaydı), 2) Müşteriye kargo göndermek (Kafka olayı). Eğer sözleşmeyi kasaya koyduktan hemen sonra elektrikler kesilirse ve kargocuya haber veremezseniz sistem bozulur! Transactional Outbox deseni şudur: Kasanın içine küçük bir 'Giden Evrak Kutusu' (Outbox Tablosu) koyarsınız. Sözleşmeyi kilitlerken kargo emrini de AYNI kasanın içine tek bir anahtar çevirmeyle kilitlersiniz (ACID Transaction)! Sonra kargo çırağı (Debezium/CDC) kasadaki kutuyu okur ve kargoyu güvenle yola çıkarır!

🇬🇧 ENGLISH (ELI5)

Imagine signing a commercial contract requiring two atomic actions: 1) Lock the contract in the vault (Database write), 2) Dispatch a courier to the client (Kafka event). If lightning strikes right after locking the vault, the courier is never alerted, corrupting the system! The Transactional Outbox Pattern solves this: Place an 'Outgoing Mailbox' inside the exact same vault! In a single twist of the vault key (one atomic ACID transaction), you lock both the contract and the courier dispatch ticket together! A dedicated courier bot (Debezium / CDC) monitors the outgoing mailbox and dispatches events with zero risk of desynchronization!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

İki Fazlı İki Ayrı Sistem Problemi (Dual-Write Problem): Uygulama önce veritabanına `INSERT` yapıp ardından Kafka'ya `send()` yaparsa; DB başarılı olup Kafka çağrısı çökerse sistem tutarsızlaşır. Transactional Outbox Deseni bu ikili yazma açığını çözer: 1) İş mantığı verisi (`orders`) ve dışarıya fırlatılacak olay (`outbox`) AYNI veritabanında TEK bir yerel ACID transaction içinde commit edilir. 2) Change Data Capture (CDC / Debezium): Veritabanının işlem günlüğünü (PostgreSQL WAL veya MySQL Binlog) doğrudan dinler. Veritabanına hiçbir sorgu atmadan, disk seviyesindeki binary log değişikliklerini yakalayarak Kafka'ya sıfır veri kaybıyla yazar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Dual-Write Anti-Pattern: A service writes to a relational database and then publishes to Kafka. If the database commits but the network blip crashes the Kafka producer, data desynchronizes permanently. The Transactional Outbox Pattern cures this: 1) Business entities and an `outbox` event table commit together in a single local ACID database transaction. 2) Change Data Capture (CDC / Debezium): An external tailer intercepts the database Write-Ahead Log (WAL / Binlog) directly at the storage engine level with zero query polling overhead, streaming mutations into Kafka with absolute At-Least-Once delivery guarantees.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Transactional Outbox Veri Yapısı ve Atomic Commit Temsili
public class TransactionalOutboxPattern {
    public record OrderRecord(long orderId, String customer, double amount) {}
    public record OutboxMessage(String eventId, String aggregateType, String payload) {}

    // Tek bir yerel ACID veritabanı transaction'ında çalışan metod:
    public void createOrderWithOutbox(OrderRecord order) {
        // BEGIN TRANSACTION;

        // 1. İş tablosuna yaz:
        IO.println("1. DB INSERT INTO orders: ID=" + order.orderId());

        // 2. Outbox tablosuna yaz (Aynı DB Transaction'ı!):
        var outboxEvent = new OutboxMessage(
            "evt_" + System.currentTimeMillis(),
            "ORDER",
            "{"orderId": " + order.orderId() + ", "amount": " + order.amount() + "}"
        );
        IO.println("2. DB INSERT INTO outbox: EventID=" + outboxEvent.eventId());

        // COMMIT TRANSACTION;
        // CDC (Debezium) arka planda WAL'ı okuyup Kafka'ya iletir!
    }
}

void main() {
    var outbox = new TransactionalOutboxPattern();
    outbox.createOrderWithOutbox(new TransactionalOutboxPattern.OrderRecord(9021L, "Ali Bozkurt",
250.0));
    IO.println("✔ Atomik Outbox Kaydı Tamamlandı (Dual-Write Engellendi)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Outbox tablosunu periyodik olarak `SELECT * FROM outbox WHERE sent = false` şeklinde polling yaparak okumak mülakatta Junior cevabıdır! Tablo büyüdükçe veritabanını kilitler. Senior/Staff seviye cevap: Debezium gibi bir CDC aracının veritabanının binary Write-Ahead Log'unu (WAL) doğrudan dinlemesidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Proposing a background cron poller (`SELECT * FROM outbox WHERE processed = false`) is an interview red flag that degrades database CPU. The Staff-level answer is transaction log tailing via Change Data Capture (CDC with Debezium reading PostgreSQL WAL).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Dual-Write problemi, DB ve Kafka'ya aynı anda ayrı ayrı yazmaya çalışmanın felaketidir.
- Outbox tablosu, olayları iş verisiyle aynı ACID transaction'ında güvenceye alır.
- Change Data Capture (CDC / Debezium) veritabanı WAL kütüğünü dinleyerek sıfır yükle çalışır.

🎯 Key Takeaways for the Interview

- The Dual-Write anti-pattern risks fatal desynchronization between DB state and event topics.
- The Outbox table commits domain entities and event envelopes within a single ACID transaction.
- CDC engines (Debezium) tail database WAL logs with zero query polling penalty.

BÖLÜM 6 / MODULE 6

Dağıtık Sistem Temelleri ve Eşgüdüm

Distributed Systems Fundamentals, Consensus & Coordination

Saat kaymaları ve Vektör Saatleri, Raft fikir birliği algoritması, etcd/ZooKeeper kilitleri, Twitter Snowflake ID üreteci, Saga deseni ve Circuit Breaker.

Clock drift & Vector Clocks, Raft consensus mechanics, etcd/ZooKeeper coordination, Twitter Snowflake ID generators, Saga vs 2PC, and Circuit Breaker resilience.

31

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Saatler ve Zaman Sorunları: Saat Kayması (Clock Drift), NTP ve Vektör Saatleri

Distributed Clocks & Time: Physical Drift, NTP & Vector Clocks

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ FİZİKSEL SAAT KAYMASI (CLOCK DRIFT & NTP SKEW) ]
Sunucu A Saati: [ 12:00:00.100 ] ──► Olay X (Gerçekte önce oldu!)
Sunucu B Saati: [ 12:00:00.050 ] ──► Olay Y (Saat geri kaldığı için önce gibi görünüyor!)
(Last-Write-Wins uygularsanız Olay X haksız yere ezilir ve silinir!)

[ MANTIKSAL VEKTÖR SAATI (VECTOR CLOCK) ]
Her düğüm kendi sayacını tutar: V = [NodeA: 1, NodeB: 0, NodeC: 2]
Eğer V1'deki tüm elemanlar <= V2 ise: V1 kesinlikle V2'den ÖNCE olmuştur (Happens-Before)!
Eğer çelişki varsa (V1 > V2 ve V2 > V1): Eşzamanlı çakışma (Concurrent Conflict) tespit edilir!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kolunuzdaki mekanik saatin her gün 2 saniye ileri gittiğini, arkadaşınızın saatinin ise 3 saniye geri kaldığını düşünün. İkiniz de 'Ben mesajı önce attım' diye kavga ederseniz kol saatlerinize bakarak kimin haklı olduğunu ASLA çözemezsiniz! Bilgisayarlardaki kuvars saatler de sıcaklıktan ve elektrikten dolayı kayar (Clock Drift). Dağıtık sistemlerde 'fiziksel duvardaki saate' güvenemezsiniz! Bunun yerine basketbol maçındaki paslaşma sırası gibi çalışan 'Mantıksal Saatler' (Lamport / Vector Clocks) kullanılır: 'Top önce bendeydi, ben sana attım, sonra sen kaleye attın; saatin kaç olduğu kimin umurunda!'

🇬🇧 ENGLISH (ELI5)

Imagine your wristwatch gains 2 seconds every day, while your friend's wristwatch loses 3 seconds. If you both send letters claiming 'I mailed mine first', comparing your wristwatches is completely useless! Computer quartz crystals drift due to thermal fluctuations (Clock Drift). In distributed systems, relying on physical wall-clock time (`System.currentTimeMillis()`) is dangerous! Instead, systems use Logical Clocks (Lamport & Vector Clocks): 'I held the ball first, passed it to you, and then you scored; physical clock ticks are irrelevant, causal sequence is everything!'

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık sistemlerin en büyük yanılgısı: 'Fiziksel saatler eşzamanlıdır'. NTP (Network Time Protocol) saatleri senkronize etmeye çalışır ancak ağ dalgalanmaları nedeniyle sunucular arasında 10ms - 100ms arasında saat kayması (Clock Skew) kaçınılmazdır. 'Last-Write-Wins' (LWW) fiziksel saate dayanırsa veri kaybına yol açar (Google Spanner bunu çözmek için atomik saatler ve GPS alıcıları kullanan TrueTime API ile belirsizlik aralığını [ε ≈ 7ms] sınırlandırmıştır). Leslie Lamport'un Mantıksal Saatleri ve Vektör Saatleri (Vector Clocks), fiziksel saate ihtiyaç duymadan 'Happens-Before' nedensellik ilişkisini (A → B) matematiksel olarak modeller ve eşzamanlı çakışmaları (Concurrent Updates) tespit eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The fallacy of physical time in distributed nodes: NTP continuously steers hardware crystals, but asymmetric network jitter introduces 10ms-100ms clock skew. Naive Last-Write-Wins (LWW) relying on wall-clock time causes silent data loss. Google Spanner tackled this by placing GPS receivers and atomic clocks in every datacenter, bounding physical drift uncertainty to a narrow window (ε ≈ 7ms via TrueTime). Leslie Lamport proved that causality (A → B) can be modeled logically without physical clocks. Vector Clocks track an array of monotonic counters across nodes: if neither clock dominates the other, a concurrent conflict is flagged and handed to application logic for resolution.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Vektör Saati (Vector Clock) ile Nedensellik ve Çakışma Tespiti
import java.util.*;

public class VectorClock {
    private final Map<String, Integer> clock = new HashMap<>();

    public VectorClock() {}
    public VectorClock(Map<String, Integer> copy) { clock.putAll(copy); }

    public synchronized void tick(String nodeId) {
        clock.put(nodeId, clock.getOrDefault(nodeId, 0) + 1);
    }

    // İki vektör saatinin nedensellik durumunu karşılaştır
    public enum Relation { BEFORE, AFTER, CONCURRENT }

    public static Relation compare(VectorClock v1, VectorClock v2) {
        boolean v1Greater = false;
        boolean v2Greater = false;

        Set<String> allNodes = new HashSet<>(v1.clock.keySet());
        allNodes.addAll(v2.clock.keySet());

        for (String node : allNodes) {
            int c1 = v1.clock.getOrDefault(node, 0);
            int c2 = v2.clock.getOrDefault(node, 0);
            if (c1 > c2) v1Greater = true;
            if (c2 > c1) v2Greater = true;
        }

        if (v1Greater && !v2Greater) return Relation.AFTER;
        if (!v1Greater && v2Greater) return Relation.BEFORE;
        return Relation.CONCURRENT; // Çelişki / Eşzamanlı Çakışma!
    }
}

void main() {
    var v1 = new VectorClock();
    v1.tick("NodeA"); // [A:1]

    var v2 = new VectorClock(v1.clock);
    v2.tick("NodeB"); // [A:1, B:1] -> v2, v1'den sonra olmuştur!

    IO.println("V1 ve V2 İlişkisi: " + VectorClock.compare(v1, v2)); // BEFORE
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta dağıtık bir sistemde iki olayın hangisinin önce gerçekleştiğini belirlemek için `System.currentTimeMillis()` kullanacağınızı ASLA söylemeyin! NTP saati geriye atlayabilir (Leap Second veya NTP sync). Her zaman Mantıksal Saat veya Monotonic Sequence Number önerin.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose sorting distributed transactions across multiple servers using `System.currentTimeMillis()`. NTP adjustments can cause system clocks to tick backwards! Always prescribe Logical Clocks, Lamport timestamps, or monotonic sequence counters.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Fiziksel saatler donanım kayması ve NTP gecikmesi nedeniyle güvenilmezdir.

• Last-Write-Wins (LWW) saati geri kalan sunucudaki güncel verilerin silinmesine yol açabilir.

• Vektör saatleri (Vector Clocks) fiziksel zamana ihtiyaç duymadan çakışmaları tespit eder.

🎯 Key Takeaways for the Interview

• Physical clocks drift constantly due to hardware temperature and NTP transit jitter.

• Last-Write-Wins risks silently dropping valid writes if clock skew is present.

• Vector Clocks accurately trace causal order and isolate concurrent update conflicts.

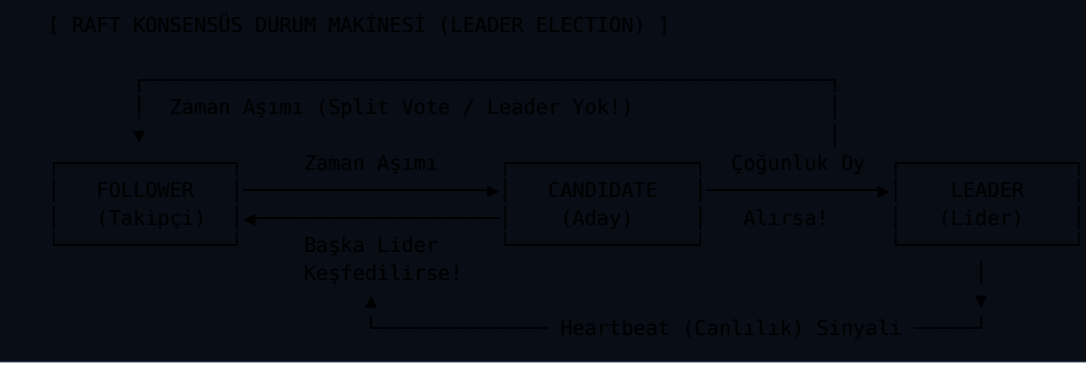
32

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Fikir Birliği (Consensus): Raft Algoritması (Lider Seçimi ve Replikasyon)

Distributed Consensus: The Raft Algorithm (Leader Election & Logs)

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

5 kişilik bir arkadaş grubunun tatile nereye gideceğine karar vermesini düşünün. Herkes aynı anda konuşursa kaos çıkar. Raft algoritması şudur: 1) İçlerinden biri 'Ben grup lideri olmak istiyorum, bana oy verin' der (Candidate). 2) Eğer arkadaşlarının çoğunluğu (en az 3 kişi) 'Tamam, sen lidersin' derse o kişi Lider olur. 3) Artık tüm kararları lider alır, gruba söyler ve çoğunluk onaylayınca deftere yazar. Lider aniden bayılırsa (Crash), arkadaşlarından biri 'Liderden 150 milisaniyedir ses çıkmıyor' diyerek yeni bir seçim başlatır!

🇬🇧 ENGLISH (ELI5)

Imagine 5 friends voting on a vacation spot. If everyone screams conflicting orders, total chaos erupts. Raft brings democratic order: 1) A member proposes: 'I volunteer as Leader, vote for me!' (Candidate). 2) If a majority quorum (at least 3 out of 5) grants their vote, they become the undisputed Leader. 3) The Leader proposes decisions, replicates the entry to nodes, and commits once a majority confirms. If the Leader faints (crashes), an election timer expires and nodes elect a fresh leader within milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Raft (Ongaro & Ousterhout, Stanford 2014), anlaşılması imkansız derecede karmaşık olan Paxos algoritmasının yerine geliştirilmiş, anlaşılabilir bir konsensüs algoritmasıdır (etcd, Kubernetes, Kafka KRaft, TiDB tarafından kullanılır). 3 temel alt problemden oluşur: 1) Lider Seçimi (Leader Election): Liderden belirli bir süre (Heartbeat timeout: 150-300ms) kalp atışı alamayan Follower, rastgele bir seçim zaman aşımıyla (Randomized Election Timeout) Candidate olur ve `RequestVote` fırlatır. Çoğunluk oyu alan ($N/2 + 1$) lider seçilir. 2) Kütük Replikasyonu (Log Replication): Lider istemciden komutu alır, kendi kütüğüne yazar, follower'lara `AppendEntries` gönderir. Düğümlerin çoğunluğu diske yazdığında işlem 'Committed' kabul edilir. 3) Güvenlik (Safety): Yeni seçilen lider, kütüğü en güncel olan düğüm olmak zorundadır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Raft decomposes consensus into three decomposed state machines: 1) Leader Election: Followers transition to Candidates when heartbeat deadlines (150-300ms randomized timer) expire without leader contact. A candidate requests votes; securing a strict majority quorum ($(N/2) + 1$) crowns the Leader. 2) Log Replication: The Leader appends entries locally, broadcasting `AppendEntries` RPCs. When an entry is replicated across a majority, the Leader commits it and applies to the state machine. 3) Election Safety: A follower will deny vote if the candidate's log is less up-to-date than its own, guaranteeing committed entries are never overwritten or lost. Powers etcd, Kubernetes control planes, and Kafka KRaft.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Raft Rastgele Seçim Zaman Aşımı (Randomized Election Timeout)
import java.util.concurrent.*;

public class RaftElectionTimer {
    public enum Role { FOLLOWER, CANDIDATE, LEADER }

    private Role currentRole = Role.FOLLOWER;
    private long currentTerm = 0;

    public synchronized void onHeartbeatTimeout() {
        if (currentRole != Role.LEADER) {
            currentRole = Role.CANDIDATE;
            currentTerm++;
            int randomTimeout = ThreadLocalRandom.current().nextInt(150, 300);
            IO.println("Term #" + currentTerm + ": Lider düştü! Aday olundu (Timeout: " +
randomTimeout + "ms)");
        }
    }

    public synchronized void becomeLeader() {
        currentRole = Role.LEADER;
        IO.println("🏆 DÜĞÜM LİDER SEÇİLDİ! Term #" + currentTerm);
    }
}

void main() {
    var node = new RaftElectionTimer();
    node.onHeartbeatTimeout();
    node.becomeLeader();
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Raft kümesindeki düğüm sayısı daima TEK SAYI (Odd Number: 3, 5 veya 7) olmalıdır! 4 düğümlü bir kümede 2'ye 2 bölünme (Split Vote) yaşanır ve çoğunluk ($4/2 + 1 = 3$) sağlanamayabilir. 3 düğüm 1 hatayı, 5 düğüm 2 hatayı tolere eder.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Always size Raft clusters with an ODD number of nodes (3, 5, or 7). An even count (4 nodes) offers zero additional fault tolerance over 3 nodes (both tolerate only 1 failure) while elevating split-vote deadlocks.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Raft, dağıtık sistemlerde lider seçimi ve veri replikasyonunu güvenceye alır.
- Rastgele seçim zaman aşımları (Randomized Timeouts) oyların bölünmesini (Split Vote) önler.
- Kubernetes (etcd) ve Kafka (KRaft) fikir birliği omurgasını Raft üzerine kurmuştur.

🎯 Key Takeaways for the Interview

- Raft guarantees leader election and deterministic log replication across distributed nodes.
- Randomized election timeouts disperse candidacy announcements, avoiding voting ties.
- Kubernetes (via etcd) and Kafka KRaft rely directly on Raft for operational consensus.

33

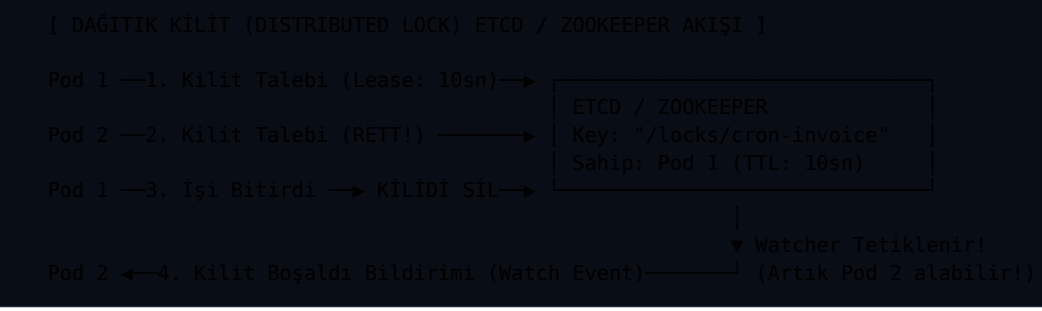
BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Eşgüdüm Servisi: Apache ZooKeeper ve etcd (Dağıtık Kilit & Lider)

Distributed Coordination: ZooKeeper & etcd (Distributed Locks & Leader)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

10 odalı bir pansiyonda tek bir ortak banyo olduğunu düşünün. Banyonun kapısına 'Dolu / Boş' yazan bir kilit asarsınız (Dağıtık Kilit - Distributed Lock). İçeri giren kişi kapıyı kilitler; dışarıdakiler kapının açılmasını bekler (Watcher). Eğer içerideki kişi banyoda bayılırsa kapı sonsuza kadar kilitli kalmasın diye kilit 10 dakika sonra otomatik açılır (Lease / TTL). İşte `etcd` ve `ZooKeeper`, binlerce sunucunun aynı anda tek bir kaynağa hücum edip birbirini ezmesini engelleyen bu güvenilir kilit servisidir!

🇬🇧 ENGLISH (ELI5)

Picture a boarding house with 10 rooms and a single shared bathroom. On the door hangs a physical latch (Distributed Lock). When someone enters, they slide the latch; others queue outside watching the door (Watcher). To prevent the door locking forever if someone falls asleep inside, the latch springs open automatically after 10 minutes (Lease / TTL). `etcd` and `ZooKeeper` serve as that centralized, bulletproof lock coordinator for thousands of server pods!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

etcd ve Apache ZooKeeper; hiyerarşik anahtar-değer ağacı (znodes), güçlü tutarlılık (Raft / Zab), geçici düğümler (Ephemeral nodes / Leases) ve olay dinleyicileri (Watchers) sunan dağıtık eşgüdüm servisleridir. Kullanım Alanları: 1) Dağıtık Kilit (Distributed Lock): Tek bir cron görevinin sadece tek bir podda çalışmasını sağlamak. 2) Lider Seçimi (Leader Election): Kümedeki podlardan biri geçici bir düğüm (`/leader`) oluşturur; çökerse Lease düşer ve Watcher diğer podları uyandırarak yeni lideri seçtirir. 3) Konfigürasyon Yönetimi: Sistem konfigürasyonunu merkezi olarak tutar; konfigürasyon değiştiğinde tüm mikroservisler anında Watcher ile güncellenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

etcd and ZooKeeper provide strongly consistent, linearizable key-value hierarchies backed by Raft/ZAB consensus. Core primitives: 1) Distributed Locking: A worker acquires a key bound to a time-limited Lease (heartbeat keep-alive). If the worker crashes, the lease expires and purges the key automatically, avoiding deadlocks. 2) Leader Election: Nodes race to claim an ephemeral key (`/leader/worker-A`). The winner leads; competitors register Watchers on the key path to react instantly to failures. 3) Dynamic Configuration: Centralizes runtime flags; changes trigger push notifications across thousands of microservice Watcher streams within milliseconds.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Dağıtık Kilit (Distributed Lock) Mantığı Simülasyonu
import java.util.concurrent.*;

public class DistributedLockSim {
    private String lockOwner = null;
    private long leaseExpiration = 0;

    public synchronized boolean acquireLock(String workerId, long leaseDurationMillis) {
        long now = System.currentTimeMillis();
        // Kilit boş mu veya süresi dolmuş mu?
        if (lockOwner == null || now > leaseExpiration) {
            lockOwner = workerId;
            leaseExpiration = now + leaseDurationMillis;
            IO.println("🔒 Kilit Alındı: " + workerId + " (Süre: " + leaseDurationMillis + "ms)");
            return true;
        }
        return false; // Kilit meşgul!
    }

    public synchronized void releaseLock(String workerId) {
        if (workerId.equals(lockOwner)) {
            lockOwner = null;
            IO.println("🔓 Kilit Bırakıldı: " + workerId);
        }
    }
}

void main() {
    var locker = new DistributedLockSim();
    boolean pod1 = locker.acquireLock("Pod-1", 5000);
    boolean pod2 = locker.acquireLock("Pod-2", 5000); // Başarısız olmalı!

    IO.println("Pod-1 Başarılı mı? " + pod1);
    IO.println("Pod-2 Başarılı mı? " + pod2);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

etcd veya ZooKeeper'ı devasa veriler (büyük JSON'lar, resimler) saklamak için KULLANMAYIN! Bu sistemlerin bellek sınırı ve mimarisi yalnızca küçük metadata ve kilitler (birkaç kilobayt) içindir; büyük veriler Raft konsensüs kütüğünü kilitler.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never store bulky payloads inside etcd or ZooKeeper. They are strictly designed for micro-metadata and consensus coordination (under 1MB per value). Storing heavy state degrades Raft commit replication across nodes.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- etcd ve ZooKeeper, dağıtık sistemlerde kilit ve lider seçiminin güvenilir adresidir.
- Lease (Kira / TTL) mekanizması, düğüm çöktüğünde kilitlerin sonsuza kadar kilitli kalmasını (Deadlock) önler.
- Watcher API, veritabanını sürekli sorgulamak (polling) yerine olay tabanlı anlık bildirim sağlar.

🎯 Key Takeaways for the Interview

- etcd and ZooKeeper serve as the authoritative standard for distributed locks and elections.
- Time-bound leases prevent distributed deadlocks by releasing locks upon node failure.
- Watcher streams replace expensive database polling with event-driven notifications.

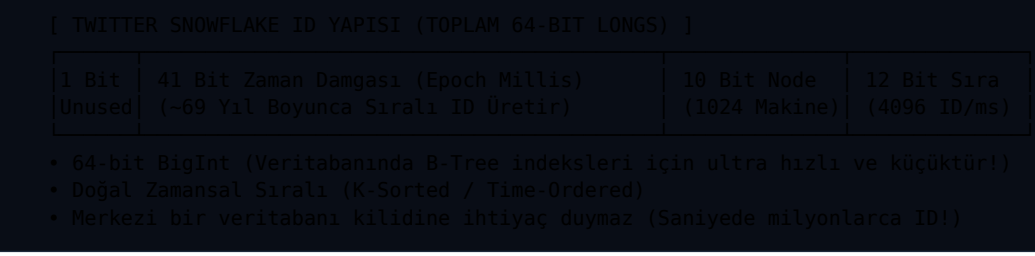
34

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık Benzersiz ID Üretimi: Twitter Snowflake, UUIDv7 ve Ticket Server

Distributed Unique ID Generation: Twitter Snowflake & UUIDv7

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir doğum hastanesini düşünün: Her doğan bebeğe benzersiz bir kimlik numarası verilmelidir. Eskiden tek bir memur sırayla 1, 2, 3… diye numara verirdi (Auto-Increment). Ama dünyada aynı anda 100 hastanede bebek doğuyorsa hepsi Ankara'daki o tek memura telefon açamaz (Darboğaz ve çöküş!). Twitter Snowflake ise her hastaneye özel bir formül verir: 'Önce şu anki saati yaz, yanına kendi hastane kodunu ekle, yanına da o saniyedeki bebek sırasını ekle!' Böylece hiçbir hastane birbirine telefon etmeden saniyede milyonlarca benzersiz ve kronolojik olarak sıralı kimlik numarası üretir!

🇬🇧 ENGLISH (ELI5)

Imagine assigning national birth certificate numbers. A naive approach is a single bureaucrat sitting in the capital stamping 1, 2, 3… sequentially (Auto-Increment ID). But if 1,000 maternity hospitals across the nation deliver babies simultaneously, that single bureaucrat causes nationwide traffic jams! Twitter Snowflake equips every hospital with a decentralized formula: 'Stamp the current millisecond timestamp, append your hospital ID, append a millisecond sequence counter!' No hospital coordinates with another, yet every generated ID is universally unique and strictly chronological!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık mimarilerde ID gereksinimleri: Benzersizlik, zamana göre sıralanabilirlik (K-Sortable / B-Tree indeks dostu), 64-bit tamsayı (RAM ve disk tasarrufu). Yöntemler: 1) UUIDv4: 128-bit rastgeledir; zamana göre sıralı değildir, B-Tree indekslerinde sayfa parçalanması (Page Fragmentation) yaratır. 2) UUIDv7: 128-bit'in ilk 48 bitini zaman damgası yaparak sıralı hale getirmiştir. 3) Twitter Snowflake: 64-bit'lik endüstri standardıdır: 1 bit işaret (0), 41 bit zaman damgası (69 yıllık ömür), 10 bit makine/veri merkezi ID'si (1024 bağımsız düğüm), 12 bit sıra numarası (Aynı milisaniyede düğüm başına 4096 ID). Tek bir düğüm saniyede 4 milyon üzerinde sıralı ID üretebilir; sıfır ağ çağrısı, sıfır kilit!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed ID generation requirements: 64-bit integer density, time-ordering (K-sortable to preserve B-Tree append locality), and zero network coordination. 1) UUIDv4: 128-bit completely random; destroys B-Tree index cache efficiency via catastrophic page splits. 2) UUIDv7: Modern 128-bit standard embedding a 48-bit millisecond timestamp prefix for sequential B-Tree friendliness. 3) Twitter Snowflake: The 64-bit gold standard: 1 unused sign bit, 41 bits millisecond epoch timestamp (69 years span), 10 bits machine/datacenter identifier (1024 unique generator nodes), 12 bits sequence counter (4096 IDs per ms per node). A single node generates 4.096 million unique, sortable, collision-free IDs per second in pure local CPU memory without network RPCs.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Gerçek Twitter Snowflake 64-Bit ID Üreteci Implementasyonu
public class SnowflakeIdGenerator {
    private static final long EPOCH = 1704067200000L; // Özel Başlangıç (1 Ocak 2024)
    private static final long WORKER_ID_BITS = 10L;
    private static final long SEQUENCE_BITS = 12L;

    private static final long MAX_WORKER_ID = -1L ^ (-1L << WORKER_ID_BITS); // 1023
    private static final long MAX_SEQUENCE = -1L ^ (-1L << SEQUENCE_BITS);   // 4095

    private final long workerId;
    private long sequence = 0L;
    private long lastTimestamp = -1L;

    public SnowflakeIdGenerator(long workerId) {
        if (workerId > MAX_WORKER_ID || workerId < 0) {
            throw new IllegalArgumentException("Worker ID 0 ile " + MAX_WORKER_ID + " arasında
olmalıdır");
        }
        this.workerId = workerId;
    }

    public synchronized long nextId() {
        long currentTimestamp = System.currentTimeMillis();

        if (currentTimestamp == lastTimestamp) {
            sequence = (sequence + 1) & MAX_SEQUENCE;
            if (sequence == 0) {
                // Aynı milisaniyede 4096 sınırına ulaşıldı -> Sonraki milisaniyeyi bekle!
                while (currentTimestamp <= lastTimestamp) {
                    currentTimestamp = System.currentTimeMillis();
                }
            }
        } else {
            sequence = 0L;
        }

        lastTimestamp = currentTimestamp;

        return ((currentTimestamp - EPOCH) << (WORKER_ID_BITS + SEQUENCE_BITS))
                | (workerId << SEQUENCE_BITS)
                | sequence;
    }
}

void main() {
    var generator = new SnowflakeIdGenerator(7); // Düğüm ID: 7
    for (int i = 0; i < 3; i++) {
        IO.println("Snowflake 64-bit ID: " + generator.nextId());
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Snowflake algoritmasının tek zayıf noktası 'Saat Geriye Kayması' (Clock Moving Backwards) durumudur! Eğer NTP sunucu saatini 5 milisaniye geriye çekerse, daha önce üretilmiş bir ID tekrar üretilebilir (ID Çarpışması!). Gerçek implementasyonlarda saat geri giderse hata fırlatılmalı veya saat yakalanana kadar beklenmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

The critical Achilles' heel of Snowflake is NTP clock skew moving backwards. If system time rolls backward by 5ms, duplicate IDs can collide! Production implementations must throw exceptions or sleep until current time catches up with `lastTimestamp`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Snowflake ID'leri 64-bit boyutuyla veritabanı indekslerinde minimum yer kaplar.
- Doğal zaman sıralaması (K-Sorted) sayesinde en son kayıtlar daima tablonun sonuna eklenir.
- Merkezi bir veritabanı kilit servisine ihtiyaç duymadan yerel olarak üretilir.

🎯 Key Takeaways for the Interview

- 64-bit Snowflake integers maximize B-Tree clustering index density and speed.
- Time-ordered IDs prevent random B-Tree page splits and fragmentation.
- Snowflake nodes operate autonomously without distributed locks or network latency.

35

BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Dağıtık İşlemler: 2-Phase Commit (2PC) vs Saga Pattern (Orchestration vs Choreography)

Distributed Transactions: 2-Phase Commit (2PC) vs Saga Pattern

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. TWO-PHASE COMMIT (2PC) - KATI KİLİT, YAVAŞ, BLOKLAYAN ]
Koordinatör ──1. Hazır mısın? (Prepare)──► [DB 1] (Kilitler ve bekler!)
Koordinatör ──1. Hazır mısın? (Prepare)──► [DB 2] (Kilitler ve bekler!)
Koordinatör ──2. COMMIT / ROLLBACK ──────► İkisi de aynı anda onaylar!

[ 2. SAGA PATTERN (TELAFİ EDİCİ İŞLEMLER / COMPENSATING TRANSACTIONS) ]
Adım 1: Para Çekildi ($100) ──► Başarılı
Adım 2: Stok Rezerve Edildi ──► Başarılı
Adım 3: Kargo Oluşturuldu   ──► HATA ÇIKTI! (Kargo Şirketi Çöktü)
Geri Alma (Compensating):
Adım 2-Ters: Stok Rezervasyonunu İPTAL ET
Adım 1-Ters: Parayı Kullanıcıya İADE ET ($100)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

3 arkadaşın birlikte tatile gitmesini düşünün: 1) İki Fazlı Onay (2PC): 'Üçümüz de aynı anda uçak biletini alıyoruz, 3'e kadar sayacağım: 1-2-3 al!' dersiniz. Biri son anda 'İnternetim koptu' derse diğer ikisi de bileti alamaz, herkes donar kalır! 2) Saga Deseni: Herkes kendi işini sırayla yapar: Ali bileti alır, Veli oteli tutar, Can arabayı kiralar. Can 'Araba kalmamış' derse ne olur? Veli hemen oteli ücretsiz iptal eder, Ali de uçak biletini iade eder (Telafi İşlemi - Compensating Transaction)! Herkes kendi telafi planını çalıştırarak sistemi eski haline döndürür!

🇬🇧 ENGLISH (ELI5)

Booking a vacation across 3 independent systems: 1) Two-Phase Commit (2PC): The coordinator freezes all databases: 'I am counting to 3; everyone lock resources and prepare to commit simultaneously!' If airline booking stalls, hotel and car rental databases sit locked hostage! 2) Saga Pattern: Decoupled local transactions with compensatory rollbacks: Step 1 charges the credit card. Step 2 reserves the flight. Step 3 reserves the hotel. If hotel reservation fails, compensate backwards: Step 2-Compensate refunds the flight, Step 1-Compensate refunds the card! No long-lived database locks!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Two-Phase Commit (2PC / XA): Güçlü ACID garantisi verir. Koordinatör önce 'Prepare' aşamasında tüm düğümlerden 'Hazırım' sözü alır, ardından 'Commit' gönderir. Büyük kusurları: Koordinatör çökerse kilitler sonsuza kadar asılı kalır (Blocking); ağ gecikmeleri nedeniyle throughput berbattır; mikroservislerde ASLA ölçeklenmez. 2) Saga Pattern: Uzun süren dağıtık işlemleri yerel ACID transaction'lar zincirine böler. Bir adım başarısız olursa, önceki başarılı adımlar 'Compensating Transactions' (Telafi Edici İşlemler) ile geri alınır. İki uygulama biçimi: A) Choreography: Olay tabanlıdır (Event-Driven); servisler Kafka olaylarını dinleyip birbirini tetikler. B) Orchestration: Merkezi bir 'Saga Orchestrator' (Devlet Makinesi - State Machine) tüm adımları yönetir ve hata anında telafi adımlarını sırayla tetikler (Büyük sistemlerde takibi kolay olduğu için önerilir).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Two-Phase Commit (2PC): Provides strict linearizable ACID across databases. Coordinator executes Phase 1 (Prepare) where participants acquire locks, then Phase 2 (Commit). Flaws: Synchronously blocks threads and database locks; if the coordinator crashes mid-flight, participants deadlock indefinitely. Strictly unviable for high-scale microservices. 2) Saga Pattern: Decomposes distributed transactions into sequences of isolated local transactions. If a local transaction fails, compensating transactions execute in reverse order to undo side-effects. Execution models: A) Choreography: Decentralized event pub/sub across Kafka topics; difficult to trace at scale. B) Orchestration: A centralized state machine orchestrator coordinates step execution and rollback compensation trees; universally preferred in enterprise architectures for auditability.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Saga Orchestrator (Telafi Edici İşlem Motoru)
import java.util.*;

public class SagaOrchestrator {
    public interface SagaStep {
        boolean execute();
        void compensate(); // Telafi edici geri alma
    }

    public static boolean executeSaga(List<SagaStep> steps) {
        Deque<SagaStep> executedSteps = new ArrayDeque<>();
        for (SagaStep step : steps) {
            if (step.execute()) {
                executedSteps.push(step);
            } else {
                IO.println("⚠ HATA OLUŞTU! Geri sarma (Compensating) başlatılıyor...");
                while (!executedSteps.isEmpty()) {
                    executedSteps.pop().compensate();
                }
                return false;
            }
        }
        return true;
    }
}

void main() {
    var paymentStep = new SagaOrchestrator.SagaStep() {
        public boolean execute() { IO.println("1. Ödeme Alındı: $100"); return true; }
        public void compensate() { IO.println("1-TELAFİ: Kullanıcıya $100 İade Edildi!"); }
    };

    var inventoryStep = new SagaOrchestrator.SagaStep() {
        public boolean execute() { IO.println("2. Stok Rezerve Edilemedi! (HATA)"); return false; }
        public void compensate() { IO.println("2-TELAFİ: Stok Serbest Bırakıldı."); }
    };

    boolean success = SagaOrchestrator.executeSaga(List.of(paymentStep, inventoryStep));
    IO.println("Saga Başarılı mı? " + success);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta mikroservisler arasında veri tutarlılığı sorulduğunda 'Two-Phase Commit (2PC) kurarım' demeyin! 2PC dağıtık sistemlerde performans katili olarak bilinir. Mülakatçıya doğrudan Saga Deseni (Orchestration tabanlı) ve telafi edici işlemleri anlatın.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch 2PC for modern distributed cloud microservices. 2PC causes crippling latency and lock contention. Instantly propose the Saga Pattern with explicit Compensating Transactions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- 2PC kilitleri asılı tuttuğu için mikroservis mimarilerinde ölçeklenemez.
- Saga Deseni işlemi yerel parçalara böler ve hata anında ters telafi adımlarını çalıştırır.
- Orchestrator tabanlı Saga mimarisi, karmaşık akışlarda gözlemlenebilirlik ve yönetim kolaylığı sağlar.

🎯 Key Takeaways for the Interview

- 2PC introduces synchronous lock blocking that cripples microservice throughput.
- The Saga Pattern executes local transactions with reverse compensating rollback hooks.
- Orchestrator-based Sagas provide superior visibility, state tracking, and auditing.

36

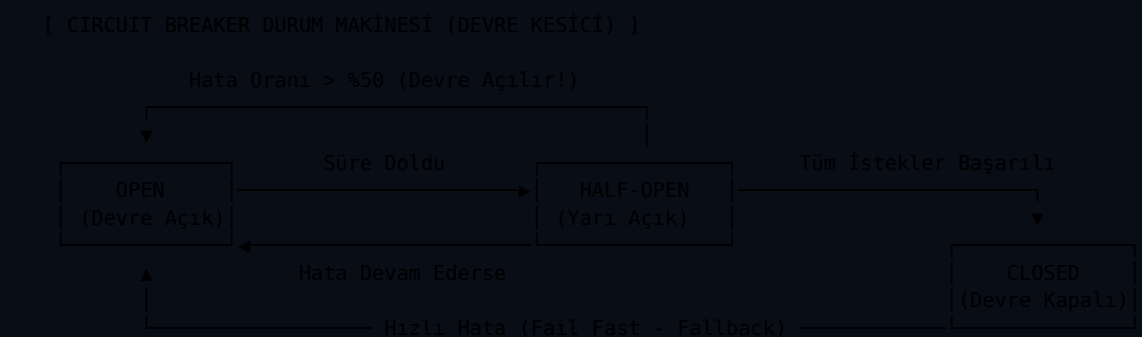
BÖLÜM: DAĞITIK SISTEM TEMELLERI VE EŞGÜDÜM | MODULE: DISTRIBUTED SYSTEMS FUNDAMENTALS, CONSENSUS & COORDINATION

Hata İzolasyonu ve Dayanıklılık: Circuit Breaker, Bulkhead ve Jitter'lı Yeniden Deneme

Fault Tolerance & Resiliency: Circuit Breaker, Bulkhead & Jittered Retry

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

1) Circuit Breaker (Evdeki Sigorta): Evde prizden kıvılcım çıkınca tüm binanın yanmasını beklemezsiniz; sigorta 'ÇAT' diye atar (Open). Sorunlu odaya giden elektrik kesilir ama evin geri kalanı yaşamaya devam eder. 2) Bulkhead (Gemideki Su Geçirmez Bölmeler): Titanik gibi devasa bir geminin gövdesi 10 ayrı su geçirmez odaya bölünmüştür. Bir oda delinip su alsa bile su diğer odalara geçmez ve gemi batmaz. 3) Jitter'lı Yeniden Deneme: 100 kişinin aynı kapıyı aynı anda tekmelemesi yerine, herkesin rastgele 1 ile 5 saniye bekleyip kapıyı çalmasıdır!

🇬🇧 ENGLISH (ELI5)

1) Circuit Breaker (Electrical Fuse): When a socket sparks, you don't let the entire house burn down; the fuse trips 'SNAP!' (Open). Electricity isolates that room while the rest of the house stays lit. 2) Bulkhead (Ship Hull Compartments): A submarine hull divides into isolated watertight chambers. If chamber 3 floods, steel doors seal it off; the submarine stays buoyant. 3) Exponential Backoff with Jitter: Instead of 1,000 clients furiously hammering an overwhelmed server every exact 1.0 second (Retry Storm), each client waits an exponentially growing duration plus random jitter (e.g. 1.2s, 2.7s, 5.1s) to disperse the thundering herd!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dağıtık dayanıklılığın 3 altın deseni: 1) Circuit Breaker (Resilience4j / Envoy): Kapalı (Normal), Açık (Hata oranı %50'yi aşarsa istekleri uzak servise göndermeden doğrudan Fallback döner - Fail-Fast), Yarı-Açık (Süre dolunca deneme amaçlı birkaç istek geçirir). 2) Bulkhead: Thread havuzlarını ve bağlantı havuzlarını birbirinden izole eder (ör. Ödeme servisi yavaşlasa bile Arama servisinin thread havuzunu tüketip tüm pod'u kilitleyemez). 3) Exponential Backoff with Full Jitter: Uzak servis çöktüğünde yeniden deneme süresini üstel olarak artırmak yetersizdir; tüm istemciler aynı anda denerse 'Retry Storm' oluşur. Formül: Sleep = rand(0, min(Cap, Base × 2^attempt)).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The trifecta of distributed stability: 1) Circuit Breaker: States: Closed (normal execution), Open (failure threshold exceeded, failing-fast immediately with graceful degraded fallback responses), Half-Open (canary testing backend recovery). 2) Bulkhead Pattern: Partitions thread pools, CPU allocations, and database connection pools across domain features. A hung Recommendation engine cannot exhaust the thread pool dedicated to Payment processing. 3) Exponential Backoff with Full Jitter: Simple exponential backoff synchronizes retry spikes into destructive waves (Retry Storms). Amazon AWS proved that injecting randomized Full Jitter (t = random(0, min(M, B × 2^i))) flattens concurrency peaks into smooth traffic profiles.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Exponential Backoff with Full Jitter Implementasyonu
import java.util.concurrent.ThreadLocalRandom;

public class ResilientRetry {
    public static long calculateFullJitterSleep(int attempt, long baseMillis, long capMillis) {
        long exponential = Math.min(capMillis, baseMillis * (1L << attempt));
        return ThreadLocalRandom.current().nextLong(0, exponential + 1);
    }
}

void main() {
    long base = 100; // 100ms
    long cap = 5000; // 5000ms tavan

    IO.println("--- Jitter'lı Yeniden Deneme Bekleme Süreleri ---");
    for (int attempt = 0; attempt < 5; attempt++) {
        long sleep = ResilientRetry.calculateFullJitterSleep(attempt, base, cap);
        IO.println("Deneme #" + (attempt + 1) + " -> Uyku: " + sleep + " ms");
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bağlantı hatası olursa hemen while(true) döngüsüyle tekrar denerim' demeyin! Kontrolsüz yeniden denemeler çökmekte olan bir sunucuyu 'Retry Storm' ile tamamen öldürür. Her zaman Circuit Breaker, Exponential Backoff ve Jitter birlikte kullanılmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch immediate tight-loop retries. Unconstrained retries create catastrophic Retry Storms that obliterate recovering downstream dependencies. Always combine Circuit Breakers with Exponential Backoff and Full Jitter.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Circuit Breaker yavaşlayan servislere giden trafiği keserek hızlı hata (Fail-Fast) üretir.

• Bulkhead deseni kaynak havuzlarını izole ederek tek bir servisin tüm pod'u kilitlemesini önler.

• Full Jitter, yeniden deneme fırtınalarını (Retry Storms) dağıtarak sistemi yumuşakça kurtarır.

🎯 Key Takeaways for the Interview

• Circuit Breakers fail fast during downstream outages, preserving upstream resource capacity.

• The Bulkhead pattern partitions thread pools, isolating failures to single sub-systems.

• Full Jitter stochastically disperses retries, dissipating destructive thundering herds.

BÖLÜM 7 / MODULE 7

Klasik MAANG Tasarım Problemleri - Bölüm 1

Classic MAANG System Designs - Part 1

TinyURL (Base62), Pastebin (S3 + TTL), Dağıtık Rate Limiter (Redis Lua), Web Crawler (URL Frontier), Bildirim Servisi ve Proximity/Yelp (Geohash vs QuadTree).

TinyURL (Base62 encoding), Pastebin (S3 + TTL), Distributed Rate Limiter (Redis Lua), Web Crawler (URL Frontier), Notification System, and Yelp Proximity (Geohash vs QuadTree).

37

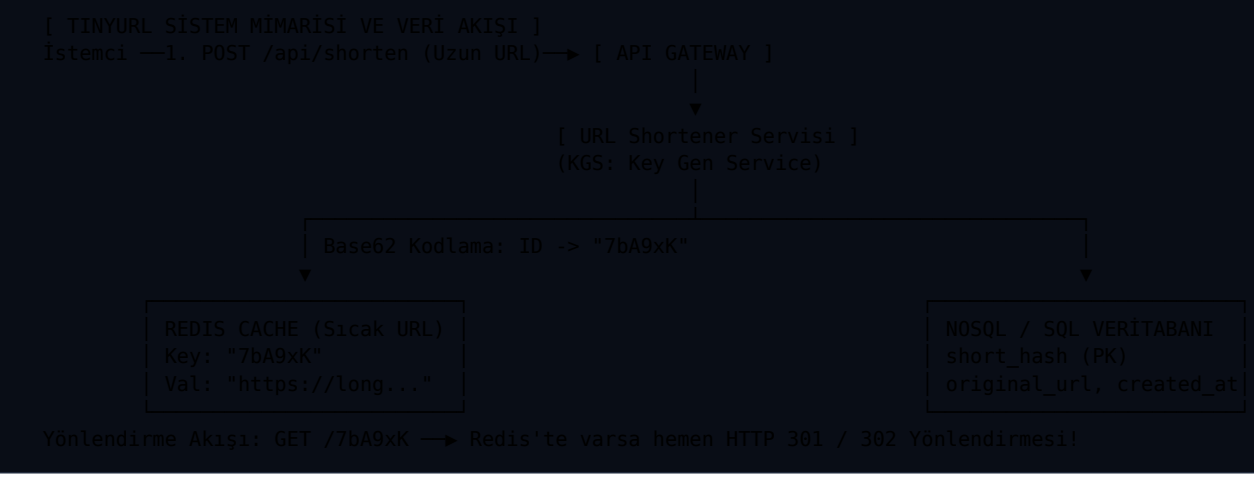
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

URL Kısaltıcı Tasarımı (TinyURL / Bit.ly): Base62, Çarpışma Önleme ve 100M DAU

Designing a URL Shortener (TinyURL / Bit.ly): Base62 & Scale

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki 500 sayfalık kalın bir kitabın adını arkadaşınıza söylerken kitabın tüm içeriğini okumazsınız; ona 'A-12 nolu raftaki kitap' dersiniz (Kısa Kod). Arkadaşınız rafa gider ve koca kitabı bulur. TinyURL de budur: 200 karakterlik devasa bir internet adresini alır (`https://example.com/very/long/url...`), onu 7 karakterlik sevimli bir plakaya (`tinyurl.com/a8B9z1`) dönüştürür. Biri o plakayı tıkladığında sistem plakayı gerçek adrese yönlendirir!

🇬🇧 ENGLISH (ELI5)

Instead of shouting the entire 500-page contents of an encyclopedia across a room, you hand your friend a tiny library call number: 'Shelf B-42' (Short Code). Your friend walks to shelf B-42 and retrieves the tome. TinyURL works identically: it takes an unruly 250-character URL and hashes it into a compact 7-character vanity token (`tiny.one/x9K2pQ`). When a browser accesses that token, the server returns an instant HTTP redirect to the long target destination!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi TinyURL Tasarım Analizi: 1) Kapasite Hesabı: 100M yeni URL/gün. 10 yılda: 100M x 365 x 10 ≈ 365 Milyar URL. 2) Base62 Kodlama: [0-9, a-z, A-Z] = 62 karakter. 7 karakterlik bir kısa kod: 62^7 = 3.52 × 10^12 (3.5 Trilyon benzersiz kombinasyon sunar; 365 milyarı fazlasıyla karşılar). 3) Hash Çarpışması Önleme: MD5/SHA256 alıp ilk 7 karakteri kesmek çarpışma (Collision) riski taşır. En iyi mimari: 'Key Generation Service' (KGS) ile bağımsız bir servisin önceden benzersiz 7 karakterlik Base62 anahtarları üretip veritabanına koymasıdır. 4) HTTP 301 vs 302: 301 (Moved Permanently) tarayıcı tarafından önbelleğe alınır, sunucu yükünü düşürür ancak tıklama analitiğini (analytics) bozar. 302 (Found) geçicidir; her istek sunucudan geçer, tıklama sayımı kusursuz çalışır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG-Grade Architecture for TinyURL: 1) Capacity: 100M writes/day. 10-year horizon: 365 billion URLs. 2) Base62 Encoding: [0-9, a-z, A-Z] yields 62 alphanumeric permutations. A 7-character token provides 62^7 ≈ 3.52 trillion unique keys. 3) Collision Elimination: Hashing the original URL via SHA-256 and truncating introduces collision loops. Optimal approach: Key Generation Service (KGS) that generates sequential monotonic Snowflake IDs and encodes them to Base62, pre-loading tokens into memory. 4) HTTP 301 vs 302 Redirects: 301 (Moved Permanently) is cached by the client browser, reducing origin QPS while breaking real-time click telemetry. 302 (Found / Temporary Redirect) forces every click through the gateway, capturing user-agent, referer, and IP analytics.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: 64-Bit Sayıyı Base62 Karakter Dizisine Dönüştürücü (URL Shortener Motoru)
public class Base62Encoder {
    private static final String BASE62_ALPHABET =
"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

    public static String encode(long number) {
        if (number == 0) return "0";
        var sb = new StringBuilder();
        while (number > 0) {
            int remainder = (int) (number % 62);
            sb.append(BASE62_ALPHABET.charAt(remainder));
            number /= 62;
        }
        return sb.reverse().toString();
    }
}

void main() {
    long uniqueId = 12519284192L; // Snowflake veya DB Identity
    String shortCode = Base62Encoder.encode(uniqueId);
    IO.println("Benzersiz ID: " + uniqueId + " ⟶ Kısa Kod: " + shortCode);
    IO.println("Oluşan URL:   https://tiny.url/" + shortCode);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'MD5 hash'i alıp ilk 7 karakterini keserim' deyip geçmeyin! İki farklı URL aynı ilk 7 karaktere sahip olabilir (Çarpışma). Mülakatçıya Snowflake ID + Base62 dönüşümünü veya KGS (Key Generation Service) ön-üretim modelini sunduğunuzda doğrudan Staff seviyesine çıkarsınız.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Suggesting a naive SHA-256 truncation without addressing hash collisions is an immediate deduction. Propose a distributed Key Generation Service (KGS) or a monotonic Snowflake sequence mapped deterministically through Base62.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Base62 [0-9, a-z, A-Z] 7 karakterde 3.5 trilyon benzersiz kombinasyon üretir.
- HTTP 302 yönlendirmesi tıklama analitiğini takip etmek için tercih edilir.
- Okuma/Yazma oranı 100:1 olduğu için Redis önbelleği okuma gecikmesini \<5ms seviyesine indirir.

🎯 Key Takeaways for the Interview

- Base62 with 7 characters yields 3.52 trillion collision-free combinations.
- HTTP 302 redirects preserve accurate click telemetry and analytics at the gateway.
- A 100:1 read-to-write ratio mandates an aggressive Redis cache for sub-5ms lookups.

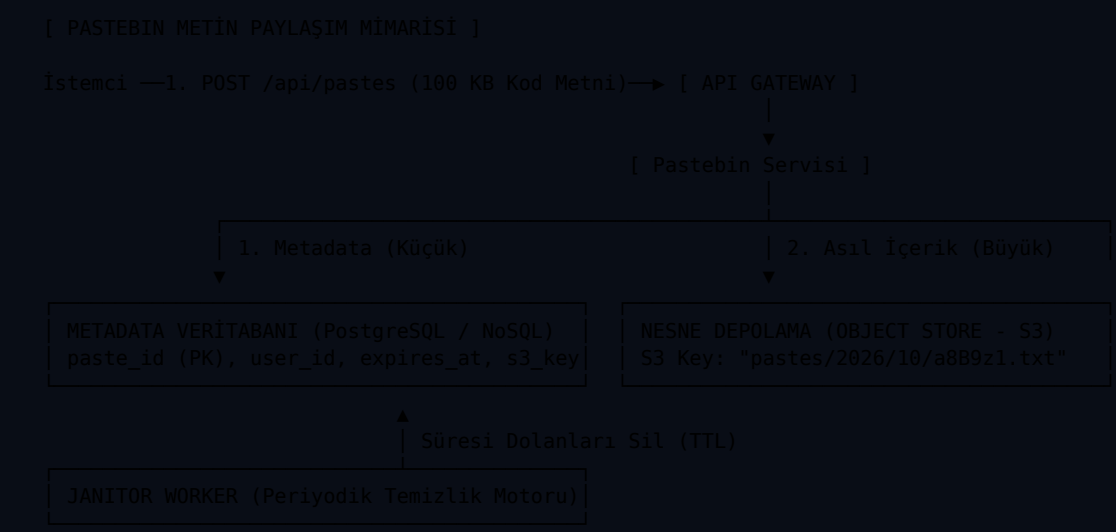
38

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Pastebin / Metin Paylaşım Servisi Tasarımı: TTL, S3 Blob Depolama ve Temizlik

Designing Pastebin: Object Storage (S3), TTL & Janitor Service

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir kütüphanede emanet dolapları olduğunu düşünün: Dolabın kapağında sadece emanet fişi ve son teslim tarihi yazar (Metadata), dolabın içinde ise koca bir bavul eşya durur (Büyük Metin / S3). Siz fiş numarasını verirsiniz, görevli kapağa bakar, bavulunuzu çıkarıp verir. Eğer dolabın süresi dolmuşsa (TTL), gece gelen temizlik görevlisi (Janitor Service) bavulu boşaltıp çöp fırınına atar!

🇬🇧 ENGLISH (ELI5)

Think of a luggage locker terminal at a central train station: The digital locker display shows only the locker ticket ID and expiration date (Metadata), while the spacious locker compartment holds a massive 50-pound travel suitcase (Text Payload / S3). You scan your ticket; the door opens. If a traveler abandons their locker beyond the paid rental window (TTL), a nightly maintenance crew (Janitor Service) empties the locker to free up capacity!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Pastebin ve TinyURL arasındaki en kritik fark: TinyURL'de veri birkaç yüz bayt iken, Pastebin'de her içerik 10 KB - 10 MB arasında devasa metinler içerebilir. 1) Veri Ayrımı (Storage Segregation): Büyük metin gövdesi veritabanında (RDBMS) ASLA `TEXT/BLOB` olarak tutulmaz! Metin doğrudan ucuz ve sınırsız ölçeklenen Nesne Depolamada (AWS S3, MinIO) saklanır. Veritabanında yalnızca metadata tutulur (`paste_id`, `s3_key`, `created_at`, `expires_at`). 2) Süre Aşımı ve Temizlik (TTL & Janitor): Süresi dolan kayıtlar veritabanından anında silinmez (DB kilitlenmesini önlemek için). İstemci okurken `expires_at \< now()` ise '404 Bulunamadı' dönülür. Arka planda çalışan bir 'Janitor Service' (gece batch worker) süresi dolan kayıtları ve S3 dosyalarını toplu olarak temizler.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The critical divergence between TinyURL and Pastebin is payload magnitude: TinyURL stores 200-byte strings, while Pastebin accepts 100KB to 10MB text blobs. Architectural Tenet: Never store unbounded text payloads directly inside relational database tables! Store raw payloads inside cost-effective Object Stores (AWS S3 / Google Cloud Storage), persisting only compact metadata inside PostgreSQL or Cassandra (`paste_id`, `author_id`, `s3_uri`, `expiration_timestamp`). TTL & Lazy Cleanup: Avoid running active delete queries on hot tables. On read paths, check `expires_at \< NOW()` and return 404. Offload actual physical deletions to an asynchronous Janitor Worker that batch-purges expired S3 blobs and DB rows during off-peak hours.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Pastebin Metadata & Süre Kontrol Modeli
import java.time.Instant;

public record PasteMetadata(String pasteId, String s3Uri, Instant expiresAt) {
    public boolean isExpired() {
        return Instant.now().isAfter(expiresAt);
    }
}

void main() {
    var activePaste = new PasteMetadata("pst_101", "s3://pastes/pst_101.txt",
Instant.now().plusSeconds(3600));
    var expiredPaste = new PasteMetadata("pst_102", "s3://pastes/pst_102.txt",
Instant.now().minusSeconds(10));

    IO.println("Paste 101 Süresi Doldu mu? " + activePaste.isExpired()); // false
    IO.println("Paste 102 Süresi Doldu mu? " + expiredPaste.isExpired()); // true -> HTTP 404!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta '10 MB'lık kod parçalarını PostgreSQL tablosunda VARCHAR sütununda saklarım' derseniz elenirsiniz! Veritabanına büyük metin yazmak disk IO ve bellek önbelleğini şişirir; metin daima S3/Blob depolamada saklanmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose storing multi-megabyte paste payloads inside an RDBMS table column. This severely fragments database pages and destroys buffer pool caching. Payloads belong in Object Storage (S3).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Büyük veri yükleri (payloads) Nesne Depolamada (S3), metadata ise veritabanında tutulur.

• Süresi dolan içerikler için tembel kontrol (Lazy Evaluation) ve periyodik Janitor Worker kullanılır.

• Kullanıcıların paste aramaları için metadata veritabanında paste_id ve user_id indekslenir.

🎯 Key Takeaways for the Interview

• Large payloads live in Object Storage (S3); metadata lives in relational/document DBs.

• Combine lazy read-time expiration checks with asynchronous background janitor purging.

• Index author IDs and expiration timestamps to support high-performance user dashboard queries.

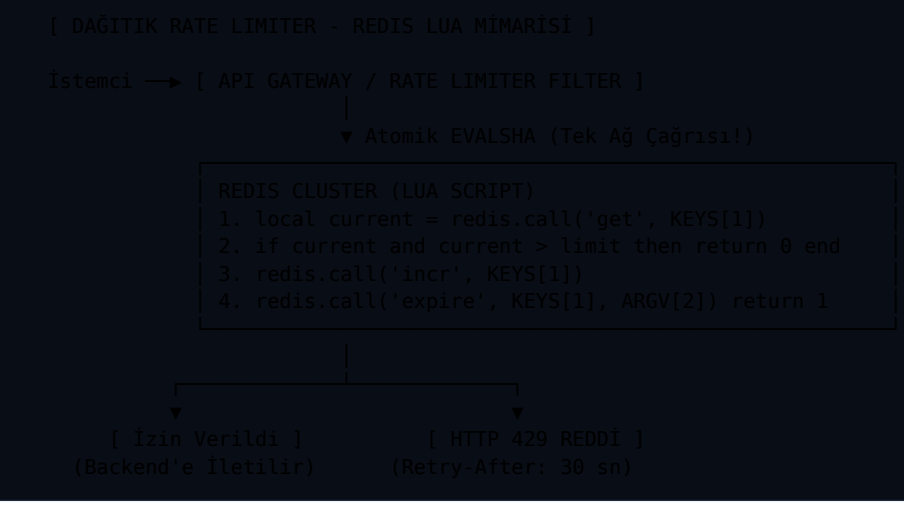
39

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Dağıtık Hız Sınırlayıcı (Distributed Rate Limiter) Tasarımı: Redis Lua ve Multi-Region

Designing a Distributed Rate Limiter: Redis Lua & Multi-Region

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gece kulübünün kapısındaki fedaiyi düşünün (Rate Limiter). Kulübün kuralı: 'Bir kişi dakikada en fazla 5 kez içeri girip çıkabilir'. Fedainin elinde telsizle bağlı olduğu merkezi bir liste (Redis) vardır. Bir kişi kapıya geldiğinde fedai listeye bakar, sayacı 1 artırır. Eğer 5 hakkını doldurduysa 'Dur bakalım, 1 dakika sokakta bekle!' der (HTTP 429). Lua Script ise fedainin listeye bakıp sayıyı artırma işini göz açıp kapayıncaya kadar tek bir saniyede (Atomik olarak) yapmasıdır; böylece iki kişi aynı anda kapıya yüklense bile fedaiyi kandıramaz!

🇬🇧 ENGLISH (ELI5)

Picture a VIP nightclub bouncer guarding the velvet rope (Rate Limiter). House rule: 'No patron can enter more than 5 times per minute'. The bouncer holds a radio linked to the central clipboard (Redis). When a patron approaches, the bouncer checks the tally, increments the counter, and if the count exceeds 5, blocks entry with 'Wait 60 seconds on the curb!' (HTTP 429). The Redis Lua script ensures the bouncer checks and increments the clipboard atomically in a single nanosecond, preventing two patrons from rushing the rope simultaneously!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Dağıtık Hız Sınırlayıcı Mimarisi: 1) Yarış Durumu (Race Condition) Çözümü: Eğer uygulama Redis'ten `GET` yapıp kodda artırıp `SET` yaparsa, eşzamanlı iki istek aynı değeri okur ve limit aşılır. Çözüm: Redis Lua Scripting. Lua betikleri Redis motorunda tek iş parçacığında atomik olarak çalışır; sıfır kilit maliyetiyle `GET + INCR + EXPIRE` tek adımda biter. 2) Çok Bölgeli (Multi-Region) Senkronizasyon: Kullanıcı hem ABD hem Avrupa bölgesinden istek atıyorsa ne olur? Çözüm: A) Merkezi Redis (Ağ gecikmesi ekler). B) Yerel Redis + Asenkron Sync (CRDT / Eventual Consistency): Her bölgeye kotanın bir kısmı tahsis edilir (Local Quota Allocation) veya asenkron eşleme yapılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed Rate Limiter Architecture: 1) Atomic Race Condition Elimination: Executing a separate `GET`, incrementing in JVM memory, and writing back via `SET` introduces fatal concurrency race windows. Resolution: Redis Lua Scripting executes `GET`, conditional comparison, `INCR`, and `EXPIRE` atomically within the Redis single-threaded engine in a single network round-trip. 2) Multi-Region Synchronization: If a user fires requests simultaneously across US-East and EU-West, cross-datacenter synchronization is required. Architecture Options: A) Centralized primary Redis cluster (adds 100ms cross-Atlantic latency). B) Localized Quota Splitting: Partition a user's 100 req/min allowance into 50 for US and 50 for EU, synchronizing background replenishment asynchronously via batch consensus.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Redis Lua Script Tabanlı Rate Limiting Mantığı Simülasyonu
public class RedisLuaRateLimiter {
    public static final String LUA_SCRIPT = """
        local key = KEYS[1]
        local limit = tonumber(ARGV[1])
        local window = tonumber(ARGV[2])

        local current = redis.call('get', key)
        if current and tonumber(current) >= limit then
            return 0 -- İzin Verilmedi (HTTP 429)
        else
            redis.call('incr', key)
            if not current then
                redis.call('expire', key, window)
            end
            return 1 -- İzin Verildi (HTTP 200)
        end
        """;

    public record RateLimitResult(boolean allowed, int remainingLimit, int retryAfterSeconds) {}
}

void main() {
    IO.println("Redis Lua Script Hazırlandı (Atomik GET + INCR + EXPIRE)");
    IO.println("Script Boyutu: " + RedisLuaRateLimiter.LUA_SCRIPT.trim().length() + " karakter");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta rate limiting için istemci IP adresini tek başına anahtar (Key) olarak seçmeyin! Aynı kurumsal ofisteki veya üniversitedeki 10.000 kişi aynı NAT Gateway IP'sini paylaşabilir; tek bir kullanıcının kotası dolunca tüm şirket bloklanır. Anahtar: `UserId` (Giriş yapmışsa) veya `UserId + IP` olmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never rely solely on client IP for rate limiting! Behind enterprise NAT proxies or university campuses, 10,000 distinct employees share a single public IP. Key by authenticated `UserId` or a combination of `DeviceFingerprint + IP`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis Lua Scripting yarış durumlarını (Race Conditions) sıfır kilit maliyetiyle çözer.
- Kullanıcı deneyimi için HTTP 429 yanıtı ile birlikte 'Retry-After' başlığı dönülmelidir.
- Çok bölgeli sistemlerde yerel kota tahsisi (Local Quota Splitting) ağ gecikmesini engeller.

🎯 Key Takeaways for the Interview

- Redis Lua scripts execute check-and-increment operations atomically without distributed locks.
- Always accompany HTTP 429 responses with accurate Retry-After and X-RateLimit headers.
- Local quota partitioning eliminates cross-datacenter WAN latency on edge rate checks.

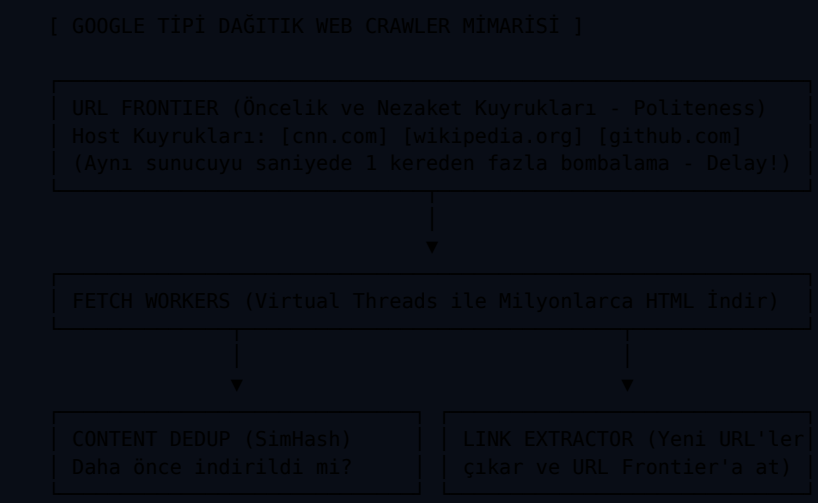
40

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Web Gezgini (Web Crawler) Tasarımı: URL Frontier, robots.txt ve Duplication

Designing a Web Crawler: URL Frontier, Politeness & Deduplication

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki tüm kitapları tek tek okuyup fihrist çıkaran çalışkan bir robot düşünün (Web Gezgini / Googlebot). Robot bir sayfa okur, sayfanın altındaki dipnotlarda başka kitapların referansını görür (Link Extraction). O yeni kitapları 'Okunacaklar Listesi'ne (URL Frontier) ekler. Ancak robot çok terbiyelidir (Politeness): Aynı yazarın odasına saniyede 100 kez girip kapıyı çalmaz; her yazarın arasına 2 saniye bekleme süresi koyar. Ayrıca daha önce okuduğu bir kitabı tekrar okumamak için hafızasındaki parmak izlerini (Deduplication) kontrol eder!

🇬🇧 ENGLISH (ELI5)

Imagine an automated librarian cataloging every book ever printed (Googlebot). It opens a book, reads the citations at the bottom (Link Extraction), and writes those new book titles onto a massive 'To-Read Clipboard' (URL Frontier). The librarian is strictly disciplined and polite (Politeness): it never pesters the same author 100 times per second, inserting a courteous 1-second delay between visits. It also checks a fingerprint catalog (Deduplication) so it never rereads identical duplicate books!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Web Crawler Mimarisi: 1) URL Frontier: İndirilecek URL'leri tutan kalıcı kuyruk sistemi. İki temel gereksinim: A) Önceliklendirme (Priority): Kaliteli ve popüler sayfalar önce taranır (PageRank tabanlı). B) Nezaket (Politeness): Aynı hedef alan adına (Host/Domain) giden isteklerin arasına gecikme (ör. 1000ms) konur; hedef sunucu DDoS saldırısına uğramaz. 2) DNS Önbellekleme: Milyonlarca istekte DNS sorguları darboğaz olmasın diye yerel DNS Cache tutulur. 3) İçerik Tekilleştirme (Content Deduplication): İnternetteki içeriğin %30'u mükerrerdir. HTML içeriğinin SimHash veya MinHash parmak izi alınarak Hamming Mesafesi (Hamming Distance) ile benzerlik kontrol edilir. 4) robots.txt: Taranmadan önce sitenin `robots.txt` kuralları okunur ve önbelleğe alınır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of a Distributed Web Crawler (Googlebot scale): 1) The URL Frontier: Governs URL scheduling via a dual-tiered queueing engine: A) Priority Queues order pages by freshness and PageRank. B) Politeness Queues group URLs by domain hostname, ensuring worker threads enforce delay intervals (e.g., 500ms) to avoid crashing target web hosts. 2) Dedicated Local DNS Caching: DNS lookups represent 30%+ of network latency; crawlers maintain in-memory high-throughput DNS caches. 3) Near-Duplicate Content Elimination: Over 30% of web pages are identical mirrors. Crawlers compute cryptographic 64-bit `SimHash` or `MinHash` locality-sensitive signatures, comparing Hamming Distance to discard duplicates before indexing. 4) robots.txt Compliance: Robots Exclusion Protocol files are fetched and cached in memory prior to scraping target paths.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: SimHash Benzerlik ve Tekilleştirme (Deduplication) Algoritması
import java.math.BigInteger;

public class SimHashDeduplicator {
    public static long computeSimHash(String text) {
        int[] v = new int[64];
        String[] words = text.toLowerCase().split("\\s+");
        for (String word : words) {
            long hash = word.hashCode(); // Basit hash simülasyonu
            for (int i = 0; i < 64; i++) {
                if (((hash >> i) & 1L) == 1L) v[i]++;
                else v[i]--;
            }
        }
        long simHash = 0L;
        for (int i = 0; i < 64; i++) {
            if (v[i] > 0) simHash |= (1L << i);
        }
        return simHash;
    }

    // İki parmak izi arasındaki Hamming Mesafesi (Farklı bit sayısı)
    public static int hammingDistance(long h1, long h2) {
        return Long.bitCount(h1 ^ h2);
    }
}

void main() {
    String doc1 = "Java 25 and Spring Boot system design interview guide";
    String doc2 = "Java 25 and Spring Boot system design interview tutorial"; // Çok benzer!

    long hash1 = SimHashDeduplicator.computeSimHash(doc1);
    long hash2 = SimHashDeduplicator.computeSimHash(doc2);

    int distance = SimHashDeduplicator.hammingDistance(hash1, hash2);
    IO.println("Hamming Mesafesi: " + distance + " bit (Mesafe <= 3 ise MÜKERRER KABUL EDİLİR!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'BFS (Breadth-First Search) ile tararım' deyip geçmeyin! Saf BFS yaparsanız tek bir sitedeki 100.000 linki sırayla indirip o web sitesini çökertebilirsiniz (İhlal: Impolite). Nezaket (Politeness) kuyrukları ile farklı hostların dönüşümlü taranması zorunludur.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose naive Breadth-First Search (BFS) for web crawling. A pure BFS traversal hammers a single domain with thousands of concurrent connections, effectively staging a DDoS attack. Explicitly design Politeness queues.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• URL Frontier, önceliklendirme ve nezaket (politeness) dengesini kuran omurgadır.

• SimHash ve MinHash algoritmaları milyarlarca web sayfasındaki benzerlikleri saniyede eler.

• Yerel DNS önbellekleme, web tarama hızını %30 artıran en kritik optimizasyondur.

🎯 Key Takeaways for the Interview

• The URL Frontier enforces both crawl prioritization and domain politeness constraints.

• SimHash algorithms detect near-duplicate pages in constant time using Hamming distance.

• Dedicated in-memory DNS caching eliminates substantial latency bottlenecks during massive crawls.

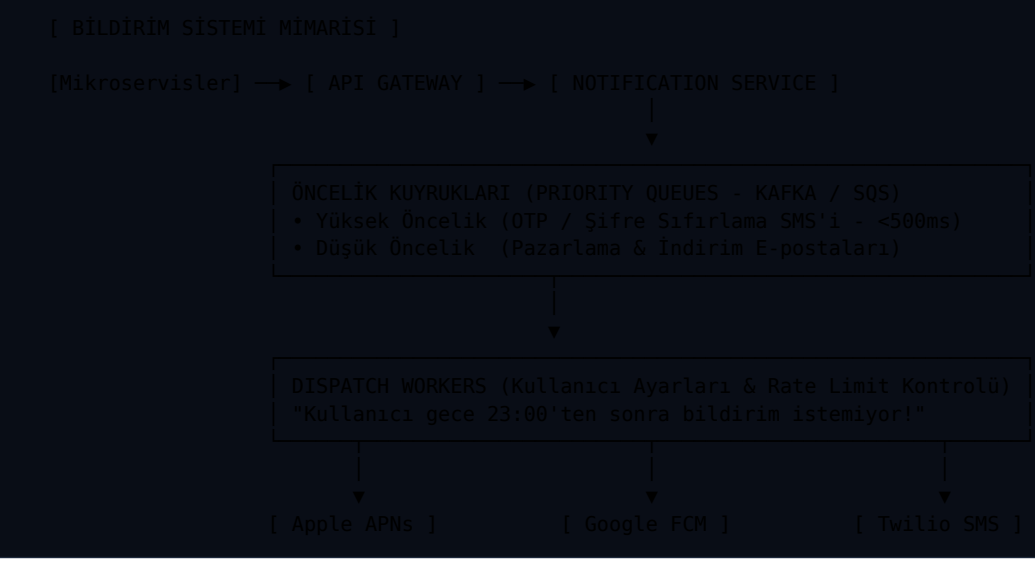
41

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Bildirim Sistemi (Notification Service) Tasarımı: Öncelik Kuyrukları ve Rate Limits

Designing a Notification System: Priority Queues & Rate Limits

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketin kargo ve mektup odasını düşünün: Masada iki sepet vardır. Kırmızı sepet (Yüksek Öncelik): 'Kullanıcının kredi kartı şifresi SMS'i'. Bu hemen 2 saniyede gönderilmelidir! Mavi sepet (Düşük Öncelik): 'Haftasonu indirim bülteni'. Bu 3 saat sonra gitse de kimse ölmez. Görevli çıraklar daima kırmızı sepetteki mektupları önce taşır. Ayrıca görevlinin elinde bir kural listesi vardır: 'Ahmet Bey'e günde 3'ten fazla reklam SMS'i atma, yoksa bizi mahkemeye verir!' Bildirim sistemi işte bu akıllı mektup odasıdır!

🇬🇧 ENGLISH (ELI5)

Picture a corporate dispatch room with two delivery trays. The Red Tray (High Priority) holds emergency 2FA one-time passwords: these MUST land on customer phones within 3 seconds! The Blue Tray (Low Priority) holds weekly marketing coupons: whether these arrive now or at 3:00 AM doesn't matter. Couriers always empty the red tray first. The dispatcher also enforces user opt-outs: 'Never send marketing pushes after 10:00 PM, and cap marketing at 2 pushes per user per day!' That is an enterprise Notification Engine!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Bildirim Sistemi Mimarisi: 1) Doğrulama & Kullanıcı Tercihleri (User Preferences): Kullanıcı hangi kanalları açtı? (iOS Push, Android FCM, SMS, E-posta). Kullanıcının 'Rahatsız Etmeyin' (Do Not Disturb - DND) saatleri ve zaman dilimi (Timezone) kontrol edilir. 2) Önceliklendirme Kuyrukları (Priority Queues): OTP ve Güvenlik alarmları yüksek öncelikli Kafka topic'lerine; bültenler düşük öncelikli kuyruklara aktarılır. Düşük öncelikli yoğunluk yüksek öncelikli SMS'leri asla bloklayamaz. 3) Hız Sınırlama & Birleştirme (Rate Limiting & De-duplication): Kullanıcıya 1 saatte en fazla 1 SMS gitmesi kuralı Redis ile denetlenir. Benzer bildirimler birleştirilir (Notification Aggregation, ör. 'Ali ve 5 kişi fotoğrafını beğendi'). 4) Üçüncü Parti Sağlayıcılar: Apple APNs, Google FCM, Twilio SMS, SendGrid E-posta entegrasyonları.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Notification System Blueprint: 1) Preference & Consent Engine: Evaluates user settings (APNs vs FCM vs Twilio SMS vs SendGrid), opt-out flags, and local timezone Do Not Disturb (DND) windows. 2) Strict Queue Isolation: Segregates critical transactional OTPs into dedicated high-priority Kafka topics, keeping them isolated from marketing campaign backlogs. 3) Frequency Capping & Aggregation: Redis counters enforce frequency caps (e.g. max 3 promotional pushes/day). Aggregation workers batch high-velocity social notifications (e.g. 'Alice and 14 others liked your post') to avoid notification fatigue. 4) Resilient Downstream Adapters: Wrap third-party gateway integrations (Twilio, APNs) in Circuit Breakers with dead-letter fallback queues.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Öncelikli Bildirim Yönlendirme Modeli
public class NotificationRouter {
    public enum Priority { HIGH_OTP, LOW_MARKETING }
    public record Notification(String userId, String channel, String message, Priority priority) {}

    public String routeToQueue(Notification notif) {
        return switch (notif.priority()) {
            case HIGH_OTP -> "QUEUE_CRITICAL_HIGH_SPEED (SLA < 1s)";
            case LOW_MARKETING -> "QUEUE_BULK_BATCH (SLA < 4 hours)";
        };
    }
}

void main() {
    var router = new NotificationRouter();
    var otp = new NotificationRouter.Notification("u1", "SMS", "Kodunuz: 9481",
NotificationRouter.Priority.HIGH_OTP);
    var promo = new NotificationRouter.Notification("u2", "PUSH", "%50 İndirim Başladı!",
NotificationRouter.Priority.LOW_MARKETING);

    IO.println("OTP Yolu:   " + router.routeToQueue(otp));
    IO.println("Promo Yolu: " + router.routeToQueue(promo));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Tüm bildirimleri tek bir RabbitMQ kuyruğuna atarım' demeyin! 10 milyon kullanıcılı bir e-posta kampanyası başladığında kuyruk şişer ve banka şifresi bekleyen bir kullanıcının SMS'i 45 dakika sonra gider. Kritik OTP'ler ile pazarlama bildirimleri KESİNLİKLE ayrı kuyruklarda izole edilmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Funneling all notifications into a single shared queue is an architectural catastrophe. A 5-million marketing campaign will choke the queue, delaying password reset OTPs by 30 minutes! High-priority OTPs require dedicated, isolated queue infrastructure.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kritik OTP ve güvenlik bildirimleri pazarlama kuyruklarından tamamen izole edilir.
- Kullanıcı tercihleri, zaman dilimi ve DND saatleri bildirim fırlatılmadan önce denetlenir.
- Benzer bildirimler (Aggregation) birleştirilerek kullanıcının spam hissetmesi engellenir.

🎯 Key Takeaways for the Interview

- Isolate transactional OTP alerts from marketing campaigns into dedicated queues.
- Filter notifications against user opt-outs, timezones, and frequency caps before dispatch.
- Notification aggregation batches high-frequency interactions into clean digests.

42

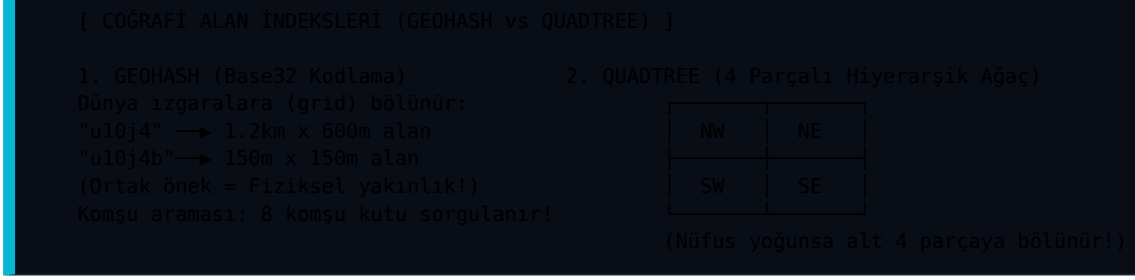
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Yakındaki Yerler / Mekan Arama Servisi (Proximity / Yelp): Geohash vs QuadTree vs Google S2

Designing a Proximity Service (Yelp / Google Maps): Geohash vs QuadTree

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Haritada 'Bana en yakın 10 kahveciyi bul' dediğinizde, dünyadaki 500 milyon kahvecinin koordinatını tek tek Pisagor teoremiyle ($x^2 + y^2$) hesaplamaya çalışırsanız sunucu patlar! Bunun yerine dünya bir satranç tahtası gibi karelere bölünür (Geohash). Her karenin posta kodu gibi bir adı vardır (ör. 'u10j4'). Siz o karenin içindeyseniz, sistem sadece o karenin ve etrafındaki 8 komşu karenin içindeki kahvecilere bakar. 500 milyon dükkan yerine sadece 50 dükkana bakarak 1 milisaniyede sonucu bulur!

🇬🇧 ENGLISH (ELI5)

Searching 'Find the 10 closest cafes near me' cannot compute the Pythagorean distance formula across all 500 million cafes worldwide on every GPS ping! Instead, divide the planet into an indexed grid of boxes (Geohash). Each grid box gets an alphanumeric postal code (e.g. `u10j4`). When you query from that box, the system simply queries that box plus its 8 immediate neighboring boxes. The search space shrinks from 500 million points down to 50 in 1 millisecond!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Mekan Arama (Proximity) Mimarisi: 1) Neden Standart SQL İndeksi Yetmez? Enlem (Latitude) ve Boylam (Longitude) iki boyutlu veridir; standart B-Tree indeksinde `WHERE lat BETWEEN ... AND lon BETWEEN ...` sorgusu indeks birleştirmesi (Index Merge) yüzünden çok yavaştır. 2) Geohash: Enlem ve boylamı iç içe geçirerek Base32 karakter dizisine kodlar (1D string). Özellik: Ortak önek (Prefix) paylaşan noktalar fiziksel olarak birbirine yakındır (Örn: `u10j4` 6 karakter ≈ 1.2 km). Kenar etkisi (Edge Effect) için merkez kutu + 8 komşu kutu sorgulanır. Redis `GEOADD` ve `GEORADIUS` komutları arka planda Geohash ve Sorted Set kullanır. 3) QuadTree: Her düğümün en fazla 4 çocuk düğümü olan hiyerarşik ağaçtır. Düğümdeki mekan sayısı eşiği (ör. 100 dükkan) aşınca o alan 4 alt parçaya bölünür (Yoğun şehir merkezlerinde derin, okyanusta sığ ağaç). 4) Google S2: Dünyayı küp projeksiyonu ve Hilbert Eğrisi ile 64-bit tam sayılara böler; Google Haritalar'ın standardıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Proximity Service Architecture: 1) The 2D Spatial Indexing Challenge: Traditional B-Trees index single dimensions efficiently; evaluating `WHERE lat BETWEEN x AND y AND lon BETWEEN a AND b` degrades to full table filtering. 2) Geohash: Interleaves binary bits of latitude and longitude into a Base32 string. Key Trait: Strings sharing prefixes share spatial proximity (`u10j4` has 1.2km precision). The Boundary Problem: Two adjacent points across a boundary can share completely different prefixes; solved by querying the central box plus its 8 surrounding neighbor cells. Redis native geospatial commands (`GEOADD`, `GEOSEARCH`) leverage 52-bit Geohashes within Sorted Sets. 3) QuadTree: In-memory 2D spatial tree where each quadrant recursively splits into 4 sub-quadrants when point density exceeds a threshold (e.g., 100 POIs). Deeply sub-divided in Manhattan; undivided across deserts. 4) Google S2 Geometry: Projects the spherical earth onto a cube using Space-Filling Hilbert Curves into 64-bit integers; de facto standard for Uber and Google Maps.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Basitleştirilmiş Geohash Önek Eşleme ve Komşu Filtreleme
import java.util.*;

public class GeohashProximityEngine {
    public record Place(String name, String geohash6) {}

    private final Map<String, List<Place>> geohashIndex = new HashMap<>();

    public void indexPlace(Place place) {
        geohashIndex.computeIfAbsent(place.geohash6(), k -> new ArrayList<>()).add(place);
    }

    public List<Place> searchNearby(String userGeohash5Prefix) {
        List<Place> results = new ArrayList<>();
        // 5 karakterlik önek (~4.9 km x 4.9 km alan) ile başlayan tüm yerleri topla
        for (var entry : geohashIndex.entrySet()) {
            if (entry.getKey().startsWith(userGeohash5Prefix)) {
                results.addAll(entry.getValue());
            }
        }
        return results;
    }
}

void main() {
    var engine = new GeohashProximityEngine();
    engine.indexPlace(new GeohashProximityEngine.Place("Kadıköy Kahvecisi", "sxk97a"));
    engine.indexPlace(new GeohashProximityEngine.Place("Moda Dondurmacısı", "sxk97b"));
    engine.indexPlace(new GeohashProximityEngine.Place("Ankara Cafe", "sx5f2a"));

    IO.println("Kadıköy (sxk97) Civarındaki Mekanlar:");
    for (var p : engine.searchNearby("sxk97")) {
        IO.println("• " + p.name() + " [Geohash: " + p.geohash6() + "]");
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta mekan bilgilerinin (isim, adres, menü) QuadTree veya Redis'in içinde saklanacağını söylemeyin! Mekan verisi (metadata) PostgreSQL/MySQL'de tutulur; QuadTree veya Geohash indeksinde SADECE `Place_ID` ve `Geohash` tutulmalıdır. Aksi halde önbellek belleği çöker.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not pack heavy business payloads (reviews, menu photos, descriptions) inside the spatial QuadTree/Geohash index. The spatial index must hold only compact `(Place_ID, Coordinates)` tuples; full metadata is resolved from the primary DB.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Geohash 2D koordinatları tek boyutlu string'lere dönüştürerek hızlı arama sağlar.

• Kenar etkisi (Boundary Effect) problemini çözmek için merkez hücre + 8 komşu taranır.

• QuadTree bellek içi yoğunluğa göre dinamik olarak alt parçalara bölünür.

🎯 Key Takeaways for the Interview

• Geohash flattens 2D coordinates into 1D strings, enabling standard index range queries.

• Mitigate Geohash boundary cliffs by querying the central cell plus its 8 adjacent neighbors.

• QuadTrees adapt dynamically to population density, allocating deep nodes only in dense cities.

BÖLÜM 8 / MODULE 8

Klasik MAANG Tasarım Problemleri - Bölüm 2

Classic MAANG System Designs - Part 2

WhatsApp/Slack Sohbet Sistemi, Twitter/Instagram News Feed (Fan-Out), YouTube Video Akışı (HLS/DASH), Dropbox/Drive Senkronizasyonu, Google Typeahead (Trie) ve Flaş İndirim (Ticketmaster).

WhatsApp/Slack Chat (WebSockets), Twitter News Feed (Fan-Out trade-offs), YouTube Video Streaming (HLS/DASH), Dropbox File Sync (Block Chunks), Google Search Typeahead (Trie), and Ticketmaster Flash Sales.

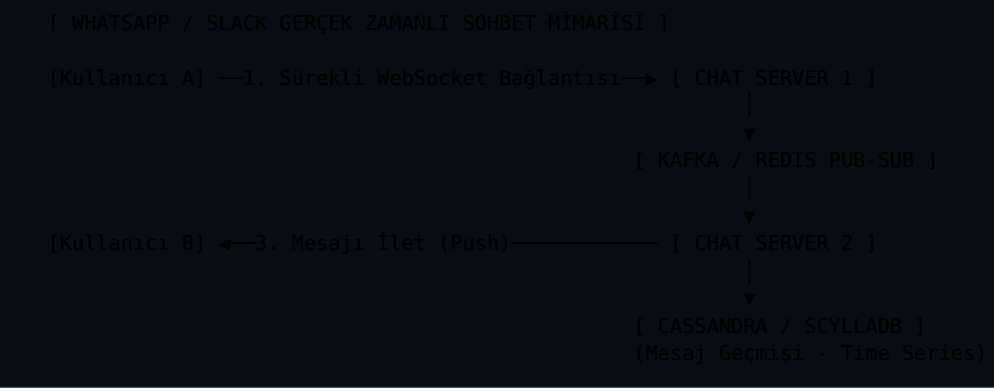
43

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Gerçek Zamanlı Sohbet Sistemi (WhatsApp / Slack): WebSockets, Presence ve Cassandra

Designing a Chat System (WhatsApp / Slack): WebSockets & Presence

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski internet sitelerinde bir arkadaşınızın size mesaj yazıp yazmadığını görmek için sayfayı her 3 saniyede bir 'Yenile' (Polling) yapardınız (kapıyı 100 kez açıp kapatmak gibi). WhatsApp ve Slack ise iki ev arasına çekilmiş kesintisiz bir telefon hattı gibidir (WebSocket). Hat bir kez açılır ve 24 saat boyunca açık kalır. Arkadaşınız 'Selam' dediği anda ses kulağınızda yankılanır. 'Çevrimiçi / Online' göstergesi ise kapının önündeki lambadır: Siz odaya girdiğinizde lamba yanar (Presence Server); 30 saniye boyunca hareket etmezseniz lamba otomatik söner!

🇬🇧 ENGLISH (ELI5)

In early web chat rooms, checking for new messages meant mashing the 'Refresh' button every 3 seconds (HTTP Polling)—like constantly slamming your front door open and shut. Modern chat apps like WhatsApp and Slack deploy a persistent, open fiber-optic intercom (WebSocket connection). The duplex socket opens once and stays active. When a friend types 'Hi', the byte stream transmits instantly. The 'Online' status indicator is a motion sensor: when your phone pings the presence gateway, the light glows green; if no heartbeat arrives for 30 seconds, it shifts to offline!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Sohbet Mimarisi Bileşenleri: 1) İletişim Protokolü: İki yönlü, düşük gecikmeli, başlık yükü minimum WebSocket protokolü kullanılır. 2) Durum Takibi (Presence Service): Kullanıcının çevrimiçi/çevrimdışı durumunu yönetir. İstemciler her 10-15 saniyede bir hafif Heartbeat (Canlılık sinyali) gönderir. Heartbeat kesilirse Redis'teki oturum süresi (TTL) dolar ve kullanıcı 'Offline' işaretlenir. 3) Mesaj Depolama (Neden Cassandra?): Sohbet mesajları zaman serisi (Time-series) verisidir; yazma hacmi aşırı yüksektir, eski mesajlar nadiren güncellenir. Cassandra'nın LSM-Tree mimarisi saniyede milyonlarca mesaj yazmayı mükemmel kaldırır. Bölümleme: `Partition Key: chat_id, Clustering Key: message_id (Snowflake)` ile mesajlar kronolojik sırada diske dökülür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Chat System Topology: 1) Transport: Full-duplex stateful WebSockets over TCP, avoiding HTTP connection handshake penalties. 2) Presence Cluster: Tracks ephemeral user heartbeats. Clients dispatch heartbeat pings every 15s. Absence of pings expires a Redis lease, broadcasting offline status to user contact channels. 3) Storage Engine (Why Cassandra / ScyllaDB?): 1-on-1 chat history exhibits append-only sequential access patterns with billions of writes per day. Relational databases choke on index rebalancing. Cassandra maps `Partition Key = conversation_id` with `Clustering Key = message_id (Snowflake time-ordered)`, storing millions of chronological messages sequentially on disk for lightning-fast pagination.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: WebSocket Oturum Eşleme ve Chat Yönlendirici Modeli
import java.util.concurrent.*;

public class ChatSessionRouter {
    // Aktif WebSocket sunucuları haritası: UserID -> Hangi Chat Server'a bağlı?
    private final ConcurrentMap<String, String> userServerRegistry = new ConcurrentHashMap<>();

    public void registerSession(String userId, String serverId) {
        userServerRegistry.put(userId, serverId);
        IO.println("Kullanıcı " + userId + " bağlandı -> Sunucu: " + serverId);
    }

    public String routeMessage(String recipientId) {
        String server = userServerRegistry.get(recipientId);
        if (server != null) {
            return "FORWARD_WEBSOCKET -> " + server;
        }
        return "DISPATCH_APNS_OR_FCM_PUSH (Kullanıcı Çevrimdışı!)";
    }
}

void main() {
    var router = new ChatSessionRouter();
    router.registerSession("user_ali", "chat-gateway-pod-1");

    IO.println("Mesaj Yönlendirme (Ali):  " + router.routeMessage("user_ali"));
    IO.println("Mesaj Yönlendirme (Ayşe): " + router.routeMessage("user_ayse"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 100.000 kişilik bir grupta biri mesaj attığında 'Mesajı gruptaki 100.000 kişinin WebSocket soketine tek tek yazarım' demeyin! Bu 'Fan-out' darbesi sunucuyu kilitler. Büyük gruplarda mesajlar merkezi bir kuyruğa yazılır ve istemciler kendi pencereleri açıldığında mesajları çeker.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Pushing group messages immediately to all 100,000 members of a mega-channel concurrently over WebSockets introduces a fatal fan-out write storm that saturates network buffers. Large channels require pull-based pagination or aggregated pub/sub partitions.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- WebSocket sürekli açık kalan iki yönlü TCP kanalıyla anlık sohbetin omurgasıdır.
- Kullanıcı durumu (Presence) periyodik Heartbeat ve Redis TTL anahtarlarıyla yönetilir.
- Sohbet geçmişi için Cassandra/ScyllaDB LSM-Tree mimarisiyle en yüksek yazma hızını sunar.

🎯 Key Takeaways for the Interview

- Persistent WebSockets eliminate handshake overhead for sub-50ms message delivery.
- Ephemeral user presence states are managed via heartbeat loops and auto-expiring Redis keys.
- Cassandra's sequential append-only LSM-Tree storage is the gold standard for chat persistence.

44

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Sosyal Medya Haber Akışı (News Feed / Twitter): Fan-Out-on-Write vs Fan-Out-on-Read

Designing a News Feed (Twitter / Instagram): Fan-Out Architectures

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. FAN-OUT-ON-WRITE (Push Modeli - Normal Kullanıcılar İçin) ]
Ali Tweet Atar ──► Takipçileri Bul (150 kişi) ──► Her birinin Feed Önbelleğine Yaz!
(Okuma anında akış hazır bekler: O(1) okuma hızı!)

[ 2. FAN-OUT-ON-READ (Pull Modeli - Ünlüler / Celebrities İçin) ]
Cristiano Ronaldo Tweet Atar (600 Milyon Takipçi!)
600 Milyon önbelleğe yazmaya kalkarsanız sunucular çöker (Celebrity Problem)!
Çözüm: Ronaldo'nun tweet'i sadece kendi profiline yazılır. Takipçi akışını açtığında
normal akış ile Ronaldo'nun tweet'i dinamik olarak harmanlanır (Merge)!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Haber akışı oluşturmanın iki yolu vardır: 1) Fan-Out-on-Write (Gazete Dağıtıcısı Modeli): Mahallenin muhtarı bir duyuru yaptığında, çırak gidip mahalledeki 50 evin kapısının altından duyuruyu atar. Sabah uyanan herkes kapının önündeki duyuruyu saniyesinde okur (Okuma anında sıfır bekleme!). 2) Fan-Out-on-Read (Ünlü Problemi): Eğer Cristiano Ronaldo veya Taylor Swift duyuru yaparsa ne olur? 600 milyon takipçinin evine tek tek kağıt dağıtmaya kalkarsanız posta teşkilatı iflas eder! Ünlüler duyuruyu meydana asar; siz sabah sokağa çıktığınızda kendi mektuplarınızla meydandaki büyük afişi birleştirip okursunuz!

🇬🇧 ENGLISH (ELI5)

Two contrasting architectures for social feeds: 1) Fan-Out-on-Write (Push Model): When an ordinary user with 200 friends posts a photo, background workers push a reference into each of those 200 friend's private in-memory feed caches. When a friend opens their phone, their timeline is already pre-computed (O(1) read latency!). 2) Fan-Out-on-Read (Pull Model / The Celebrity Problem): When Cristiano Ronaldo or Taylor Swift posts to 600 million followers, pushing to 600M individual caches burns terabytes of RAM and locks queues for hours! Instead, store the celebrity tweet once on their wall; when a fan opens their feed, the app dynamically merges their friends' cached feed with Ronaldo's latest tweet!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG News Feed Mimarisi (Hibrit Model): 1) Fan-Out-on-Write (Push): Normal kullanıcılar (<25.000 takipçi) için geçerlidir. Kullanıcı tweet attığında bir worker takipçilerin Redis ZSET akışlarına tweet ID'sini ekler. Okuma $O(1)$'dir. 2) Fan-Out-on-Read (Pull): Ünlüler (Celebrity / VIP kullanıcılar) için geçerlidir. Tweet takipçilerin önbelleğine yazılmaz; sadece ünlünün kendi kütüğüne yazılır. İstemci akışını yenilediğinde, önbelleğindeki akış ile takip ettiği ünlülerin en son tweetleri birleştirilerek (K-Way Merge) kullanıcıya sunulur. Bu hibrit mimari hem Twitter'ın hem de Instagram'ın omurgasıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Hybrid News Feed Architecture: 1) Fan-Out-on-Write (Push): Applied to standard users (<25,000 followers). Asynchronous workers fan out the post ID into follower timeline caches (Redis ZSET keyed by timestamp). Delivers $O(1)$ instantaneous feed generation. 2) Fan-Out-on-Read (Pull): Applied to high-degree celebrities (e.g. 50M+ followers). Writing to 50M Redis sets exhausts cluster bandwidth. The celebrity tweet is stored only in their user timeline; when an active follower requests their feed, the system executes an in-memory K-way merge between the user's pre-computed inbox and their followed celebrities' latest posts. This hybrid approach represents the production standard at Twitter and Instagram.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Hibrit Fan-Out Karar Motoru (Celebrity Problem Çözümü)
public class HybridFeedFanOut {
    private static final int CELEBRITY_THRESHOLD = 50_000;

    public record User(String id, String username, int followerCount) {
        public boolean isCelebrity() { return followerCount >= CELEBRITY_THRESHOLD; }
    }

    public String determineFanOutStrategy(User author) {
        if (author.isCelebrity()) {
            return "PULL_STRATEGY: Takipçilere yazma! Sadece ünlünün profiline kaydet. (Followers: " +
author.followerCount() + ")";
        } else {
            return "PUSH_STRATEGY: " + author.followerCount() + " takipçinin Redis Feed önbelleğine
asenkron fırlat!";
        }
    }
}

void main() {
    var engine = new HybridFeedFanOut();
    var normalUser = new HybridFeedFanOut.User("u1", "ali_bozkurt", 240);
    var celebrity = new HybridFeedFanOut.User("u2", "elon_musk", 180_000_000);

    IO.println("Ali için Strateji:  " + engine.determineFanOutStrategy(normalUser));
    IO.println("Elon için Strateji: " + engine.determineFanOutStrategy(celebrity));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Her tweet atıldığında takipçilerin feed'ini yeniden hesaplarım (Fan-out on Read)' derseniz sistem 100M kullanıcıda çöker; 'Herkes için Fan-out on Write yaparım' derseniz Elon Musk tweet attığında sistem çöker! Çözüm: Kesinlikle HİBRİT modeldir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Choosing purely Fan-out on Read causes database exhaustion during timeline loads; choosing purely Fan-out on Write destroys queues when celebrities post. Propose the HYBRID model: push for ordinary users, pull for celebrities.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Fan-out on Write okuma hızını O(1) seviyesine indirir ancak ünlülerde patlar.
- Celebrity Problem, takipçi sayısı yüksek hesapların akışını okuma anında dinamik harmanlayarak çözülür.
- Haber akışları Redis Sorted Sets (ZSET) veri yapısında zaman damgası skoruyla tutulur.

🎯 Key Takeaways for the Interview

- Fan-out on Write provides instantaneous O(1) timeline reads but stumbles on celebrity fan-out.
- The Celebrity Problem is resolved by dynamically blending hot celebrity timelines on read.
- User feed inboxes are maintained inside Redis Sorted Sets keyed by epoch millisecond scores.

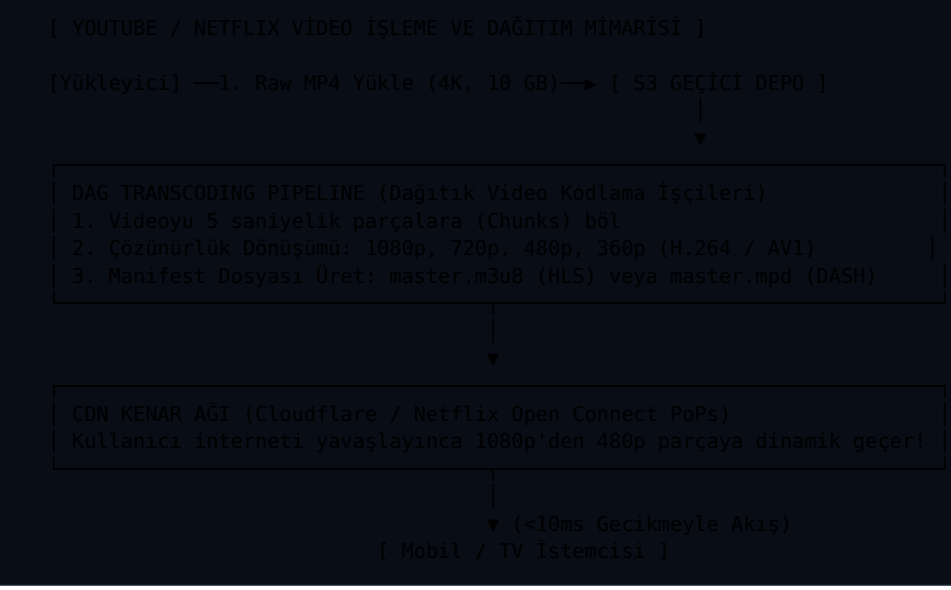
45

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Video Akış Platformu (YouTube / Netflix) Tasarımı: Transcoding, Chunking ve HLS/DASH

Designing Video Streaming (YouTube / Netflix): HLS/DASH & Transcoding

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

YouTube'a 10 GB'lık dev bir film yüklediğinizde, bir kullanıcının o filmi tek parça halinde indirmesini beklemezsiniz; telefonun hafızası dolar ve interneti donardı! YouTube videoyu bir ekmek gibi 5 saniyelik dilimlere (Chunks) böler. Üstelik her dilimin 4K, 1080p, 720p ve 360p olmak üzere 4 farklı kalitede kopyasını çıkarır (Transcoding). Siz metroya binip internetiniz zayıfladığında video durmaz; oynatıcı anında 1080p dilim almayı bırakıp 360p dilimi almaya başlar (Adaptif Akış - HLS)! Metrodan çıkınca tekrar kristal netliğinde 4K dilimlere geri döner!

🇬🇧 ENGLISH (ELI5)

When uploading a 10GB 4K raw film to YouTube, you don't stream that gargantuan file in a single monolithic blob; mobile devices would freeze and crash! YouTube slices the video like a loaf of bread into hundreds of 5-second crumbs called 'Chunks'. Furthermore, an automated transcoding farm converts each crumb into 1080p, 720p, 480p, and 360p bitrates. When you step into an elevator and mobile signal weakens, playback doesn't halt; the player dynamically switches from fetching 1080p chunks to 360p chunks (Adaptive Bitrate Streaming via HLS/DASH) seamlessly!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Video Platformu Mimarisi: 1) Video Yükleme ve DAG Pipeline: Video S3'e yüklendikten sonra bir Kafka olayı dağıtık kodlama motorunu (Transcoding Cluster - FFmpeg) tetikler. DAG (Directed Acyclic Graph) yapısıyla video ses, altyazı ve video parçalarına ayrıştırılır. 2) Adaptif Bit Hızı Akışı (Adaptive Bitrate Streaming - ABR): Endüstri standartları Apple HLS (`.m3u8` manifest ve `.ts` segmentleri) ve MPEG-DASH'tir (`.mpd`). Oynatıcı istemci, ağ bant genişliğini milisaniye seviyesinde ölçer ve bir sonraki 5 saniyelik parçayı en uygun çözünürlükten ister. 3) CDN Dağıtımı: Video segmentleri değiştirilemez (Immutable) statik dosyalardır; CDN PoP noktalarında %99 oranında önbelleğe alınır. Netflix'in Open Connect mimarisi, popüler filmleri gece saatlerinde doğrudan yerel telekom ISP'lerinin veri merkezlerindeki sunuculara kopyalar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of Scale Video Platforms (YouTube / Netflix): 1) Video Ingestion & DAG Transcoding: Uploaded raw video drops into an ingestion S3 bucket, spawning a Directed Acyclic Graph (DAG) workflow across GPU compute worker pools. Tasks parallelize video chunking, audio extraction, watermarking, and multi-codec encoding (H.264, H.265, AV1). 2) Adaptive Bitrate Streaming (ABR): Packaged via HLS (`.m3u8` playlists and `.ts` chunk files) or MPEG-DASH (`.mpd`). Client players dynamically monitor throughput and buffer depths, swapping bitrates up or down mid-stream without frame drops. 3) Edge CDN Distribution: Chunks are immutable static files cached at edge PoPs with 98%+ cache hit rates. Netflix Open Connect appliances deploy custom storage servers directly inside thousands of residential ISP data centers globally.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: HLS Manifest ve Adaptif Segment Çözümleyici Modeli
public class HlsManifestEngine {
    public record VideoSegment(int sequenceNumber, String resolution, String chunkUrl, int
durationSecs) {}

    public static VideoSegment selectAdaptiveChunk(int currentBandwidthKbps, int chunkIndex) {
        // İstemcinin anlık internet hızına göre en iyi çözünürlüğü seç:
        String quality = switch (currentBandwidthKbps) {
            case int b when b >= 8000 -> "4K_2160p";
            case int b when b >= 3000 -> "FHD_1080p";
            case int b when b >= 1200 -> "HD_720p";
            default                   -> "SD_360p";
        };
        return new VideoSegment(chunkIndex, quality, "/video/chunk_" + chunkIndex + "_" + quality +
".ts", 5);
    }
}

void main() {
    // Metroya girildi: Hız 9000 Kbps'den 800 Kbps'e düştü!
    var highSpeedChunk = HlsManifestEngine.selectAdaptiveChunk(9500, 42);
    var lowSpeedChunk = HlsManifestEngine.selectAdaptiveChunk(800, 43);

    IO.println("Evdeki Parça (Wi-Fi):  " + highSpeedChunk.resolution() + " -> " +
highSpeedChunk.chunkUrl());
    IO.println("Metrodaki Parça (Edge):" + lowSpeedChunk.resolution() + " -> " +
lowSpeedChunk.chunkUrl());
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Videoları tek bir sunucuda FFmpeg ile dönüştürürüm' demeyin! 2 saatlik bir 4K video tek makinede saatlerce sürer. Çözüm: Video 5 dakikalık parçalara bölünür, 50 ayrı makinede paralel kodlanır ve sonunda manifest dosyasında birleştirilir (MapReduce tarzı paralel video işleme).

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Proposing a single server running FFmpeg to transcode 4K videos is a fatal architectural flaw. Slice raw video into discrete chunks, fan out encoding across distributed GPU worker fleets in parallel, and stitch the master manifest together.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- HLS ve DASH adaptif protokolleri videoyu 5 saniyelik parçalar halinde sunar.
- İstemci oynatıcı ağ hızına göre sonraki parçanın çözünürlüğünü dinamik olarak seçer.
- Video parçaları değiştirilemez statik dosyalar olduğu için CDN'lerde %98 oranında önbelleğe alınır.

🎯 Key Takeaways for the Interview

- HLS and DASH slice video into 5-second chunks for adaptive bitrate switching.
- Client players probe network throughput continuously to fetch optimal chunk resolutions.
- Immutable video segments enable near-perfect CDN edge cache hit ratios.

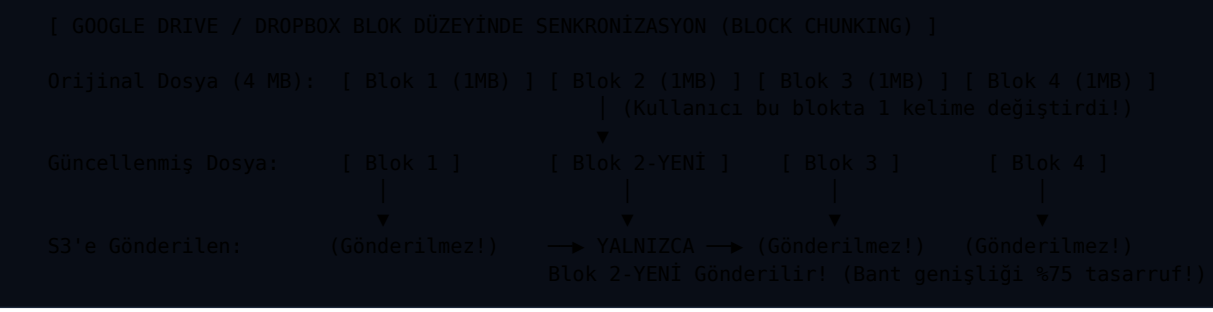
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

46

Dosya Depolama ve Senkronizasyon (Google Drive / Dropbox): Chunking ve Sync

Designing Cloud Storage (Google Drive / Dropbox): Chunking & Sync

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

100 sayfalık kalın bir kitap üzerinde çalıştığınızı düşünün. 52. sayfada tek bir harf hatasını düzelttiğinizde koca kitabı matbaaya gönderip 100 sayfayı baştan bastırmazsınız; sadece 52. sayfayı değiştirirsiniz! Dropbox ve Google Drive da böyle çalışır: 2 GB'lık bir video dosyasını 4 MB'lık yüzlerce bloğa (Chunk) böler. Dosyada tek bir satır değiştiğinde 2 GB'ı baştan yüklemez; sadece değişen o 4 MB'lık küçük bloğu bulur ve buluta fırlatır (Delta Sync). İnternet kotanız ve zamanınız kurtulur!

🇬🇧 ENGLISH (ELI5)

Editing a single typo on page 52 of a 500-page book doesn't require incinerating the entire print run and re-printing all 500 pages; you replace page 52! Cloud storage providers like Dropbox and Google Drive partition large files into discrete 4MB chunks. When you modify one sentence in a 1GB file, the desktop sync engine calculates cryptographic hashes across all chunks, identifies the single mutated block, and uploads only that 4MB delta! Bandwidth consumption drops by 99%!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Dosya Depolama Mimarisi: 1) Blok Düzeyinde Bölme (Chunking): Dosyalar 4 MB'lık bloklara bölünür. Her bloğun SHA-256 hash'i alınır. 2) Küresel Tekilleştirme (Cross-User Deduplication): Eğer Ahmet ve Mehmet aynı 1 GB'lık Ubuntu ISO dosyasını yüklüyorsa, sistem blok hash'lerine bakar. Bloklar zaten S3'te varsa İKİNCİ KEZ YÜKLENMEZ; sadece veritabanında referans pointer'ı açılır. Milyonlarca dolar depolama ve ağ tasarrufu sağlanır. 3) Senkronizasyon Çakışması (Sync Conflict): İki kullanıcı çevrimdışı aynı dosyayı düzenleyip aynı anda bağlanırsa ne olur? Git tarzı versiyon çakışması üretilir: 'Dosya (Ali's conflicted copy)' şeklinde yeni bir dosya dallandırılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Cloud Storage & Sync Architecture: 1) Block-Level Chunking: Files split into fixed (4MB) or variable-length (Rabin fingerprinting) chunks. Each block generates a unique cryptographic SHA-256 hash. 2) Cross-User Deduplication: If 10,000 corporate employees save the same 500MB PDF presentation, block hashing detects matching fingerprints in S3; the storage cluster skips the upload entirely and increments metadata reference pointers, saving petabytes of storage. 3) Delta Sync Engine: Local sync daemons monitor file system watch events, compute modified block diffs, and upload strictly modified chunks. 4) Conflict Resolution: Concurrent offline mutations are resolved by branching diverging file trees ('Project Plan (Alice's conflicted copy)').

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Blok Düzeyinde SHA-256 Hash Hesaplama ve Tekilleştirme Simülasyonu
import java.security.MessageDigest;
import java.util.*;

public class BlockDeduplicationEngine {
    private final Set<String> cloudBlockStorage = new HashSet<>();

    public String computeBlockHash(byte[] blockBytes) throws Exception {
        var md = MessageDigest.getInstance("SHA-256");
        byte[] digest = md.digest(blockBytes);
        return HexFormat.of().formatHex(digest).substring(0, 12); // Kısaltılmış hash
    }

    public boolean uploadBlockIfMissing(String blockHash) {
        // Blok zaten S3'te varsa YÜKLEME! (Deduplication)
        return cloudBlockStorage.add(blockHash);
    }
}

void main() throws Exception {
    var engine = new BlockDeduplicationEngine();
    byte[] chunk1 = "Ders Notu Parça 1".getBytes();
    String hash1 = engine.computeBlockHash(chunk1);

    IO.println("Blok 1 Yüklendi mi (İlk Kez)? " + engine.uploadBlockIfMissing(hash1));
    IO.println("Blok 1 Yüklendi mi (Tekrar)?  " + engine.uploadBlockIfMissing(hash1) + "
(Deduplication Devrede!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Dosyaları tek parça halinde yüklerim' demeyin! 5 GB'lık bir dosya yüklenirken %99'da internet koparsa tüm dosya çöpe gider. Blok tabanlı yüklemede (Chunking) sadece kopan 4 MB'lık blok yeniden denenir (Resumable Upload).

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch uploading monolithic multi-gigabyte files in single HTTP streams. If connectivity drops at 99%, the entire upload fails. Chunking guarantees resumable uploads where only the interrupted 4MB block re-transmits.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Dosyaları 4MB'lık bloklara bölmek yalnızca değişen kısımların yüklenmesini (Delta Sync) sağlar.
- Blok hash'leri sayesinde aynı dosya bulutta tek bir kez saklanır (Cross-User Deduplication).
- Kopan yüklemeler baştan başlamaz; yalnızca eksik kalan bloklar kaldığı yerden devam eder.

🎯 Key Takeaways for the Interview

- Chunking files into 4MB blocks enables delta sync, uploading only modified bytes.
- Cross-user deduplication detects identical block hashes, eliminating petabytes of duplicate S3 data.
- Chunked uploads enable robust resumable uploads after network disconnects.

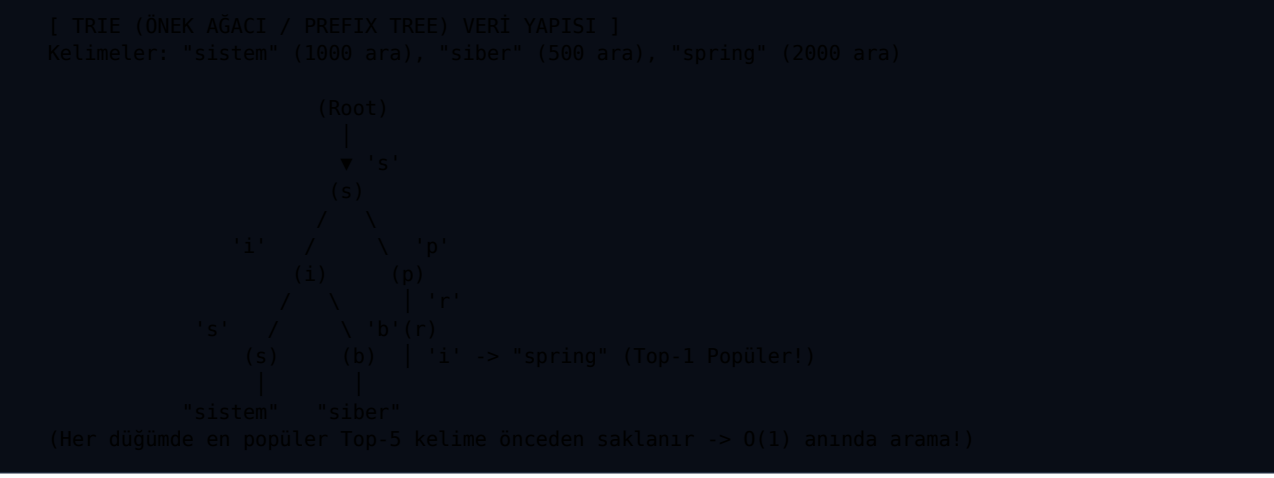
47

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

Arama Otomatik Tamamlama (Typeahead / Google Suggestion): Trie ve Redis ZSET

Designing Search Typeahead (Google Suggestion): Trie & Redis ZSET

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Google'ın arama kutusuna 'sis' yazdığınız anda kutunun altında 'sistem tasarımı', 'siber güvenlik' gibi önerilerin şak diye belirmesini düşünün. Dünyada 1 milyar kelime varken Google her harf bastığınızda tüm veritabanını tarayamaz! Bunun yerine akıllı bir soy ağacı (Trie - Önek Ağacı) kurar. 's' harfine basınca 's' ile başlayan tüm dalları, 'i' harfine basınca 'si' ile başlayan dalları görür. Üstelik her dalın tepesinde en çok aranan ilk 5 kelime altın tepside hazır tutulur. Siz tuşa bastığınız anda 2 milisaniyede liste önünüze dökülür!

🇬🇧 ENGLISH (ELI5)

The instant you type 'sys' into Google, a dropdown flashes: 'system design', 'system requirements'. With a corpus of billions of search terms, Google cannot run a database SQL scan on every keystroke! Instead, it navigates an in-memory Prefix Tree (Trie). Typing 's' lands on the 's' branch; typing 'y' traverses to 'sy'. Crucially, every tree node pre-caches the Top-5 most popular global search phrases right on its branch. The moment your finger hits the key, suggestions render in 2 milliseconds!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Typeahead Mimarisi: 1) Trie Veri Yapısı: Kökten yapraklara karakter bazlı dallanan ağaçtır. Arama süresi kelime uzunluğu k kadardır ($O(k)$); kelime sayısından (N) tamamen bağımsızdır. 2) Top-K Önbellekleme: Her Trie düğümünde en sık aranan ilk 5-10 kelime ve frekansı saklanır. Kullanıcı 'sis' yazdığında ağacın derinliklerine inmeden o düğümdeki hazır listeyi döner ($O(1)$). 3) Çevrimdışı Analiz Hattı (Offline Aggregation Pipeline): Trie çalışma anında güncellenmez (yavaşlatmamak için). Arama logları Kafka -> Flink/Spark ile toplanır; gece batch işinde yeni frekanslar hesaplanır ve güncel Trie bellek kümesine (Redis veya Custom C++ / Java servisi) yüklenir. 4) İstemci Tarafı Optimizasyon: Tarayıcı tarafında 200ms `debounce` (kullanıcı yazmayı bırakana kadar bekleme) ve yerel önbellek uygulanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Search Typeahead Architecture: 1) The Trie Data Structure: Character-level prefix tree providing $O(L)$ traversal time (L = query length), completely independent of total vocabulary size N. 2) Node-Level Top-K Caching: To eliminate subtree traversals during runtime lookups, each Trie node caches the Top-5 most frequent completions directly on its node struct. Querying prefix 'sys' reads the precomputed Top-5 array in $O(1)$ time. 3) Offline Aggregation Pipeline: Live search keystrokes stream into Kafka and map-reduce via Apache Flink to aggregate weekly frequency scores. Updated Trie snapshots rebuild offline and hot-swap into distributed memory clusters. 4) Client-Side Optimizations: Frontends implement a 200ms debounce buffer and local LRU cache to slash upstream keystroke queries by 60%.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Top-K Destekli Trie (Önek Ağacı) Implementasyonu
import java.util.*;

public class TypeaheadTrie {
    private static class TrieNode {
        Map<Character, TrieNode> children = new HashMap<>();
        List<String> topSuggestions = new ArrayList<>(); // Önceden hesaplanmış Top-K
    }

    private final TrieNode root = new TrieNode();

    public void insert(String phrase) {
        TrieNode curr = root;
        for (char c : phrase.toLowerCase().toCharArray()) {
            curr = curr.children.computeIfAbsent(c, k -> new TrieNode());
            if (!curr.topSuggestions.contains(phrase)) {
                if (curr.topSuggestions.size() < 3) {
                    curr.topSuggestions.add(phrase);
                }
            }
        }
    }

    public List<String> getSuggestions(String prefix) {
        TrieNode curr = root;
        for (char c : prefix.toLowerCase().toCharArray()) {
            curr = curr.children.get(c);
            if (curr == null) return List.of();
        }
        return curr.topSuggestions;
    }
}

void main() {
    var trie = new TypeaheadTrie();
    trie.insert("sistem tasarimi");
    trie.insert("siber guvenlik");
    trie.insert("sistem gereksinimleri");
    trie.insert("spring boot action");

    IO.println("Öneri ('sis'): " + trie.getSuggestions("sis"));
    IO.println("Öneri ('sp'):  " + trie.getSuggestions("sp"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta Trie ağacını her kullanıcı arama yaptıkça canlı (real-time) olarak güncelleyeceğinizi söylemeyin! Milyonlarca eşzamanlı yazma işlemi Trie ağacında kilitlenme (Lock Contention) yaratır ve belleği tüketir. Güncellemeler daima çevrimdışı batch pipeline ile yapılmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose updating Trie frequencies synchronously on live search queries. Concurrent lock contention on root nodes will crater latency. All frequency score recalculations must occur asynchronously via offline batch pipelines.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Trie (Önek Ağacı) arama süresini kelime sayısından bağımsız olarak O(kelime_uzunluğu) yapar.

• Düğüm üzerinde Top-K önbellekleme arama anında tüm ağacı tarama zorunluluğunu kaldırır.

• İstemci tarafında Debouncing (200ms) uygulamak sunucuya gelen istek yükünü %60 azaltır.

🎯 Key Takeaways for the Interview

• Tries bound search latency to O(prefix_length), completely independent of dictionary scale.

• Pre-computing Top-K suggestions at each Trie node delivers instant O(1) query returns.

• Client-side debouncing buffers rapid typing, cutting keystroke query volume by 60%+.

48

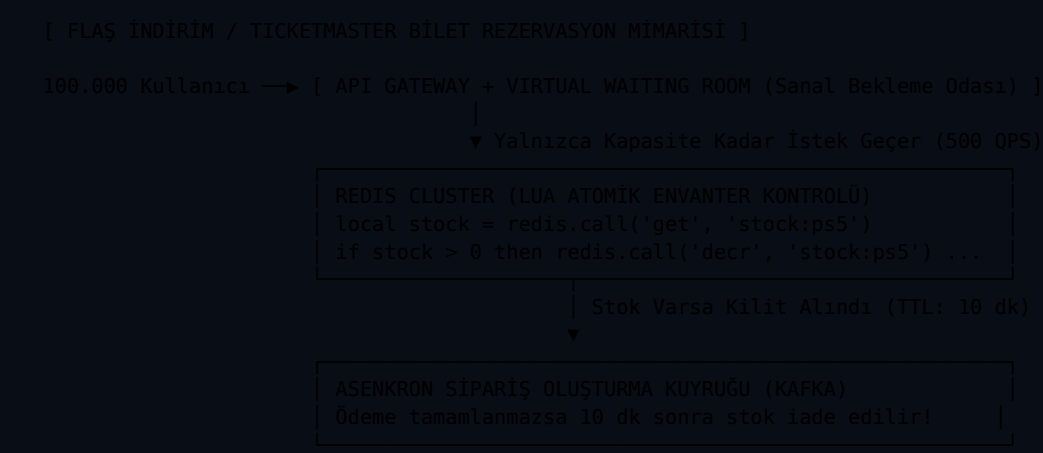
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 2 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 2

E-Ticaret Flaş İndirim ve Bilet Satış Sistemi (Flash Sale / Ticketmaster)

Designing a Flash Sale & Ticketing System (Ticketmaster / Amazon Prime Day)

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Stadyumda bir rock konseri için sadece 1.000 bilet olduğunu ve kapıda 100.000 kişinin beklediğini düşünün. Kapıyı ardına kadar açarsanız 100.000 kişi içeri dalar, gişeyi kırar ve izdihamda herkes ezilir (Veritabanı çöküşü!). Çözüm: 1) Kapının önüne döner bir bariyer (Virtual Waiting Room) koyarsınız. İçeriye sadece sırayla 1.000 kişi alınır. 2) Gişedeki biletçi sepete eklenen bilete 10 dakikalık bir geri sayım sayacı yapıştırır (Geçici Kilit). 10 dakika içinde parayı ödemezseniz bilet elinizden alınır ve sıradaki kişiye verilir!

🇬🇧 ENGLISH (ELI5)

Picture a stadium rock concert with only 1,000 tickets, but a mob of 100,000 fans waiting outside. If you throw open the gates, 100,000 people trample the box office and smash the counters (Database meltdown!). The solution: 1) Set up a velvet rope turnstile outside (Virtual Waiting Room). Fans queue in order; only batches of 500 enter at a time. 2) When a fan grabs a seat, the system slaps a 10-minute countdown timer on the ticket (Atomic Reservation with TTL). If they don't enter credit card info within 10 minutes, the hold expires and the seat returns to the pool!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Flaş İndirim Mimarisi: 1) Sanal Bekleme Odası (Virtual Waiting Room): Cloudflare Waiting Room veya Kafka kuyrukları ile kullanıcılar adil bir sıra numarası (FIFO) ile bekletilir; veritabanı kapasitesi kadar kullanıcı içeri alınır. 2) Bellek İçi Atomik Envanter (In-Memory Atomic Inventory): Veritabanına `UPDATE products SET stock = stock - 1 WHERE id = 1 AND stock > 0` yapmak satır kilidi (Row Lock) yüzünden DB'yi kilitler. Stok Redis'te tutulur ve Lua Script ile atomik olarak düşürülür (`DECR`). Sıfırın altına düşmesine asla izin verilmez. 3) Geçici Rezervasyon ve Dağıtık TTL: Kullanıcı bilet seçtiğinde bilet 10 dakika rezerve edilir (`SET seat:42:lock user_101 EX 600`). Ödeme başarılı olursa DB'ye kesin satış yazılır. Ödeme başarısız olursa veya 10 dakika dolarsa TTL biter ve bilet havuza geri döner.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Flash Sale & Ticketing Architecture: 1) Virtual Waiting Room (Fair Queuing): Deploys an edge ingress queue (Cloudflare Waiting Room) metering traffic based on backend throughput capacity, preventing database saturation. 2) Atomic In-Memory Stock Depletion: Executing relational database updates (`UPDATE stock SET qty = qty - 1 WHERE qty > 0`) creates severe row-level lock contention. Inventory is decremented in-memory inside Redis via atomic Lua scripts in microsecond speeds. 3) Two-Phase Inventory Hold (Reservation TTL): Stock is not permanently sold until payment commits. Grabbing an item creates a temporary hold with a 10-minute expiration (`SET item:hold:user EX 600`). If payment confirmation completes, a Kafka event finalizes the database commit; if the timer expires, the hold releases atomically back to available inventory.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Flaş İndirim Atomik Stok ve Geçici Rezervasyon Motoru
import java.util.concurrent.*;

public class FlashSaleEngine {
    private int availableStock;
    private final ConcurrentMap<String, Long> activeReservations = new ConcurrentHashMap<>();

    public FlashSaleEngine(int initialStock) {
        this.availableStock = initialStock;
    }

    public synchronized boolean tryReserveItem(String userId, long holdDurationMillis) {
        if (availableStock <= 0) {
            IO.println("❌ STOK TÜKENDİ! Kullanıcı: " + userId);
            return false; // Stok bitti
        }

        availableStock--;
        activeReservations.put(userId, System.currentTimeMillis() + holdDurationMillis);
        IO.println("✔ Stok Rezerve Edildi: " + userId + " (Kalan Stok: " + availableStock + ")");
        return true;
    }

    public synchronized void cancelExpiredReservation(String userId) {
        Long expiration = activeReservations.remove(userId);
        if (expiration != null) {
            availableStock++; // Stok havuza iade edildi
            IO.println("⏱ Süre Doldu! Stok İade Edildi: " + userId + " (Güncel Stok: " +
availableStock + ")");
        }
    }
}

void main() {
    var sale = new FlashSaleEngine(2); // Toplam 2 adet PS5
    sale.tryReserveItem("user_ali", 600_000);
    sale.tryReserveItem("user_veli", 600_000);
    sale.tryReserveItem("user_can", 600_000); // Stok kalmadı!

    sale.cancelExpiredReservation("user_ali"); // Ali ödemedi -> Stok iade!
    sale.tryReserveItem("user_can", 600_000);  // Can artık alabildi!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Stoku doğrudan MySQL'de kilit açarak (SELECT FOR UPDATE) yönetirim' demeyin! Saniyede 100.000 kişinin aynı satırı kilitlemeye çalışması veritabanında Connection Pool tükenmesine ve sistemin çökmesine neden olur. Stok kontrolü daima Redis'te atomik yapılmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch relational `SELECT ... FOR UPDATE` row locks for flash sales. 50,000 concurrent threads vying for the exact same database row lock will crash the connection pool in seconds. Manage hot stock counts in Redis memory via atomic Lua scripts.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sanal Bekleme Odası (Virtual Waiting Room) veritabanını aşırı yükten korur.

• Stok düşürme işlemi veritabanı yerine Redis üzerinde atomik Lua scripti ile yapılır.

• Geçici rezervasyon ve TTL sayesinde ödeme yapmayan kullanıcıların ürünleri havuza döner.

🎯 Key Takeaways for the Interview

• Virtual waiting rooms throttle user concurrency to match backend database processing limits.

• Atomic in-memory decrements in Redis eliminate relational database row-locking contention.

• Reservation TTL windows release unclaimed inventory automatically back to waiting fans.

BÖLÜM 9 / MODULE 9

İleri Seviye MAANG Tasarım Problemleri

Advanced MAANG Distributed System Designs

Uber Yolculuk Eşleştirme (H3 Altıgenleri), Dağıtık Metrik Sistemi (TSDB), DynamoDB Klonu, Dağıtık Kuyruk (Zero-Copy), Stripe Ödeme Sistemi ve Canlı Skor Tablosu (SkipList).

Uber Ride-Hailing (H3 Hexagons), Time-Series Monitoring (Prometheus TSDB), Distributed Key-Value Store (DynamoDB), Kafka Clone, Stripe Payments, and Gaming Leaderboard (SkipLists).

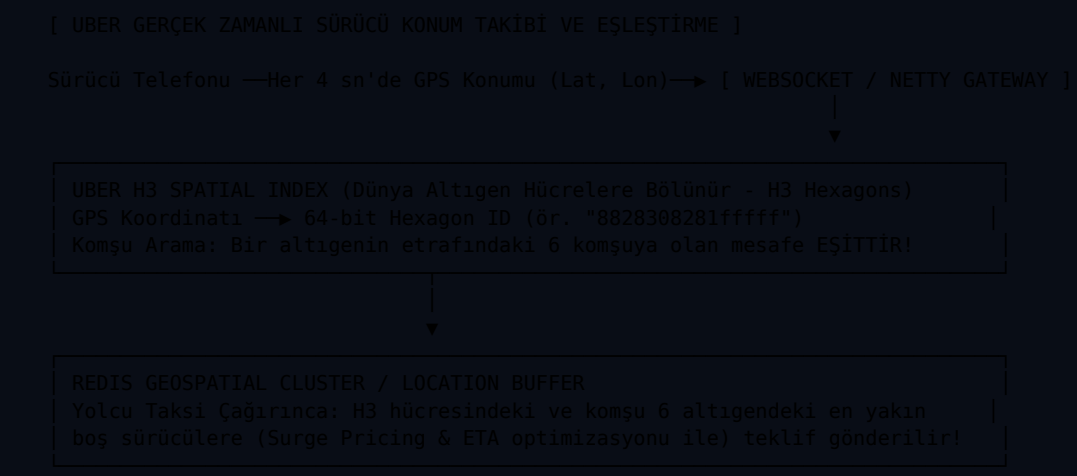
49

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Gerçek Zamanlı Yolculuk Eşleştirme (Uber / Lyft): Uber H3 Altıgenleri ve Konum Takibi

Designing Ride-Hailing (Uber / Lyft): Uber H3 Hexagons & Real-Time Dispatch

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Kare bir ızgarada merkezdeki bir kutunun yan komşusu ile çapraz komşusunun mesafesi aynı değildir (çapraz komşu %41 daha uzaktır!). Bu yüzden Uber dünyayı karelere değil, arıların peteği gibi altıgenlere (H3 Hexagonal Grid) bölmüştür! Bir altıgenin etrafındaki 6 komşusunun da merkeze olan uzaklığı BİREBİR AYNIDIR. 1 milyon taksi şoförü her 4 saniyede bir 'Ben şu altıgendeyim' diye sinyal gönderir. Siz 'Taksi çağır' butonuna bastığınızda, sistem sizin bulunduğunuz altıgene ve etrafındaki 6 komşu peteğe bakar; 1 saniyede en yakın 3 taksiciyi ekranınıza düşürür!

🇬🇧 ENGLISH (ELI5)

On a square grid, the distance to an edge neighbor differs from a diagonal neighbor (diagonals are 41% further away!). To eliminate this mathematical distortion, Uber divided the entire planet into a honeycomb of hexagons (Uber H3 Spatial Index). Every neighbor of a hexagon shares the EXACT same distance to its center! 1 million drivers broadcast their GPS location every 4 seconds. When a rider taps 'Request Ride', the dispatch engine evaluates drivers in the rider's active hexagon and its 6 adjacent neighbor cells, matching the optimal driver in under 1 second!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Uber Mimarisi: 1) Konum Güncelleme Hacmi (Location Ingestion): 1M aktif sürücü x her 4 saniyede 1 GPS pingi = 250,000 QPS yazma yükü! Bu yükü doğrudan ilişkisel veritabanına yazmak sistemi eritir. Sürücü konumları hafif WebSocket veya gRPC bağlantılarıyla alınır ve bellek içi Redis Geospatial (`GEOADD`) veya RAM tabanlı H3 spatial index'te tutulur. 2) Neden Uber H3 (Altıgen Mimarisi)?: H3, dünyayı hiyerarşik 16 çözünürlük seviyesinde altıgenlere bölen açık kaynaklı bir kütüphanedir. Altıgenlerin tüm komşularına olan mesafeleri sabittir; bu sayede dinamik fiyatlandırma (Surge Pricing) ve sürücü arama algoritmaları çok daha homojendir. 3) Eşleştirme Motoru (Dispatch Engine): Yolcu istek attığında, H3 k-ring (örneğin 1 km yarıçapındaki altıgenler) taranır. Aday sürücüler arasından ETA (Tahmini varış süresi), sürücü puanı ve rota uygunluğuna göre en iyi eşleşme yapılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Uber Scale Spatial Architecture: 1) Ingestion Scale: 1M active drivers sending GPS pings every 4 seconds generates 250,000 writes/sec! Relational engines collapse under this load. Ingestion routes over persistent WebSockets/gRPC into distributed in-memory Redis clusters or custom H3 memory rings. 2) Why Uber H3 (Hexagonal Topology)?: Unlike square grids where diagonals are √2 times longer, hexagons exhibit uniform distance to all 6 adjacent neighbors. H3 provides 16 discrete hierarchical resolutions (Resolution 8 spans 450m, perfect for urban dispatch). Surge pricing boundaries map cleanly to H3 cells. 3) Real-Time Matchmaker: When a rider requests a pickup, the dispatcher executes an `H3.kRing(originHex, radius=2)` query. Candidate drivers are ranked via real-time routing engines (taking one-way streets, traffic congestion, and driver acceptance rates into account) before dispatching the offer.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Simüle Edilmiş Uber H3 Altıgen Eşleştirme Modeli
import java.util.*;

public class UberDispatchEngine {
    // Sürücü Modeli
    public record Driver(String driverId, String h3HexId, boolean isAvailable) {}

    private final Map<String, List<Driver>> hexagonalGrid = new HashMap<>();

    public void updateDriverLocation(Driver driver) {
        hexagonalGrid.computeIfAbsent(driver.h3HexId(), k -> new ArrayList<>()).add(driver);
    }

    // Yolcunun altıgen hücresindeki ve komşu hücrelerdeki müsait sürücüleri bul
    public List<Driver> findNearbyDrivers(String riderHexId, List<String> neighborHexIds) {
        List<Driver> candidates = new ArrayList<>();

        // 1. Yolcunun kendi hücresine bak
        for (Driver d : hexagonalGrid.getOrDefault(riderHexId, List.of())) {
            if (d.isAvailable()) candidates.add(d);
        }
        // 2. Komşu 6 altıgene bak (H3 k-Ring)
        for (String neighbor : neighborHexIds) {
            for (Driver d : hexagonalGrid.getOrDefault(neighbor, List.of())) {
                if (d.isAvailable()) candidates.add(d);
            }
        }
        return candidates;
    }
}

void main() {
    var dispatch = new UberDispatchEngine();
    dispatch.updateDriverLocation(new UberDispatchEngine.Driver("drv_1", "hex_kadikoy_c1", true));
    dispatch.updateDriverLocation(new UberDispatchEngine.Driver("drv_2", "hex_kadikoy_c2", true)); //
Komşu altıgen

    var nearby = dispatch.findNearbyDrivers("hex_kadikoy_c1", List.of("hex_kadikoy_c2",
"hex_kadikoy_c3"));
    IO.println("Eşleşen Müsait Sürücü Sayısı: " + nearby.size());
    for (var d : nearby) IO.println("• Sürücü #" + d.driverId() + " (Hücre: " + d.h3HexId() + ")");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta sürücülerin saniyelik konumlarını PostgreSQL veya MySQL'e yazacağınızı ASLA söylemeyin! Saniyede 250.000 disk yazması veritabanını felç eder. Sürücünün anlık konumu uçucu (ephemeral) veridir; sadece RAM'de (Redis) tutulur. Sadece yolculuk bittiğinde tamamlanan rota veritabanına arşivlenir.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch writing raw GPS pings directly to a persistent relational database! 250,000 writes/sec will incinerate database IOPS. Current driver location is ephemeral data stored exclusively in in-memory spatial caches (Redis); only finalized trip routes commit to persistent databases.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Uber H3 altıgen ızgarası komşular arası mesafeyi eşit tutarak arama algoritmasını optimize eder.

• 250.000 QPS'lik konum akışı ilişkisel DB yerine tamamen bellek içi Redis'te tutulur.

• Yolculuk eşleştirmede H3 k-ring hücreleri taranarak en düşük ETA'ya sahip sürücü seçilir.

🎯 Key Takeaways for the Interview

• Uber's H3 hexagonal grid ensures uniform neighbor distances, eliminating diagonal distortions.

• High-frequency GPS telemetry streams into in-memory Redis clusters, bypassing disk bottlenecks.

• Dispatchers search surrounding H3 k-ring clusters to identify drivers minimizing rider ETA.

50

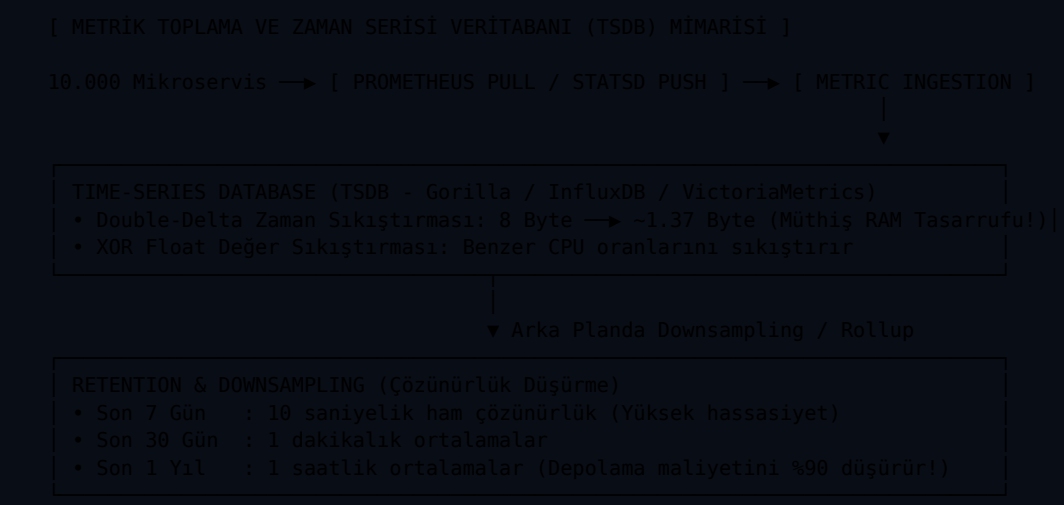
BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Metrik ve İzleme Sistemi (Datadog / Prometheus): TSDB ve Downsampling

Designing Metrics & Monitoring (Datadog / Prometheus): TSDB & Aggregations

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir hastanın yoğun bakımda saniyede 1 kez nabzını ölçtüğünüzü düşünün. Bir günde 86.400 nabız verisi çıkar. 1 ay sonra doktor hastanın 3 hafta önceki durumuna bakmak istediğinde saniye saniye 2.5 milyon nabız verisini tek tek okumak istemez; '3 hafta önce her günkü ortalama nabız neydi?' diye sorar! Dağıtık metrik sistemleri de böyledir: Sunucuların CPU kullanımını saniye saniye kaydeder (TSDB); ancak 1 ay geçince o saniyelik verileri silip yerine '1 saatlik ortalama' kaydını bırakır (Downsampling). Böylece hard diskler milyonlarca gereksiz sayıyla dolup taşmaz!

🇬🇧 ENGLISH (ELI5)

Imagine logging an intensive care patient's heart rate every single second. That generates 86,400 numbers a day. Six months later, the doctor reviewing recovery progress doesn't need to scroll through 15 million individual second-by-second data points; they only care about the daily average and peak trends! That is what a Time-Series Monitoring System (Prometheus, Datadog) does: it ingests high-resolution telemetry every 10 seconds, but as data ages past 30 days, background jobs roll up seconds into hourly averages (Downsampling), slashing storage footprint by 90%!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Metrik Sistemi Mimarisi: 1) Pull vs Push Modeli: Prometheus 'Pull' modelini kullanır (Sunucuları kendisi tarar / scrape eder; istemciye yük binmez). Datadog/CloudWatch 'Push' modelini kullanır (Agent metrikleri merkezi API'ye fırlar). 2) Zaman Serisi Veritabanı (TSDB) Sıkıştırması (Facebook Gorilla Mimarisi): Zaman serisi verisi `(Timestamp, Value)` çiftidir. Zaman damgaları 'Double-Delta Compression' ile (ör. her 10 saniyede bir gelen veri için farkın farkı sıfır olur ve tek bir 0 bitiyle temsil edilir); Float değerler ise 'XOR Floating-Point Compression' ile sıkıştırılır. 16 baytlık bir metrik noktası ortalama 1.37 bayta düşürülür (10 kat bellek sıkıştırması!). 3) Downsampling & Rollup: Veri eskidikçe çözünürlük kademeli düşürülür (Raw 10s -> 5m Rollup -> 1h Rollup).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of Scale Metrics Systems: 1) Pull vs Push: Prometheus implements a Pull architecture (periodically scrapes `/metrics` endpoints via service discovery; auto-detects dead hosts). Datadog/StatsD uses Push (agents dispatch UDP/TCP packets to gateways; ideal for ephemeral serverless lambdas). 2) TSDB Compression (Facebook's Gorilla Paper): Time-series data comprises `(timestamp, float_value)` streams. Gorilla compresses timestamps via Double-Delta encoding (D = (t[i] - t[i-1]) - (t[i-1] - t[i-2])); periodic intervals evaluate to zero, encoded in a single bit. Float values are compressed by XORing against previous values. Slashes 16-byte tuples down to 1.37 bytes per metric point in memory. 3) Downsampling & Rollups: Background compaction jobs aggregate granular metrics into downsampled buckets (10s raw for 7 days -> 1m rollup for 30 days -> 1h rollup for 1 year), bounding long-term storage growth.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Basitleştirilmiş Zaman Serisi Metrik Veri Noktası ve Rollup Modeli
import java.util.*;

public class MetricsRollupEngine {
    public record MetricPoint(long timestampSeconds, double value) {}

    // Saniyelik verileri 1 dakikalık ortalama noktasına indirgeme (Downsampling)
    public static MetricPoint rollupToMinute(List<MetricPoint> rawPoints) {
        if (rawPoints.isEmpty()) return null;
        double sum = 0;
        for (var p : rawPoints) sum += p.value();
        double avg = sum / rawPoints.size();
        return new MetricPoint(rawPoints.getFirst().timestampSeconds(), avg);
    }
}

void main() {
    // 1 dakikalık sürede toplanan 3 saniyelik CPU verisi: %40, %60, %50
    var raw = List.of(
        new MetricsRollupEngine.MetricPoint(1700000000L, 40.0),
        new MetricsRollupEngine.MetricPoint(1700000010L, 60.0),
        new MetricsRollupEngine.MetricPoint(1700000020L, 50.0)
    );

    var rolledUp = MetricsRollupEngine.rollupToMinute(raw);
    IO.println("Ham Veri Noktası Sayısı: " + raw.size());
    IO.println("Downsampled 1-Dk Ortalama: %" + rolledUp.value() + " [Zaman: " +
rolledUp.timestampSeconds() + "]");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta metrikleri standart ilişkisel veritabanına (MySQL/PostgreSQL) kaydedeceğinizi söylemeyin! Milyonlarca metrik satırı B-Tree indekslerini patlatır ve disk doldurur. Metrikler için daima TSDB (Prometheus, TimescaleDB, VictoriaMetrics, ClickHouse) önerilmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never pitch storing high-velocity time-series metrics inside a traditional relational database (MySQL/PostgreSQL). The volume of writes will shred B-Tree indexes and consume petabytes of disk. Prescribe purpose-built TSDBs like VictoriaMetrics or ClickHouse.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- TSDB veritabanları zaman serisi verisini Gorilla double-delta algoritmasıyla 10 kat sıkıştırır.
- Prometheus Pull modeli sunucu keşfiyle hafif çalışırken, Push modeli sunucusuz ortamlar için uygundur.
- Downsampling politikaları eskiyen verileri özetleyerek disk maliyetlerini %90 düşürür.

🎯 Key Takeaways for the Interview

- TSDBs compress timestamp-value streams by 10x using Gorilla double-delta and XOR algorithms.
- Pull architecture suits stateful microservice discovery; Push fits ephemeral serverless functions.
- Downsampling policies aggregate aging historical metrics, bounding storage costs sustainably.

51

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Web Önbelleği (Distributed Key-Value Store / DynamoDB Klonu)

Designing a Distributed Key-Value Store (DynamoDB Clone)

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ AMAZON DYNAMO DÖNÜM NOKTASI MİMARİSİ (THE DYNAMO PAPER) ]

1. Veri Bölümleme : Tutarlı Karışıklık Halkası (Consistent Hash Ring + Vnodes)
2. Yüksek Yazma   : Sloppy Quorum & Hinted Handoff (Düğüm ölse bile komşuya yaz!)
3. Çakışma Çözümü : Vektör Saatleri (Vector Clocks) & İstemci Çözümlemesi
4. Arka Plan Onarım: Merkle Trees & Anti-Entropy (Arka planda sessizce eşitleme)
5. Düğüm Keşfi    : Gossip Protocol (Dedikodu Protokolü ile merkeziyetsiz sağlık kontrolü)
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Amazon Dynamo, dünyanın en sağlam kargo ağı gibidir: Amazon'un kuralı şudur: 'Müşteri sepetine ürün eklerken sistem ASLA hata vermeyecek!' Bir düğüm arıza yapsa bile paket hemen yanındaki komşu düğüme emanet edilir (Sloppy Quorum & Hinted Handoff). Arızalı düğüm kendine geldiğinde emanetçi düğüm 'Bu paket senindi, buyur al' der. İki şube arasında anlaşmazlık çıkarsa Vektör Saatleri devreye girer. Sunucular aralarında fısıldaşarak (Gossip Protocol) kimin ayakta olduğunu birbirine haber verir. Tamamen merkeziyetsiz, lideri olmayan (Leaderless) ölümsüz bir mimaridir!

🇬🇧 ENGLISH (ELI5)

Amazon Dynamo is an un-killable decentralized courier grid designed around Amazon's prime directive: 'A customer adding an item to their shopping cart must NEVER see an error screen!' If the designated server node is down, the adjacent neighbor temporarily accepts the write (Sloppy Quorum & Hinted Handoff). Once the dead node recovers, the neighbor hands the package back. If conflicting updates clash, Vector Clocks preserve causal history. Nodes continuously whisper among themselves like neighbors at a fence (Gossip Protocol) to detect cluster topology and outages without a centralized master!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Werner Vogels ve ekibinin ünlü Amazon Dynamo Paper (2007) tasarımı MAANG mülakatlarının zirvesidir: 1) Sloppy Quorum & Hinted Handoff: Yazma sırasında belirlenen N düğümden biri erişilemezse, halkadaki bir sonraki sağlıklı düğüm veriyi geçici olarak kabul eder (`hinted handoff`). Düğüm ayağa kalkınca veri asıl sahibine teslim edilir (Erişilebilirlik tavan yapar). 2) Anti-Entropy with Merkle Trees: Düğümler arasındaki veri tutarsızlıklarını tüm veriyi ağdan taşımadan bulmak için Merkle Ağaçları (Hash Trees) kullanılır; ağaçların kök hash'leri uyuşuyorsa veriler eşittir. 3) Dedikodu Protokolü (Gossip Protocol): Merkezi bir ZooKeeper/etcd olmadan düğümler rastgele seçtikleri komşularla durum bilgilerini paylaşarak küme üyeliğini merkeziyetsiz olarak ($O(log N)$ sürede) senkronize eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Amazon Dynamo Paper (SOSP 2007) is the magnum opus of distributed systems architecture: 1) Sloppy Quorum & Hinted Handoff: During node partitions, if healthy nodes within the primary preference list fall below quorum W, the coordinator writes to fallback nodes outside the list with a 'hint' metadata tag. Once the primary recovers, the fallback pushes the mutation home, preserving write availability above all. 2) Anti-Entropy via Merkle Trees: Replicas synchronize divergent key ranges by comparing cryptographic Merkle Tree hash roots; branches are traversed only where hashes diverge, slashing network bandwidth. 3) Gossip Protocol: Nodes execute periodic peer-to-peer heartbeat exchanges with randomly selected nodes, propagating membership and failure detector states across the cluster in $O(log N)$ time without a central master coordinator.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Merkle Tree (Özet Ağacı) Mantığı ile Hızlı Veri Uyuşmazlığı Tespiti
import java.security.MessageDigest;

public class MerkleTreeSim {
    public static String hash(String input) {
        try {
            var md = MessageDigest.getInstance("SHA-256");
            return HexFormat.of().formatHex(md.digest(input.getBytes())).substring(0, 8);
        } catch (Exception e) { return ""; }
    }

    public static String computeRootHash(String dataBlockA, String dataBlockB) {
        String hashA = hash(dataBlockA);
        String hashB = hash(dataBlockB);
        // İki yaprağın hash'ini birleştirip üst düğümü oluştur:
        return hash(hashA + hashB);
    }
}

void main() {
    // İki farklı sunucudaki Merkle Kök Hash'lerini karşılaştır:
    String server1Root = MerkleTreeSim.computeRootHash("User:Ali", "Order:101");
    String server2Root = MerkleTreeSim.computeRootHash("User:Ali", "Order:101"); // Veriler aynı
    String server3Root = MerkleTreeSim.computeRootHash("User:Ali", "Order:999"); // Veri farklı!

    IO.println("Sunucu 1 ve 2 Eşit mi? " + server1Root.equals(server2Root) + " (Kök: " + server1Root +
")");
    IO.println("Sunucu 1 ve 3 Eşit mi? " + server1Root.equals(server3Root) + " (Uyuşmazlık Var!
Senkronize et)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta DynamoDB'nin 'Liderli (Master-based)' bir mimari olduğunu iddia etmeyin! Dynamo lidersizdir (Leaderless); her düğüm eşit yetkilere sahiptir. Bu sayede tek hata noktası (SPOF) barındırmaz.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never describe Dynamo as a master-slave system. Dynamo is strictly decentralized and leaderless; all nodes act as peers, eliminating single points of failure.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Sloppy Quorum ve Hinted Handoff düğüm arızalarında bile yazma erişilebilirliğini korur.
- Merkle Trees, düğümler arasındaki veri farklarını minimum ağ trafiğiyle senkronize eder.
- Gossip Protocol merkezi bir koordinatör olmadan küme sağlığını merkeziyetsiz yönetir.

🎯 Key Takeaways for the Interview

- Sloppy Quorum and Hinted Handoff guarantee write availability during node downtime.
- Merkle Trees identify replica divergence with minimal network bandwidth overhead.
- The Gossip protocol maintains decentralized cluster membership and failure detection.

52

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Dağıtık Mesaj Kuyruğu (Distributed Message Queue / Kafka Klonu) Tasarımı

Designing a Distributed Message Queue (Kafka Clone): Zero-Copy & Commit Log

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ KAFKA KLONU PERFORMANS SIRRI: ZERO-COPY DISK-TO-NETWORK ]

GELENEKSEL YÖNTEM (4 KULLANICI ALANI - KERNEL KOPYASI, YAVAŞ!):
Disk ──1. DMA──► OS Cache ──2. CPU──► JVM Heap ──3. CPU──► Socket Buffer ──4. DMA──► NIC

KAFKA ZERO-COPY YÖNTEMİ (LINUX sendfile(): SIFIR JVM KOPYASI!):
Disk ──1. DMA Kopya──► OS PageCache ──2. DMA Kopya──► Network Interface Card (NIC)
(CPU ve JVM tamamen atlanır; diskten doğrudan ağ kartına ışık hızında akar!)
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Normalde bir bilgisayarın diskindeki bir dosyayı internete göndermek, bir kargocunun paketi depodan alıp kendi ofis masasına getirmesi, paketi açması, tekrar bantlaması ve sonra kamyona taşıması gibidir (4 kez gereksiz yere taşınır). Kafka'nın Sıfır Kopyalama (Zero-Copy) sihri ise kargocuyu aradan çıkarıp depodan çıkan raylı sistemi doğrudan kamyonun kasasına bağlamaktır! Veri asla işletim sisteminin veya Java'nın hafızasına kopyalanıp vakit kaybetmez; doğrudan diskten ağ kablosuna fırlar!

🇬🇧 ENGLISH (ELI5)

Traditional file transmission over the network is like a warehouse worker loading a box onto their office desk, unboxing it, inspecting it, taping it shut again, and then carrying it to the delivery truck (4 redundant data copies and context switches!). Kafka's Zero-Copy optimization connects a direct gravity conveyor chute from the warehouse shelf straight into the back of the cargo truck! Data never touches JVM user-space memory; the Linux kernel routes raw disk PageCache bytes directly onto the Network Interface Card (NIC) at hardware line speed!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Dağıtık Mesaj Kuyruğu Tasarımının 3 Performans Direği: 1) Sıralı Disk Erişimi (Sequential IO): Diskler rastgele okumada (Random Seek) yavaştır ancak sıralı yazmada (Sequential Append) SSD'lerde 2-3 GB/sn hızlara ulaşır (RAM hızına çok yakındır). Kafka tüm mesajları sadece kütüğün sonuna ekler (Append-only). 2) Linux sendfile() (Zero-Copy): `transferTo()` sistem çağrısıyla veriler OS PageCache'ten doğrudan NIC buffer'ına kopyalanır. Context Switch (Kullanıcı alanı - Çekirdek geçişi) sayısı 4'ten 2'ye, veri kopyası sayısı 4'ten 2'ye düşer; CPU tüketimi sıfırlanır. 3) Toplu İşleme (Batching & Compression): Mesajlar tek tek değil, 64 KB'lık paketler halinde LZ4/ZSTD ile sıkıştırılarak ağdan aktarılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The 3 engineering pillars of Kafka's throughput mastery: 1) Sequential Disk IO: Mechanical and NVMe storage drives deliver massive bandwidth (gigabytes/sec) when written sequentially, rivaling raw DRAM speeds. Kafka leverages append-only commit logs exclusively. 2) Linux Zero-Copy (`sendfile` system call): Traditional reads transfer bytes: Disk -> PageCache -> User Space -> Socket Buffer -> NIC (4 buffer copies, 4 context switches). The `sendfile()` primitive transfers bytes directly from PageCache to the Network Interface Card via DMA, executing zero user-space memory copies and zero CPU load. 3) Batch Compression: Producers aggregate micro-messages into 64KB batches, compressing with LZ4/Snappy before transmission, maximizing network MTU utilization.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Zero-Copy (FileChannel transferTo) Simülasyonu
import java.nio.channels.*;

public class ZeroCopyDemonstrator {
    // Java NIO FileChannel.transferTo() arka planda Linux sendfile() kullanır!
    public static void zeroCopyTransfer(FileChannel sourceFile, WritableByteChannel targetSocket)
throws Exception {
        long position = 0;
        long count = sourceFile.size();

        // Veri JVM User-Space belleğine HİÇ girmeden doğrudan diskten ağ soketine uçar!
        while (position < count) {
            long transferred = sourceFile.transferTo(position, count - position, targetSocket);
            position += transferred;
        }
        IO.println("✔ Zero-Copy Transferi Tamamlandı! Boyut: " + count + " bytes");
    }
}

void main() {
    IO.println("Zero-Copy Metodu: FileChannel.transferTo() Hazır");
    IO.println("Avantaj: Sıfır JVM Garbage Collection, Sıfır CPU Buffer Kopyası!");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Mesajları hızlı olsun diye RAM'de (JVM Heap) saklarım' demeyin! JVM Heap'te milyonlarca mesaj saklamak Garbage Collection (GC) duraklamalarına yol açar ve sunucuyu kilitler. Kafka gibi sistemler işletim sisteminin PageCache'ini kullanarak disk ve RAM'i birlikte en verimli şekilde kullanır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose holding high-volume message queues within the JVM heap. Millions of heap objects trigger catastrophic Stop-The-World garbage collection pauses. Real systems delegate caching entirely to the underlying OS PageCache.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Sıralı disk yazması (Sequential IO) rastgele bellek erişiminden bile daha hızlı olabilir.
- Linux sendfile() Zero-Copy tekniği CPU ve JVM hafıza kopyalama yükünü tamamen sıfırlar.
- Toplu mesajlaşma (Batching) ağ başlık yükünü minimize ederek throughput'u katlar.

🎯 Key Takeaways for the Interview

- Sequential disk writes achieve multi-gigabyte throughput, outpacing random memory access.
- Linux sendfile() zero-copy streams bytes directly from disk cache to the network card.
- Batching micro-messages into compressed frames maximizes network packet density.

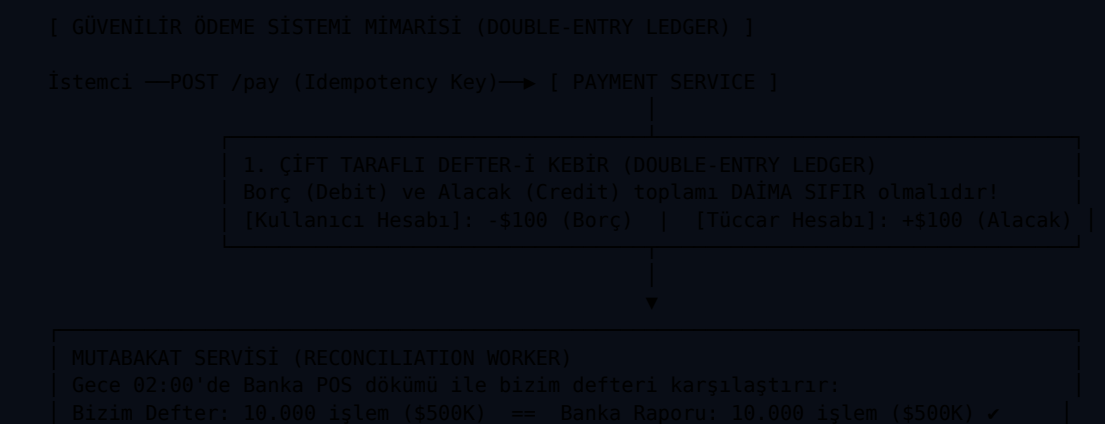
53

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Ödeme Sistemi (Payment Gateway / Stripe): Çift Taraflı Muhasebe ve Mutabakat

Designing a Payment System (Stripe): Double-Entry Bookkeeping & Reconciliation

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Cüzdanınızdan 100 TL çıkarıp bir market sahibine verdiğinizde ne olur? Para havadan yok olmaz veya gökten zembille inmez! Sizin cebinizden 100 TL eksilir (-100 TL), marketçinin kasasına 100 TL girer (+100 TL). İkisini toplarsanız sonuç SIFIRDIR! 500 yıldır Venedikli tüccarların kullandığı 'Çift Taraflı Muhasebe' (Double-Entry Bookkeeping) budur. Stripe gibi devasa bir ödeme sisteminde tek bir 'bakiye' sayısı tutulmaz; tüm para hareketleri borç ve alacak olarak kaydedilir. Gece olunca da bankanın hesap dökümü ile bizim kayıtlarımız kuruşu kuruşuna karşılaştırılır (Mutabakat - Reconciliation)!

🇬🇧 ENGLISH (ELI5)

When you hand a $100 bill to a grocery clerk, cash doesn't materialize out of thin air or vanish into the void! Your wallet decreases by -$100 (Debit), while the register increases by +$100 (Credit). The net sum of the universe remains exactly ZERO! This is 500-year-old Double-Entry Bookkeeping. In a world-class payment platform like Stripe, you never store a simple mutable `balance = 500` column. You record immutable debit and credit ledger entries. At night, an automated auditor (Reconciliation Engine) cross-references internal ledgers against bank settlement files cent-for-cent!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Ödeme Sistemi Mimarisi Gereksinimleri: 1) Idempotency Garantisi: Ağ kopmalarında mükerrer para çekilmesini önlemek için istemci tarafından gönderilen `Idempotency-Key` zorunludur. 2) Çift Taraflı Muhasebe (Double-Entry Bookkeeping): Asla `UPDATE accounts SET balance = balance - 100` yapılmaz! Her işlem en az iki bacaklı bir defter kaydıdır (Ledger Entry): Biri Borç (Debit), diğeri Alacak (Credit). Sistem genelindeki tüm borç ve alacakların toplamı her an tam olarak sıfır (0) olmak zorundadır. 3) Mutabakat (Reconciliation): Bankalar ve PSP'ler (Stripe, Adyen, Visa) her gece CSV/XML formatında hesap mutabakat dosyaları (Settlement Files) gönderir. Arka plandaki mutabakat motoru, bizim yerel defterimizdeki işlemlerle bankanın bildirdiği işlemleri kuruşu kuruşuna eşleştirir; eksik veya hatalı işlemlerde alarm çalar.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Payment Gateway Architecture: 1) Guaranteed Idempotency: Mitigates double-charging during network retries via client-supplied `Idempotency-Key` headers locked inside distributed Redis stores. 2) Double-Entry Bookkeeping Ledger: Strictly avoids mutable balance columns (`balance = balance - 100`). Money movements are immutably logged across dual ledger legs: Every transaction creates matching Debit and Credit entries across distinct accounts. Invariant: ∑ Debits - ∑ Credits = 0 across the global ledger at all times. 3) Reconciliation Engine: Financial partners (Visa, Chase, Adyen) transmit end-of-day settlement files. An automated asynchronous reconciliation pipeline cross-audits internal ledger transactions against partner clearing logs, flagging uncaptured funds or discrepancies for manual treasury review.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Çift Taraflı Muhasebe Defteri (Double-Entry Bookkeeping) Modeli
import java.math.BigDecimal;
import java.util.List;

public class DoubleEntryLedger {
    public enum EntryType { DEBIT, CREDIT }
    public record LedgerEntry(String accountId, EntryType type, BigDecimal amount) {}

    public static boolean validateTransaction(List<LedgerEntry> entries) {
        BigDecimal balance = BigDecimal.ZERO;
        for (LedgerEntry entry : entries) {
            if (entry.type() == EntryType.DEBIT) {
                balance = balance.subtract(entry.amount());
            } else {
                balance = balance.add(entry.amount());
            }
        }
        // Borç ve alacak toplamı SIFIR olmalıdır!
        return balance.compareTo(BigDecimal.ZERO) == 0;
    }
}

void main() {
    var amount = new java.math.BigDecimal("150.00");
    var entries = List.of(
        new DoubleEntryLedger.LedgerEntry("ACC_USER_ALİ", DoubleEntryLedger.EntryType.DEBIT, amount),
        new DoubleEntryLedger.LedgerEntry("ACC_MERCHANT_XYZ", DoubleEntryLedger.EntryType.CREDIT,
amount)
    );

    boolean balanced = DoubleEntryLedger.validateTransaction(entries);
    IO.println("Muhasebe Kaydı Dengeli mi? " + balanced + " (Borç = Alacak)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta para işlemlerini saklarken `FLOAT` veya `DOUBLE` veri tipi kullanacağınızı ASLA söylemeyin! Kayan noktalı sayılar (Floating-point) ikili sistemde yuvarlama hataları yaratır (0.1 + 0.2 = 0.30000000000000004). Milyonlarca dolarlık sistemlerde açık çıkar! Para daima `BIGINT` (kuruş/cent cinsinden) veya Java `BigDecimal` ile tutulmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never use FLOAT or DOUBLE primitives for currency in software engineering! Binary floating-point arithmetic introduces rounding inaccuracies that leak money. Always store currency as 64-bit BIGINT integers representing cents or Java `BigDecimal` with explicit rounding modes.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Idempotency Key ödeme sistemlerinde mükerrer para çekimini engelleyen ilk savunma hattıdır.
- Çift Taraflı Muhasebe (Double-Entry) paranın asla yoktan var veya vardan yok olmamasını garanti eder.
- Mutabakat (Reconciliation) motoru iç defter ile banka dökümlerini karşılaştırarak tutarlılık sağlar.

🎯 Key Takeaways for the Interview

- Idempotency keys prevent double-charging users during client network retries.
- Double-entry bookkeeping ensures every debit is mirrored by an exact credit.
- Reconciliation pipelines audit internal transactional state against external bank settlements.

54

BÖLÜM: İLERI SEVIYE MAANG TASARIM PROBLEMLERI | MODULE: ADVANCED MAANG DISTRIBUTED SYSTEM DESIGNS

Canlı Oyun Lider Tablosu (Gaming Leaderboard): Redis Sorted Sets ve SkipList

Designing a Real-Time Gaming Leaderboard: Redis ZSET & SkipLists

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ REDIS SORTED SET (ZSET) İÇ MİMARİSİ: SKIPLIST (ATLAMALI LİSTE) ]
Kullanıcı skorları logaritmik O(log N) sürede sıralanır ve güncellenir:

Level 3: [Head] ──────────────────────────────────────────► [Skor: 1500] ──► null
Level 2: [Head] ──────────────────► [Skor: 800] ──────────► [Skor: 1500] ──► null
Level 1: [Head] ──► [Skor: 300] ──► [Skor: 800] ──► [Skor: 950] ──► [Skor: 1500] ──► null
(10 Milyon oyuncu olsa bile 25 atlamada oyuncunun dünya sıralaması bulunur!)

[ LEADERBOARD SORGULARI ]
1. ZADD leaderboard 1500 "user_ali"  ──► Skoru O(log N) sürede güncelle
2. ZREVRANK leaderboard "user_ali"   ──► Ali'nin dünya sıralamasını getir (1. sıra)
3. ZREVRANGE leaderboard 0 9 WITHSCORES ──► Dünya Top-10 listesini anında getir!
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

10 milyon kişinin aynı anda oynadığı bir mobil oyunu düşünün. Herkes canavar öldürdükçe puanı artıyor. 'Ben dünyada kaçıncı sıradayım?' dediğinizde, 10 milyon kişiyi sıfırdan küçükten büyüğe sıralamaya kalkarsanız oyun donar! Redis Sorted Set işte bu yüzden sihirli bir ekspres trene (SkipList) benzer: En alt katta her istasyon vardır; ama üst katlarda sadece ana istasyonlar vardır. Tren üst kattan son hızla atlayarak gider ve 10 milyon kişi arasından sizin sıranızı göz açıp kapayıncaya kadar (1 milisaniyede) bulup çıkarır!

🇬🇧 ENGLISH (ELI5)

In a mobile game with 10 million active players, scores fluctuate every microsecond. If querying 'What is my global rank?' forced the database to run an $O(N log N)$ sort across 10 million rows, servers would melt! Redis Sorted Sets solve this using a multi-level express subway called a SkipList. The bottom track stops at every single station, while elevated tracks skip 10 or 100 stations at a time. The algorithm rides the express tracks, finding any player's exact global rank among 10 million contenders in under 1 millisecond ($O(log N)$)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Oyun Lider Tablosu Mimarisi: 1) Neden İlişkisel DB Yetmez? Milyonlarca oyuncunun olduğu bir tabloda `SELECT COUNT(*) WHERE score > my_score` sorgusu tam bir felakettir; her puan değişiminde indeks kilitlenir. 2) Redis ZSET (Sorted Set): İki veri yapısının mükemmel birleşimidir: A) HashMap: Oyuncu adına göre skoru $O(1)$ sürede bulur. B) SkipList (Atlamalı Liste): Skorları sıralı tutar. Skor güncelleme (`ZADD`), sıralama bulma (`ZREVRANK`) ve aralık okuma (`ZREVRANGE`) $O(log N)$ sürede biter. 10 milyonluk bir tabloda $log_2(10^7) approx 24$ işlemde sonuç alınır. 3) Eşit Puanların Ayrılması: İki oyuncu aynı puana sahipse (ör. ikisi de 1500), skora zaman damgası eklenerek erken puan alan öne geçirilir: `EfektifSkor = Skor - (Timestamp / 10^13)`.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Scale Gaming Leaderboard Architecture: 1) Relational DB Breakdown: Executing `SELECT COUNT(*) FROM leaderboard WHERE score > user_score` executes full table/index scans that crush concurrency. 2) Redis ZSET (Sorted Set Duality): Blends a Hash Map (for $O(1)$ player lookup) with a probabilistic multi-level SkipList (for $O(log N)$ insertions, removals, and rank lookups). For a leaderboard of 10 million gamers, traversing the SkipList requires at most $approx 24$ pointer hops. 3) Common Leaderboard APIs: `ZADD leaderboard score user_id` updates a score; `ZREVRANK leaderboard user_id` returns global rank; `ZREVRANGE leaderboard 0 9` fetches the global Top-10. 4) Tie-Breaking Mechanics: When two players share identical scores (e.g. 1500 points), prioritize the player who achieved it first by subtracting a scaled epoch timestamp: `CompositeScore = Score + (1 - (Timestamp / 10^13))`.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Eşit Puanları Zaman Damgasıyla Çözen Lider Tablosu Skoru
public class LeaderboardScoreCalculator {
    // Eşit puanlarda önce ulaşanı üst sıraya taşıyan formül:
    public static double computeCompositeScore(long points, long timestampMillis) {
        // Puan tamsayı kısmında kalır, zaman damgası kesirli kısma küçük bir eksi olarak eklenir
        double timePenalty = (double) timestampMillis / 1_000_000_000_000.0;
        return points + (1.0 - (timePenalty % 1.0));
    }
}

void main() {
    long score = 2500;
    long timeAli = 1700000000000L;       // Ali saat 12:00'de ulaştı
    long timeVeli = 1700000060000L;      // Veli 1 dakika sonra ulaştı

    double aliComposite = LeaderboardScoreCalculator.computeCompositeScore(score, timeAli);
    double veliComposite = LeaderboardScoreCalculator.computeCompositeScore(score, timeVeli);

    IO.println("Ali'nin Skoru:  " + String.format("%.8f", aliComposite));
    IO.println("Veli'nin Skoru: " + String.format("%.8f", veliComposite));
    IO.println("Ali Veli'den Önde mi? " + (aliComposite > veliComposite)); // TRUE!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Dünya Top-100 listesini her oyuncunun ekranında saniyede 1 kez polling ile yenilerim' demeyin! 5 milyon aktif oyuncu saniyede 5 milyon istek demektir. Top-100 listesi CDN veya yerel bellek önbelleğinde 5 saniyelik bir TTL ile tutulmalı, kullanıcının kendi sırası WebSocket ile sadece değiştikçe iletilmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Polling the global Top-100 leaderboard every second from millions of mobile clients will saturate Redis network bandwidth. Cache the Top-100 at CDN edge PoPs with a 5-second TTL, pushing personalized rank mutations over WebSockets only when milestones cross.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis ZSET (SkipList) 10 milyonluk kümelerde bile sıralamayı O(log N) sürede bulur.
- Eşit puanlarda (Tie-Breaking) zaman damgası eklenerek erken ulaşan ödüllendirilir.
- Top-100 gibi global listeler CDN seviyesinde kısa TTL ile önbelleğe alınarak Redis korunur.

🎯 Key Takeaways for the Interview

- Redis ZSETs leverage SkipLists to resolve user ranks in O(log N) time across millions of users.
- Composite scoring incorporates timestamp fractions to break score ties deterministically.
- Edge-caching Top-100 leaderboards protects Redis from millions of redundant read queries.

BÖLÜM 10 / MODULE 10

Mülakat Ustalığı, Canlı Beyaz Tahta & Staff+ Seviyesi

MAANG Interview Mastery, Whiteboarding & Staff+ Playbook

45 dakikalık mülakatı yönetme sanatı, netleştirme soruları, Zero-Trust ve mTLS güvenliği, Çok Bölgeli (Multi-Region) felaket kurtarma, FinOps bulut maliyeti ve Staff+ Büyük Kapanışı.

Mastering the 45-minute whiteboard session, clarifying questions, Zero-Trust & mTLS, Active-Active Multi-Region DR, Cloud FinOps economics, and the Staff+ Architecture Finale.

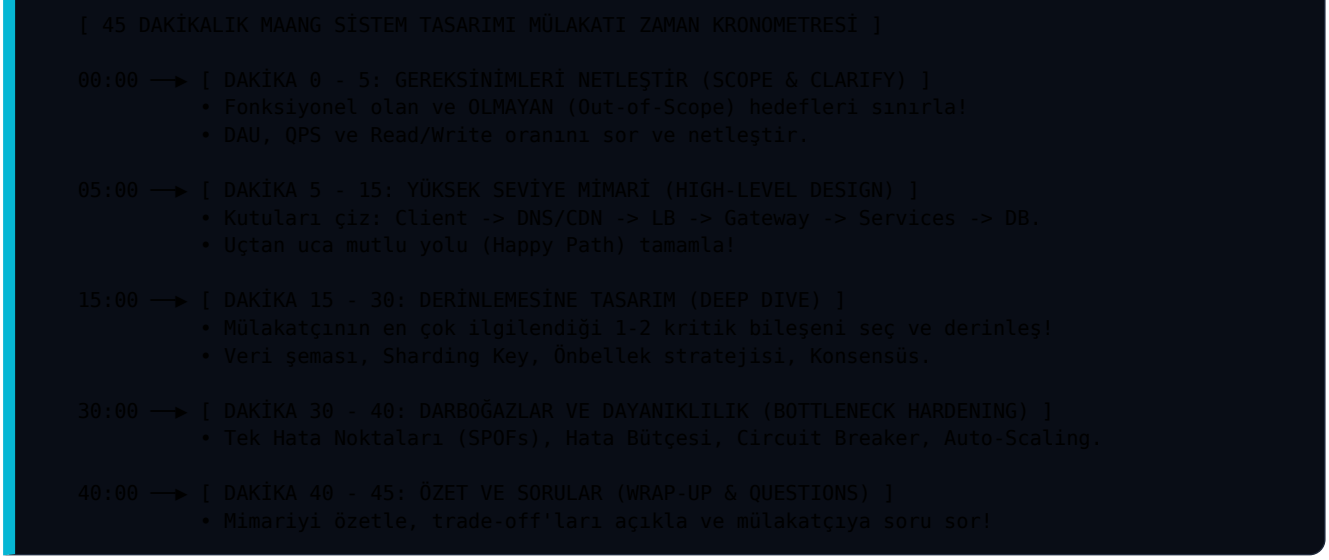
55

BÖLÜM: MÜLAKAT USTALIĞI, CANLI BEYAZ TAHTA & STAFF+ SEVIYESI | MODULE: MAANG INTERVIEW MASTERY, WHITEBOARDING & STAFF+ PLAYBOOK

Beyaz Tahtada Mülakat Yönetimi: 45 Dakikayı Dakika Dakika Yönetmek

Managing the 45-Minute Whiteboard Session: Minute-by-Minute Playbook

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Mülakatı 45 dakikalık bir futbol maçı gibi düşünün: İlk 5 dakika sahayı ve rakibi tanırsınız (Soruları sorar, taktiği belirlersiniz). Sonraki 10 dakika ilk golü atar, tahtaya genel oyun planını çizersiniz (High-Level mimari). Maçın orta kısmında (15-30. dk) en yetenekli forvetinizi sahaya sürer, sistemin en zor parçasının röntgenini çekersiniz (Deep Dive). Son dakikalarda ise defansı sağlama alır, 'Rakip kontra atağa kalkarsa ne yaparız?' diyerek olası arızaları ve yedekleri anlatırsınız (Resilience). Süreyi iyi yöneten maçı 3-0 kazanır!

🇬🇧 ENGLISH (ELI5)

Treat the 45-minute interview like a World Cup final: Minutes 0-5 are feeling out the pitch: ask clarifying questions, set game rules, and lock the scope. Minutes 5-15 score the first goal: sketch the end-to-end high-level happy path from browser to database. Minutes 15-30 showcase your star striker: deep-dive into the single most treacherous bottleneck (sharding, caching, consensus). Minutes 30-40 fortify defense: identify single points of failure and explain how the system survives datacenter outages. The final 5 minutes summarize trade-offs. Flawless time management wins offers!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG mülakatlarında zaman yönetimi, teknik bilgi kadar belirleyicidir. Adayların %70'i zamanı yönetemediği için elenir (örneğin ilk 25 dakikayı kapasite hesabında harcayıp mimariye geçemezler). 1) Direksiyonu Siz Tutun (Drive the Discussion): Mülakatçı pasif bir dinleyicidir; sizin bir Staff Mühendis gibi mülakatı yapılandırmanızı ve 'Şimdi yüksek seviye mimariyi çiziyorum, ardından izin verirseniz önbellek tutarlılığına derinlemesine ineceğim' diyerek gündemi belirlemenizi bekler. 2) Sinyal Gönderimi (Signaling): Mülakatçının yüz ifadesini ve ipuçlarını okuyun; eğer 'Veri modeline biraz daha bakalım mı?' diyorsa hemen o konuya odaklanın.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Time management in MAANG rounds is an explicit evaluation axis (Leadership & Communication signal). Over 70% of rejections stem from mismanaged pacing (e.g. spending 25 minutes calculating petabytes on napkin math without drawing a single database). 1) Drive the Agenda Proactively: Staff+ candidates command the room: 'I've scoped the core APIs; next, I will sketch the high-level architecture before deep-diving into the fan-out queue. Does that match your expectations?' 2) Pivot on Interviewer Prompts: Interviewers drop hints: 'How does this behave when redis drops?' Do not defend an ego; lean into their prompt and articulate failure modes instantly.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Mülakat Zaman Takipçisi ve Aşama Kontrol Modeli
public record InterviewStage(String stageName, int targetMinutes, String primaryDeliverable) {
    public static java.util.List<InterviewStage> getStandardPlaybook() {
        return java.util.List.of(
            new InterviewStage("1. Kapsam & SLA Netleştirme", 5, "Fonksiyonel gereksinimler, DAU, QPS,
P99 hedefleri"),
            new InterviewStage("2. Yüksek Seviye Mimari", 10, "Kutuları çiz: Client, CDN, LB, Gateway,
Services, DB"),
            new InterviewStage("3. Derinlemesine Tasarım", 15, "Veri şeması, Sharding Key, Önbellek,
Olay Akışı"),
            new InterviewStage("4. Darboğaz & Dayanıklılık", 10, "SPOF analizi, Circuit Breaker,
Felaket Kurtarma"),
            new InterviewStage("5. Kapanış & Soru-Cevap", 5, "Trade-off özeti ve mülakatçıya teknik
sorular")
        );
    }
}

void main() {
    IO.println("=== 45 DAKİKALIK MAANG MÜLAKAT KRONOMETRESİ ===");
    int total = 0;
    for (var stage : InterviewStage.getStandardPlaybook()) {
        total += stage.targetMinutes();
        IO.println("[" + String.format("%02d", total) + " dk] " + stage.stageName() + " ⟶ " +
stage.primaryDeliverable());
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçının 'Bir sonraki adıma geçelim' uyarısına rağmen inatla aynı konuyu uzatmayın! Bu durum 'İletişime kapalı' (Low coachability / stubborn) sinyali verir ve mülakatın doğrudan olumsuz sonuçlanmasına yol açar.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Ignoring an interviewer's gentle nudge ('Let's move on to the storage tier') to keep obsessing over API parameters signals poor coachability and stubbornness. Always acknowledge, pivot, and follow their conversational lead.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Mülakatın direksiyonunu bir Staff Mühendis gibi nazikçe ve proaktif olarak yönetin.
- Kapasite hesaplarında 3 dakikadan fazla oyalanmayın; mimariye hızla geçin.
- Son 10 dakikayı mutlaka darboğazları ve hata senaryolarını çözmeye ayırın.

🎯 Key Takeaways for the Interview

- Proactively command the interview agenda like an experienced Staff Engineer.
- Never waste more than 3 minutes on napkin arithmetic; advance to architecture swiftly.
- Reserve the final 10 minutes strictly for hardening failure domains and edge cases.

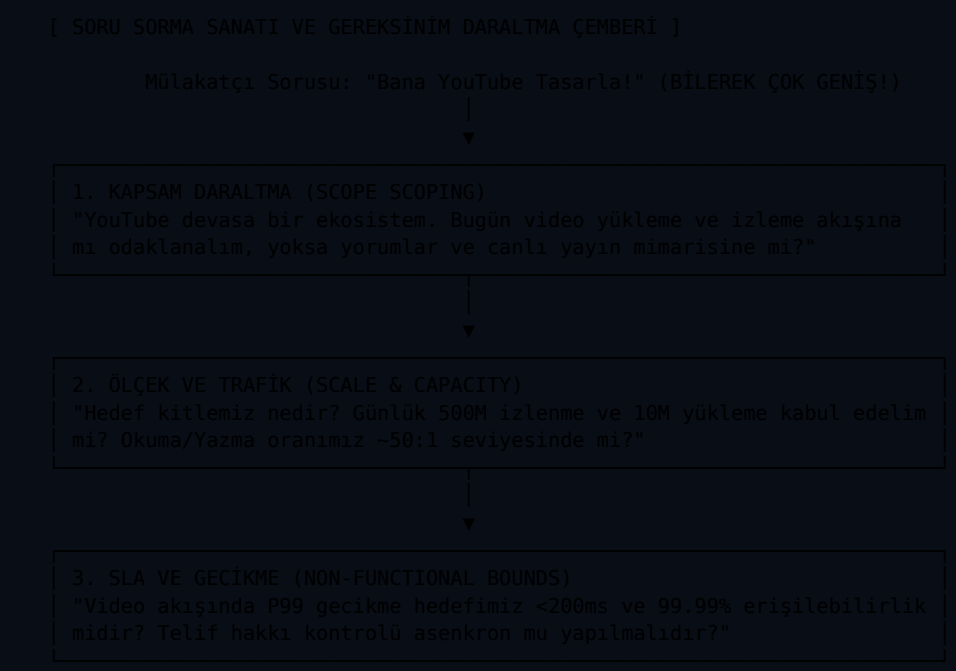
56

BÖLÜM: MÜLAKAT USTALIĞI, CANLI BEYAZ TAHTA & STAFF+ SEVIYESI | MODULE: MAANG INTERVIEW MASTERY, WHITEBOARDING & STAFF+ PLAYBOOK

Belirsizlik ve Eksik Gereksinim Yönetimi: Soru Sorma Sanatı (Clarifying Questions)

Navigating Ambiguity: The Art of Asking Clarifying Questions

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir lokantaya girdiğinizde garsona 'Bana yemek getir!' derseniz garson size bakar ve güler: 'Tatlı mı, et mi, çorba mı? Alerjiniz var mı? Bütçeniz ne kadar?' diye sorar. MAANG mülakatçısı da size bilerek 'Bana YouTube yap' der ve arkasına yaslanır. Eğer hemen 'Hemen veritabanı çiziyorum' derseniz garsona 'Bana rastgele bir şey getir' diyen acemi müşteri gibi olursunuz! Usta bir mühendis mülakatçıya doğru soruları sorarak sınırları çizer: 'Canlı yayın mı, kayıtlı video mu? Global mi, yerel mi?'

🇬🇧 ENGLISH (ELI5)

If you sit down at a 5-star restaurant and bark: 'Bring me food!', the waiter stares back baffled: 'Steak, seafood, or pasta? Do you have peanut allergies? Are we dining fast or celebratory?' MAANG interviewers intentionally throw ultra-vague prompts like 'Design Uber' and lean back to watch your reaction. Leaping straight into drawing databases marks you as a reactive junior. A seasoned engineer unpacks ambiguity with surgical questions: 'Are we optimizing for ride dispatching or driver onboarding? Real-time tracking or fare billing?'

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Belirsizlik Yönetimi (Ambiguity Navigation) MAANG değerlendirme matrisinde L5/L6 (Senior/Staff) terfi kriterinin temelidir. Mülakatçı gereksinimleri bilerek eksik bırakır; amacı adayın belirsizlik karşısındaki mühendislik olgunluğunu ölçmektir. Sorulması Gereken 4 Boyutlu Sorular: 1) Kullanıcı & Fonksiyonel Kapsam: Temel 2-3 kullanım senaryosu nedir? Kapsam dışı (Out of Scope) bırakacağımız özellikler nelerdir? 2) Ölçek (Scale): DAU (Günlük Aktif Kullanıcı), QPS, Read-to-Write oranı. 3) Ağ ve Cihaz Kısıtları: Mobil istemciler zayıf 3G/4G ağlarda mı çalışacak? 4) Güvenilirlik & Tutarlılık (SLAs): Güçlü tutarlılık mı (ACID), yoksa yüksek erişilebilirlik ve nihai tutarlılık mı (BASE)?

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Ambiguity Navigation is a tier-one rubric in L5/L6 (Senior/Staff) hiring committees. Real production problems arrive without specifications. Interviewers simulate this by withholding context. Systematic Clarification Matrix: 1) Core User Persona: What are the primary 2-3 user journeys? What are we explicitly declaring OUT OF SCOPE? (e.g. 'We will omit video recommendations and user comments today'). 2) Scale Envelope: Daily Active Users, Peak write/read QPS, average payload size. 3) Client Environmental Constraints: Mobile networks with flaky connectivity requiring offline caching and resume capability? 4) Latency & Consistency Constraints: P99 bounds, strict consistency vs eventual consistency.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Mülakat Kapsam Sözleşmesi Modeli (Scope Contract)
import java.util.List;

public record SystemScopeContract(
    String systemName,
    List<String> inScopeFeatures,
    List<String> outOfScopeFeatures,
    long dailyActiveUsers,
    int peakQps
) {
    public void printContractSummary() {
        IO.println("=== MÜLAKAT KAPSAM SÖZLEŞMESİ: " + systemName + " ===");
        IO.println("Dahil Edilenler (In-Scope):      " + inScopeFeatures);
        IO.println("Kapsam Dışı (Out-of-Scope):      " + outOfScopeFeatures);
        IO.println("Ölçek Parametreleri:             " + dailyActiveUsers + " DAU | Zirve QPS: " +
peakQps);
    }
}

void main() {
    var youtubeScope = new SystemScopeContract(
        "YouTube Video Platformu",
        List.of("Video Yükleme", "Adaptif Akış (HLS)", "İzleme Sayacı"),
        List.of("Yorumlar", "Sosyal Beğeniler", "Canlı Yayın", "Öneri Algoritması"),
        500_000_000L,
        150_000
    );
    youtubeScope.printContractSummary();
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatçıya 50 tane gereksiz detay soru sorup 'Analysis Paralysis' (analiz felci) yaşamayın! Sorularınızı 4-5 kritik soruyla sınırlayın, ardından 'Eğer onaylarsanız bu varsayımlarla devam etmek istiyorum' diyerek inisiyatif alın.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Avoid Analysis Paralysis by firing 40 trivial questions. Cap clarifications at 4-5 strategic questions, state reasonable engineering assumptions, and proactively validate: 'Unless you prefer otherwise, I will assume 100M DAU and proceed.'

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Mülakatçı soruyu bilerek muğlak sorar; amacı belirsizlik yönetimini ölçmektir.
- Kapsam dışı (Out-of-Scope) bırakılan özellikleri mülakatın başında açıkça belirleyin.
- Varsayımlarınızı sesli olarak ifade edip mülakatçının onayını alarak ilerleyin.

🎯 Key Takeaways for the Interview

- Vague prompts are intentional tests to evaluate how you structure architectural chaos.
- Explicitly declare Out-of-Scope features early to shield precious time.
- State architectural assumptions out loud and confirm alignment with the interviewer.

57

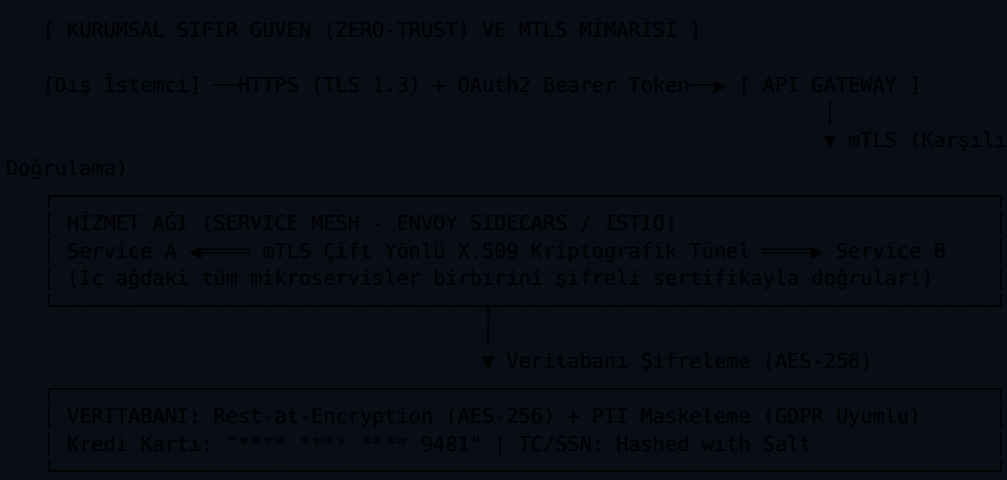
BÖLÜM: MÜLAKAT USTALIĞI, CANLI BEYAZ TAHTA & STAFF+ SEVIYESI | MODULE: MAANG INTERVIEW MASTERY, WHITEBOARDING & STAFF+ PLAYBOOK

Güvenlik, Kimlik ve Uyumluluk: OAuth2, mTLS, Zero-Trust ve GDPR

Security, Identity & Compliance: OAuth2, mTLS, Zero-Trust & GDPR

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski kale modelinde kalenin etrafında derin bir su hendeği ve tek bir asma kapı vardı (Ağ Güvenlik Duvarı - Firewall). Kapıdan içeri bir kez giren casus, kalenin içindeki tüm odalara elini kolunu sallayarak girerdi! Modern Sıfır Güven (Zero-Trust) mimarisi ise şudur: Kalenin içinde bile olsanız, mutfağa girerken mutfak kapısındaki güvenlik görevlisine kimlik gösterirsiniz, kütüphaneye girerken kütüphaneciye parmak izi verirsiniz (mTLS). Kimse kimseye babasının oğlu gibi güvenmez; her oda her kapıda şifrelenir!

🇬🇧 ENGLISH (ELI5)

Legacy security relied on the 'Castle-and-Moat' model: a deep perimeter moat and a single drawbridge (Corporate Firewall). Once a hacker sneaked past the drawbridge, they roamed internal palace corridors unhindered! Modern Zero-Trust architecture abandons perimeter trust completely: Even if you sit inside the inner castle corridor, entering the kitchen requires a biometric badge, and entering the treasury requires an armed escort (Mutual TLS / mTLS). Inside a Zero-Trust network, services verify cryptographic identities for every internal hop!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Güvenlik ve Uyumluluk Mimarisi Direkleri: 1) Zero-Trust & mTLS (Mutual TLS): 'Ağ içindekilere güven' varsayımı çökmüştür. Mikroservisler arası tüm iletişim Envoy/Istio Service Mesh üzerinden çift yönlü X.509 sertifika doğrulamasıyla (mTLS) şifrelenir. 2) OAuth 2.0 & OIDC: İstemci yetkilendirmesi için PKCE (Proof Key for Code Exchange) destekli Authorization Code Flow; servisler arası iletişim için JWT tabanlı Client Credentials. 3) GDPR ve PII (Personally Identifiable Information) Uyumluluğu: 'Unutulma Hakkı' (Right to be Forgotten) için kullanıcı verileri kriptografik olarak silinebilir (Crypto-Shredding: Kullanıcının özel şifreleme anahtarı silindiğinde veritabanındaki şifreli veri anında anlamsız çöp haline gelir). 4) Data at Rest Encryption: Veritabanı diskleri AES-256 ve AWS KMS / HashiCorp Vault anahtarlarıyla şifrelenir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Enterprise Security & Compliance Framework: 1) Zero-Trust & Mutual TLS (mTLS): Eliminates internal network perimeter trust. Microservices authenticate reciprocally via X.509 certificates managed by Istio/SPIFFE, ensuring all transit traffic is cryptographically sealed with ephemeral keys. 2) Modern Identity (OAuth 2.1 & OpenID Connect): Public mobile clients employ PKCE Authorization Code flows; internal machine-to-machine transit uses short-lived JWTs signed by asymmetric public/private keys. 3) GDPR & PII Protection via Crypto-Shredding: When a user exercises their GDPR 'Right to be Forgotten', scrubbing distributed backups is nearly impossible. Instead, encrypt the user's PII with a unique per-user DEK (Data Encryption Key). Deleting the key in KMS permanently renders all historical multi-region database and backup records irreversible garbage. 4) Encryption in Transit & at Rest: Enforce TLS 1.3 across the wire and AES-256 disk volume encryption backed by Hardware Security Modules (HSM).

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: GDPR Uyumlu Kriptografik Veri Maskeleme (PII Data Masking)
public class PiiMasker {
    public static String maskEmail(String email) {
        if (email == null || !email.contains("@")) return "*****";
        var parts = email.split("@");
        String name = parts[0];
        String maskedName = name.charAt(0) + "***" + (name.length() > 1 ? name.charAt(name.length() -
1) : "");
        return maskedName + "@" + parts[1];
    }

    public static String maskCreditCard(String pan) {
        if (pan == null || pan.length() < 10) return "****";
        return "****-****-****-" + pan.substring(pan.length() - 4);
    }
}

void main() {
    String rawEmail = "aliturgutbozkurt@example.com";
    String rawCard = "4543123456789481";

    IO.println("Maskelenmiş E-posta: " + PiiMasker.maskEmail(rawEmail));
    IO.println("Maskelenmiş Kredi Kartı: " + PiiMasker.maskCreditCard(rawCard));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'İç ağımız zaten güvenli, mikroservisler arasında şifreleme yapmaya gerek yok' demeyin! Modern MAANG standartlarında 'Zero-Trust' esastır; iç ağdaki her istek mTLS ile şifrelenmeli ve doğrulanmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never state that internal microservices communicate in unencrypted plain HTTP because they are 'inside the private VPC'. Staff engineers mandate Zero-Trust: all internal mesh transit must be encrypted via mTLS.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Zero-Trust felsefesi ağın içindeki hiçbir servise ön koşulsuz güvenmez.
- mTLS (Karşılıklı TLS) mikroservislerin birbirini X.509 sertifikalarıyla doğrulamasını sağlar.
- Crypto-Shredding tekniği GDPR 'unutulma hakkını' dağıtık yedeklerde bile saniyede çözer.

🎯 Key Takeaways for the Interview

- Zero-Trust assumes the internal network is hostile, verifying every request cryptographically.
- Mutual TLS enforces reciprocal cryptographic authentication across service mesh pods.
- Crypto-shredding satisfies GDPR deletion by destroying user-specific encryption keys.

58

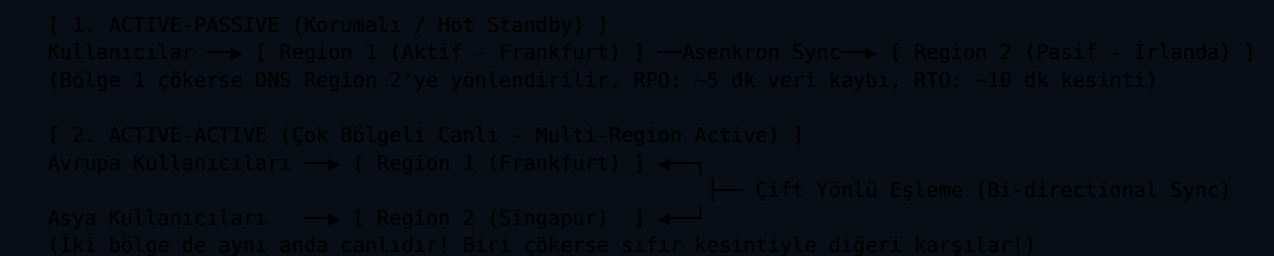
BÖLÜM: MÜLAKAT USTALIĞI, CANLI BEYAZ TAHTA & STAFF+ SEVIYESI | MODULE: MAANG INTERVIEW MASTERY, WHITEBOARDING & STAFF+ PLAYBOOK

Felaket Kurtarma ve Çok Bölgeli Mimari: Active-Active vs Active-Passive

Disaster Recovery & Multi-Region: Active-Active vs Active-Passive

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Felaket kurtarma modellerini iki farklı araba lastiği gibi düşünün: 1) Active-Passive: Arabanın bagajındaki yedek lastiktir (Stepne). Bagajda hazır bekler; yoldaki lastik patlarsa arabayı durdurur, krikoyu kaldırır, stepneyi takarsınız (15 dakika vakit kaybedersiniz - RTO). 2) Active-Active: 8 tekerlekli dev bir maden kamyonudur! 8 tekerleğin hepsi aynı anda döner. Tekerleklerden biri patlasa bile kamyon durmaz, hızını hiç kesmeden yoluna devam eder! Ancak 8 tekerleğin de aynı hızda dönmesini sağlamak (Veri senkronizasyonu) çok daha büyük bir mühendislik ister!

🇬🇧 ENGLISH (ELI5)

Compare disaster recovery to vehicular resilience: 1) Active-Passive is carrying a spare tire in your car trunk. It sits idle. When your front tire blows out, you must pull over, hoist the jack, and bolt on the spare (15 minutes of downtime - RTO). 2) Active-Active is an 8-wheeled military assault vehicle: All 8 wheels drive concurrently. If tire 3 gets shot out by a landmine, the vehicle doesn't stop for a single second; it barrels forward uninterrupted! However, keeping all 8 wheels synchronized across diverse terrain (bi-directional cross-region replication) requires supreme engineering discipline!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Felaket Kurtarma (Disaster Recovery - DR) Metrikleri ve Modelleri: 1) İki Kritik Metrik: A) RPO (Recovery Point Objective): Felaket anında kabul edilebilir maksimum veri kaybı süresi (ör. 'Son 1 dakikalık veri kaybolabilir'). B) RTO (Recovery Time Objective): Sistemin ayağa kalkması için izin verilen maksimum kesinti süresi (ör. 'Sistem 5 dakikada geri dönmeli'). 2) Active-Passive (Warm/Hot Standby): Trafik tek bir ana bölgeye (Primary) akar; veriler ikincil bölgeye (Secondary) asenkron replike edilir. Maliyeti düşüktür ancak failover sırasında RPO > 0 ve RTO gecikmesi yaşanır. 3) Multi-Region Active-Active: Her iki bölge de canlı trafik alır. Avantajı: Sıfır RTO (biri çökerse Anycast DNS veya Global Accelerator trafiği anında diğer bölgeye aktarır). Zorluğu: Çift yönlü veritabanı replikasyonu (Multi-Master / CockroachDB / Spanner) ve çakışma çözümü (Conflict Resolution: CRDTs).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Disaster Recovery Frameworks & Metrics: 1) RPO (Recovery Point Objective): The maximum tolerable temporal window of data loss measured in time (e.g. RPO = 5 seconds means mutations older than 5s are safely preserved). 2) RTO (Recovery Time Objective): The maximum permissible duration the system can remain offline before restoration (e.g. RTO = 2 minutes). 3) Active-Passive Topology: Primary region serves 100% of traffic, streaming snapshots and WAL logs to a secondary warm standby. Cost-effective, but failover requires DNS propagation delays and risks data loss on inflight replication buffers. 4) Multi-Region Active-Active: Multiple geographic regions simultaneously ingest live traffic. Traffic fails over seamlessly with zero RTO via Anycast BGP shifts. The engineering hurdle: cross-datacenter multi-master write conflict resolution, solved via Conflict-Free Replicated Data Types (CRDTs) or globally coordinated consensus databases (Google Cloud Spanner, CockroachDB).

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: RPO ve RTO Felaket Kurtarma Hedef Modeli
public record DisasterRecoveryPlan(
    String strategyName,
    int rpoSeconds,
    int rtoSeconds,
    boolean supportsZeroDowntimeFailover
) {
    public void printDrProfile() {
        IO.println("Strateji:        " + strategyName);
        IO.println("Maks Veri Kaybı: " + rpoSeconds + " saniye (RPO)");
        IO.println("Maks Kesinti:    " + rtoSeconds + " saniye (RTO)");
        IO.println("Sıfır Kesinti:   " + supportsZeroDowntimeFailover);
    }
}

void main() {
    var activePassive = new DisasterRecoveryPlan("Active-Passive Hot Standby", 60, 300, false);
    var activeActive = new DisasterRecoveryPlan("Active-Active Multi-Region", 0, 0, true);

    IO.println("--- Active-Passive Profili ---");
    activePassive.printDrProfile();
    IO.println("\n--- Active-Active Profili ---");
    activeActive.printDrProfile();
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta hemen 'Active-Active kurarım' demeyin! Çok bölgeli Active-Active mimari korkunç derecede pahalıdır ve iki bölge arasında ışık hızı gecikmesi yüzünden veri çakışmaları (Write Conflicts) çıkar. Mülakatçıya önce maliyet/fayda analizini yapıp kritiklik seviyesine göre Active-Passive veya Active-Active önereceğinizi belirtin.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never jump immediately to Multi-Region Active-Active without acknowledging costs. Cross-region data egress fees, distributed locking latency, and split-brain risks can multiply operating budgets by 5x. Always balance RPO/RTO business value against cost.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- RPO izin verilen veri kaybı süresini, RTO ise sistemin geri dönme süresini belirtir.
- Active-Passive daha ucuz ve basittir ancak failover sırasında kısa bir kesinti yaşatır.
- Active-Active sıfır kesinti (Zero RTO) sağlar ancak çift yönlü veri çakışması yönetimi gerektirir.

🎯 Key Takeaways for the Interview

- RPO bounds tolerable data loss in seconds; RTO defines maximum acceptable downtime.
- Active-Passive minimizes infrastructure costs while accepting brief failover lag.
- Active-Active delivers instantaneous failover at the cost of cross-region write synchronization.

59

BÖLÜM: MÜLAKAT USTALIĞI, CANLI BEYAZ TAHTA & STAFF+ SEVIYESI | MODULE: MAANG INTERVIEW MASTERY, WHITEBOARDING & STAFF+ PLAYBOOK

Maliyet Optimizasyonu ve Bulut Ekonomisi (FinOps): Egress, Spot ve Tiering

Cloud Economics & FinOps: Egress Costs, Spot Instances & Data Tiering

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram

```
[ BULUT MALİYET OPTİMİZASYON PİRAMİDİ (FINOPS STRATEJİSİ) ]

1. AĞ MALİYETİ (EGRESS)      : İnternete veri çıkarmak ($0.09/GB) ÇOK PAHALIDIR!
                               Çözüm: CDN önbellekleme ve sıkıştırma (Gzip/Brotli/ZSTD)
2. HESAPLAMA (COMPUTE)       : Sürekli On-Demand makine çalıştırmak israftır!
                               Çözüm: Spot/Preemptible makineler (%70-90 indirim!)
3. VERİ SAKLAMA (STORAGE)    : Tüm veriyi pahalı SSD'de tutmak maliyet intiharıdır!
                               Çözüm: Yaşam Döngüsü (Lifecycle Tiering): Hot -> Warm -> Glacier
```

💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette elektrik ve su faturalarını yönettiğinizi düşünün: 1) İnternet Çıkış Ücreti (Egress): Evinize su almak bedavadır ama dışarıya her kova su boşalttığınızda su idaresi sizden 10 TL para alır! Bulut sağlayıcıları da (AWS, Google Cloud) içeri veri alırken para almaz ama dışarıya çıkan her gigabayttan (Egress) servet kazanır. 2) Spot Makineler: Uçaktaki son dakika boş koltukları gibidir; havayolu koltuk boş gitmesin diye %80 indirimle satar! Siz de arka planda video kodlayan sunucuları bu indirimli Spot makinelerden seçersiniz. 3) Arşiv Depolama: 5 yıl önceki faturaları lüks bir gökdelende değil, şehir dışındaki ucuz bir depoda (AWS Glacier) saklamaktır!

🇬🇧 ENGLISH (ELI5)

Think of managing utility bills for a massive industrial complex: 1) Egress Bandwidth: Pumping water into your reservoir is free, but dumping water back into the city pipes incurs a $10 penalty per bucket! Cloud providers (AWS, GCP) charge zero for data ingress, but levy eye-watering tolls on every gigabyte exiting their network (Egress). 2) Spot Instances: Standby standby airline tickets sold at an 80% discount 20 minutes before takeoff. Use Spot nodes for asynchronous video transcoding or batch training! 3) Storage Tiering: Storing 5-year-old tax audits inside expensive downtown prime real estate (NVMe SSD) is financial suicide; move them to a dusty underground warehouse in the desert (AWS Glacier)!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi FinOps Mimarisi: 1) Egress Trafik Tuzağı: Bulutta bölgeler arası (Cross-Region) veya internete çıkan veri en pahalı kalemdir ($0.08 - $0.12 / GB). Çözüm: CDN kullanımıyla origin egress'ini %90 düşürmek, aynı kullanılabilirlik alanında (AZ) kalmak ve gRPC/Protobuf/ZSTD ile veriyi sıkıştırmak. 2) Spot / Preemptible Örnekler: AWS ve GCP'nin boşta duran kapasitesini %70-%90 indirimle sunmasıdır. Sağlayıcı makineyi 2 dakika önceden haber vererek geri alabilir. Durumsuz işçiler (Video Transcoding, ML eğitimi, Batch Job) için kusursuzdur. 3) S3 Yaşam Döngüsü (Lifecycle Tiering): S3 Standard ($0.023/GB) -> S3 Infrequent Access ($0.0125/GB) -> S3 Glacier Flexible ($0.0036/GB) -> S3 Glacier Deep Archive ($0.00099/GB). 90 günden eski loglar Glacier'a atılarak depolama maliyeti 23 kat düşürülür!

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Cloud FinOps Disciplines: 1) The Egress Bandit: Cloud providers charge zero for inbound traffic, but penalize internet and cross-region egress with steep fees ($0.08-$0.12/GB). Solutions: Maximize CDN cache offloading, enforce colocation within identical Availability Zones, and mandate Protobuf with Zstandard compression. 2) Spot / Preemptible Fleet Arbitrage: Cloud vendors auction spare compute at 70%-90% discounts with a 2-minute preemption warning. Ideal for stateless, containerized worker pools (video transcoding, model inference, background queue drainers) orchestrated by Kubernetes Karpenter. 3) Object Storage Lifecycle Tiering: S3 Standard ($0.023/GB) transitions automatically after 30 days to Infrequent Access ($0.0125/GB), then after 90 days to Glacier Flexible ($0.0036/GB), and finally to Glacier Deep Archive ($0.00099/GB). Slashes cold storage expenditure by 95% while retaining compliance guarantees.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: S3 Depolama Katmanlama (Tiering) Maliyet Karşılaştırma Modeli
public class FinOpsStorageCalculator {
    public static double calculateMonthlyCost(double terabytes, String tier) {
        double costPerGb = switch (tier) {
            case "S3_STANDARD"     -> 0.023;
            case "S3_INFREQUENT"   -> 0.0125;
            case "S3_GLACIER"      -> 0.0036;
            case "GLACIER_DEEP"    -> 0.00099;
            default                -> 0.023;
        };
        return (terabytes * 1024) * costPerGb;
    }
}

void main() {
    double tb = 100.0; // 100 Terabayt Veri
    double standardCost = FinOpsStorageCalculator.calculateMonthlyCost(tb, "S3_STANDARD");
    double deepArchiveCost = FinOpsStorageCalculator.calculateMonthlyCost(tb, "GLACIER_DEEP");

    IO.println("100 TB S3 Standard Aylık Maliyet:     $" + String.format("%.2f", standardCost));
    IO.println("100 TB Glacier Deep Aylık Maliyet:    $" + String.format("%.2f", deepArchiveCost));
    IO.println("Aylık Tasarruf Oranı: %" + String.format("%.1f", (1.0 - (deepArchiveCost /
standardCost)) * 100));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Maliyeti hiç düşünmem, her yere en pahalı AWS sunucularını ve 100 replika koyarım' demeyin! Staff ve Principal seviyesindeki mühendisler mimariyi sadece performans için değil, şirket bütçesini ve bulut maliyetlerini (Unit Economics) korumak için tasarlar.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Dismissing cloud costs in an interview as 'someone else's problem' guarantees an L4 downgrade. Staff and Principal engineers distinguish themselves by anchoring technical architectures to cloud unit economics and FinOps efficiency.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Egress trafiği bulut faturalarının en tehlikeli ve gizli maliyet kalemidir.
- Spot makineler durumsuz iş yüklerinde maliyetleri %80'e varan oranda düşürür.
- Otomatik S3 Lifecycle kuralları soğuk verileri Glacier'a taşıyarak %95 tasarruf sağlar.

🎯 Key Takeaways for the Interview

- Network egress is the single most expensive hidden trap in cloud architecture.
- Spot compute instances deliver up to 80% cost savings for fault-tolerant background workloads.
- S3 lifecycle tiering transitions cold historical logs into deep archive, saving 95%.

60

BÖLÜM: MÜLAKAT USTALIĞI, CANLI BEYAZ TAHTA & STAFF+ SEVIYESI | MODULE: MAANG INTERVIEW MASTERY, WHITEBOARDING & STAFF+ PLAYBOOK

Büyük Kurumsal Final: Staff+ Zihniyeti, Trade-Off Savunması ve Mülakat Altın Rehberi

Grand Enterprise Finale: The Staff+ Mindset, Trade-Off Mastery & Golden Rubric

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Tebrikler! 60 derslik muazzam bir yolculuğun zirvesine ulaştınız. Artık sistem tasarımına sadece 'kutular çizip aralarına oklar koyan' bir yazılımcı olarak bakmıyorsunuz; devasa bulut filolarını yöneten, kriz anında soğukkanlılıkla doğru kararı veren usta bir sistem mimarısınız! Bir mimarın ustalığı 'en karmaşık sistemi tasarlamak' değil, 'işi çözecek en sade sistemi kurup onun sınırlarını ve bedellerini (Trade-off) savunabilmektir'. Bu son derste öğrendiğimiz tüm ilkeleri zihnizdeki taçla birleştiriyoruz!

🇬🇧 ENGLISH (ELI5)

Congratulations! You have conquered all 60 lessons and reached the summit of distributed systems mastery. You are no longer merely someone who connects software boxes with lines on a whiteboard; you are a battle-tested Staff Systems Architect! The hallmark of true engineering seniority is never building the most convoluted system imaginable; it is designing the simplest system that completely solves the problem while lucidly defending its compromises (Trade-offs). In this grand finale, we synthesize all 60 lessons into your career playbook!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Mülakat Komitelerinin Staff+ (L6/L7) Değerlendirme Kriterleri: 1) Bedel Savunması (The 'No Free Lunch' Axiom): Mimaride mükemmel bir çözüm yoktur; her kararın bir bedeli vardır (Örn: 'SQL seçtim çünkü güçlü ACID gerekiyor; bedeli: Sharding karmaşıklığı'). Bir aday 'Benim sistemimin hiçbir eksiği yok' diyorsa elenir. 2) Day-2 Operasyonları: Sistem sadece mutlu yolda değil; sunucu yandığında, ağ bölündüğünde, faturalar şiştiğinde nasıl davranıyor? (Observability, SLOs, Circuit Breaker, FinOps). 3) İletişim ve Liderlik: Mülakatçıyı dinleme, varsayımları seslendirme, karmaşık fikirleri sade benzetmelerle aktarma yeteneği. Bu 60 derslik müfredat, sizi dünyanın en seçkin teknoloji şirketlerinin beyaz tahta mülakatlarına tam donanımlı hazırlamıştır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Staff+ (L6/L7) Hiring Committee Evaluation Rubric: 1) Trade-Off Fluency (The 'No Free Lunch' Axiom): Engineering is the science of managed compromises. (e.g. 'I chose Cassandra to absorb 1M writes/sec; the explicit compromise is that range scans require multi-partition scatter-gather queries'). Claiming a design has zero drawbacks reveals junior naivety. 2) Day-2 Operational Realism: Mature designs anticipate catastrophic failure: cascading retry storms, regional blackouts, disk full events, and billing alarms. 3) Executive Technical Communication: Driving the whiteboard collaboratively, articulating invariants clearly, and remaining coachable under interrogation. Mastering these 60 lessons positions you at the absolute vanguard of global distributed systems engineering.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Staff+ Mühendislik Karar Matrisi ve Trade-Off Değerlendiricisi
public class StaffDecisionMatrix {
    public record ArchitecturalChoice(
        String technology,
        String chosenBenefit,
        String acceptedDrawback,
        String mitigationStrategy
    ) {}

    public static void main(String[] args) {
        var decision = new ArchitecturalChoice(
            "Apache Kafka Event Streaming",
            "Milyonlarca olayı saniyede kayıpsız yutma ve geriye dönük oynatma (Replay)",
            "Nihai tutarlılık (Eventual Consistency) ve tüketici tarafında mükerrerlik riski",
            "Tüketicilerde Idempotency Key ve Dead Letter Queue (DLQ) mekanizması kurmak"
        );

        IO.println("=== STAFF+ MİMARİ KARAR SAVUNMASI ===");
        IO.println("Seçilen Teknoloji: " + decision.technology());
        IO.println("Kazanç (Benefit):  " + decision.chosenBenefit());
        IO.println("Kabul Edilen Bedel:" + decision.acceptedDrawback());
        IO.println("Emniyet Sibobu:    " + decision.mitigationStrategy());
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatın sonunda mülakatçı 'Bana sormak istediğin bir soru var mı?' dediğinde ASLA 'Hayır, her şey netti' deyip susmayın! Staff seviyedeki bir mühendis şirket içi mimari kültür, teknik borçlar veya ölçeklenme zorlukları hakkında derinlikli ve zekice sorular sorar.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never close an interview with 'No questions, everything was clear'. A Staff+ candidate leverages the final 5 minutes to interrogate the interviewer about company-specific distributed consensus challenges, technical debt, or multi-region failover drills.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Staff+ mühendisin en büyük gücü, mimari kararların getirdiği bedelleri (Trade-Offs) açıkça bilmesidir.
- En iyi sistem en karmaşık olan değil; gereksinimi en sade ve dayanıklı karşılayan sistemdir.
- 60 derslik bu devasa müfredatı tamamlayarak MAANG mülakatlarında zirveye hazırsınız!

🎯 Key Takeaways for the Interview

- Staff engineers excel at explicitly identifying and defending architectural trade-offs.
- The superior architecture is not the most complex, but the simplest resilient design.
- You have conquered all 60 lessons and earned elite mastery in distributed system design!

🎓 TEBRİKLER! / CONGRATULATIONS!

60 Derslik MAANG Sistem Tasarımı Yolculuğunu Başarıyla Tamamladınız!

You have successfully conquered all 60 lessons of the MAANG System Design Masterclass!

Architecture Schematics • Java 25 Virtual Threads • Distributed Consensus • Staff+ Trade-Offs